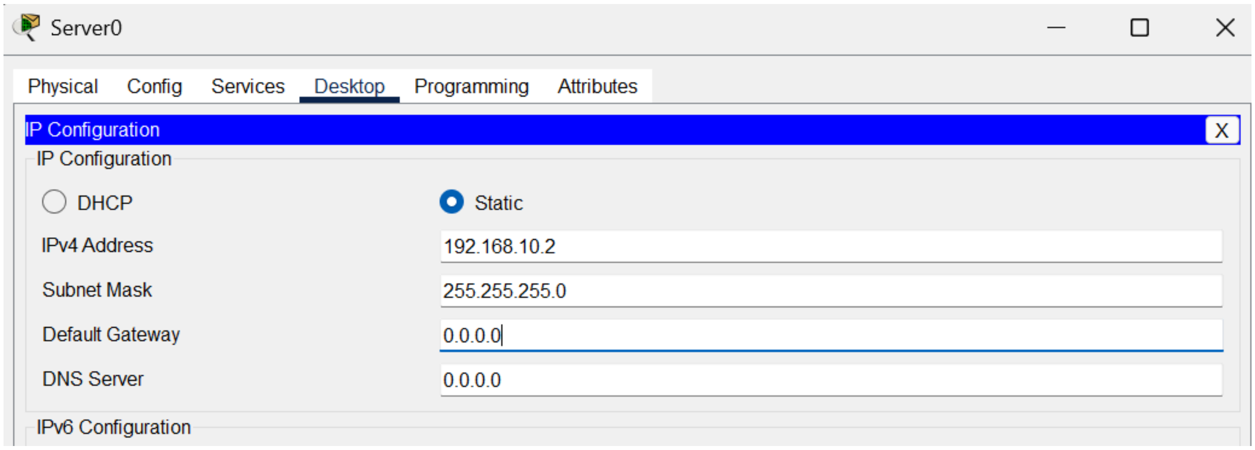
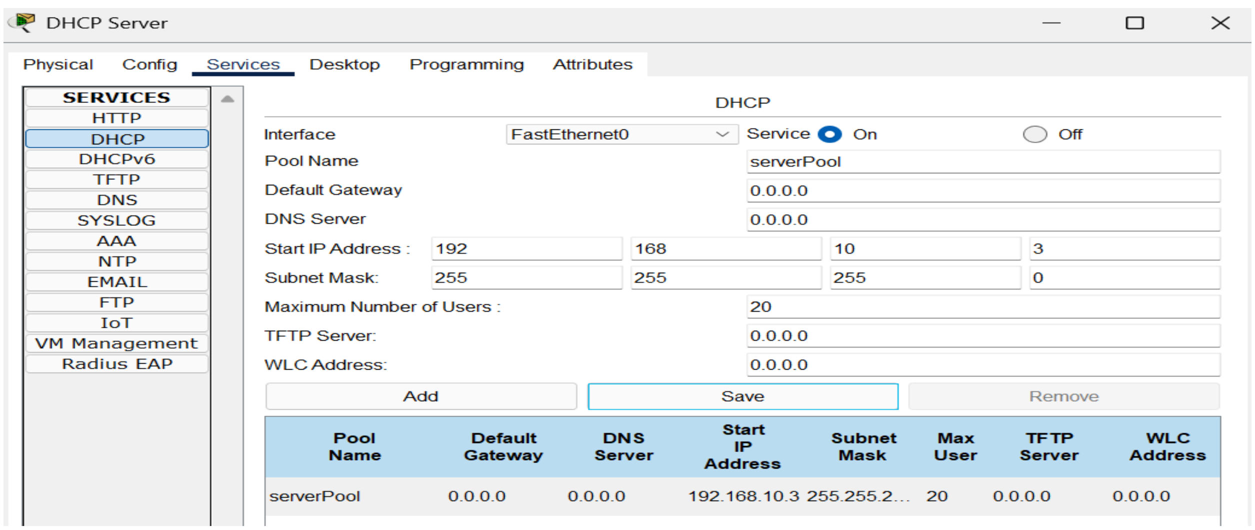
<u>CN LAB MANUAL</u>

DHCP Configuration

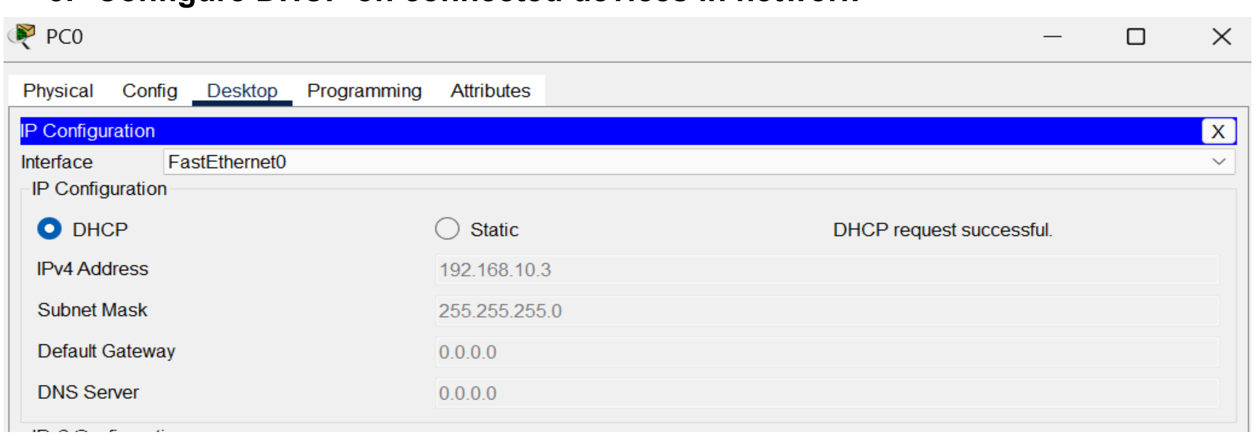
1. Assign a static IP to the DHCP server



2. Configure DHCP service by updating serverPool

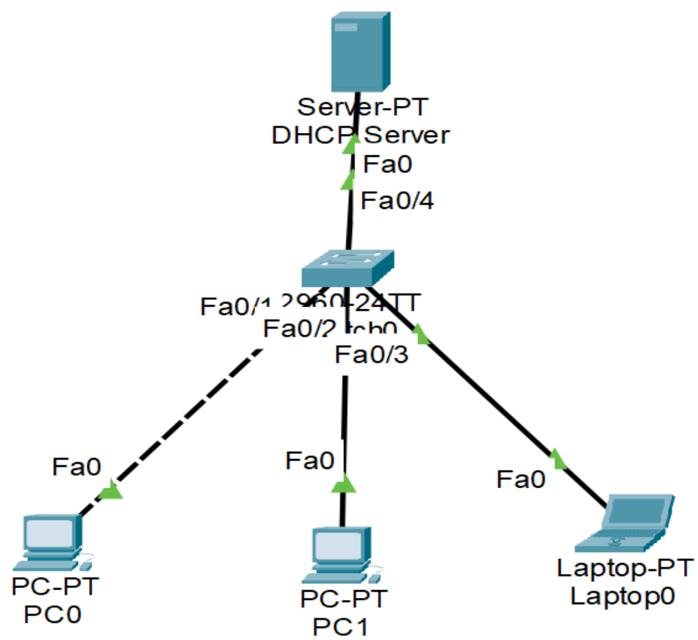


3. Configure DHCP on connected devices in network



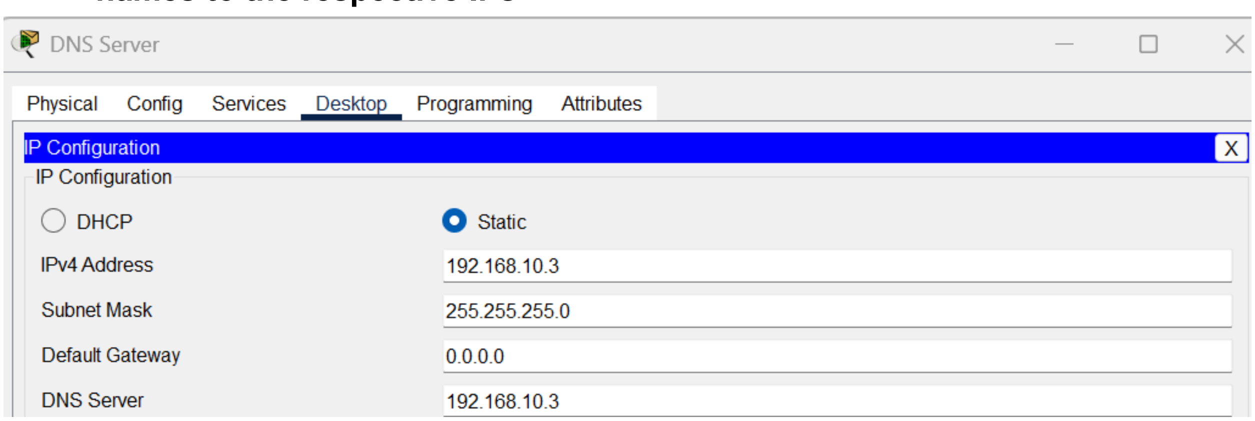
NOTE: DNS server and gateway can also be configured by updating those fields in serverPool. Additionally, separate DHCPS are to be configured for separate networks across routers.

Topology Example

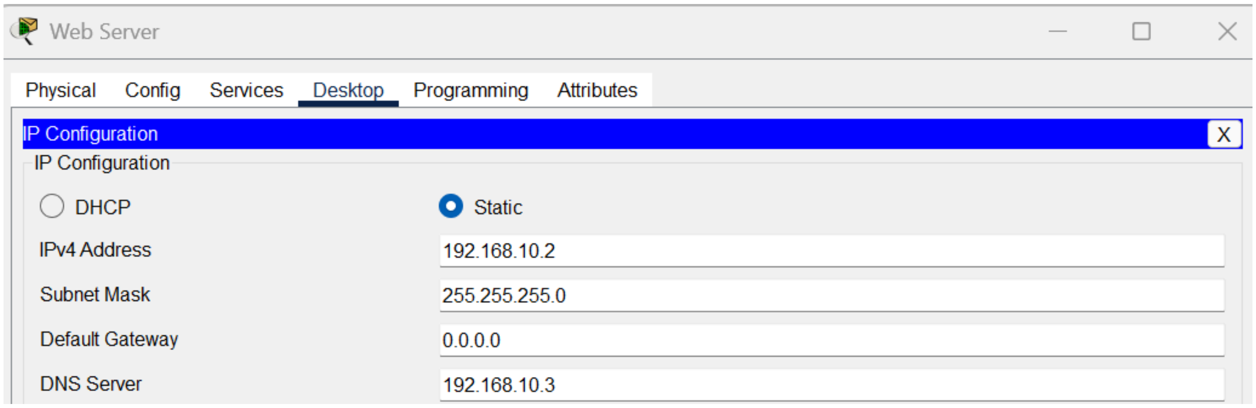


DNS Configuration

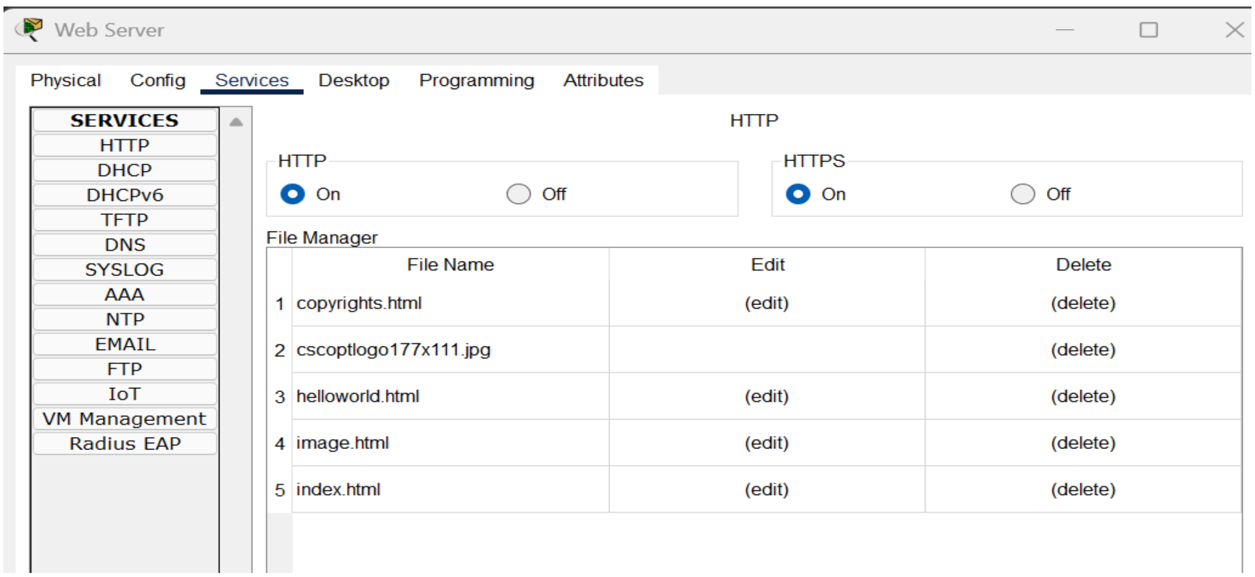
1. **Set up a static IP for the designated DNS server for resolving the domain names to the respective IPs**



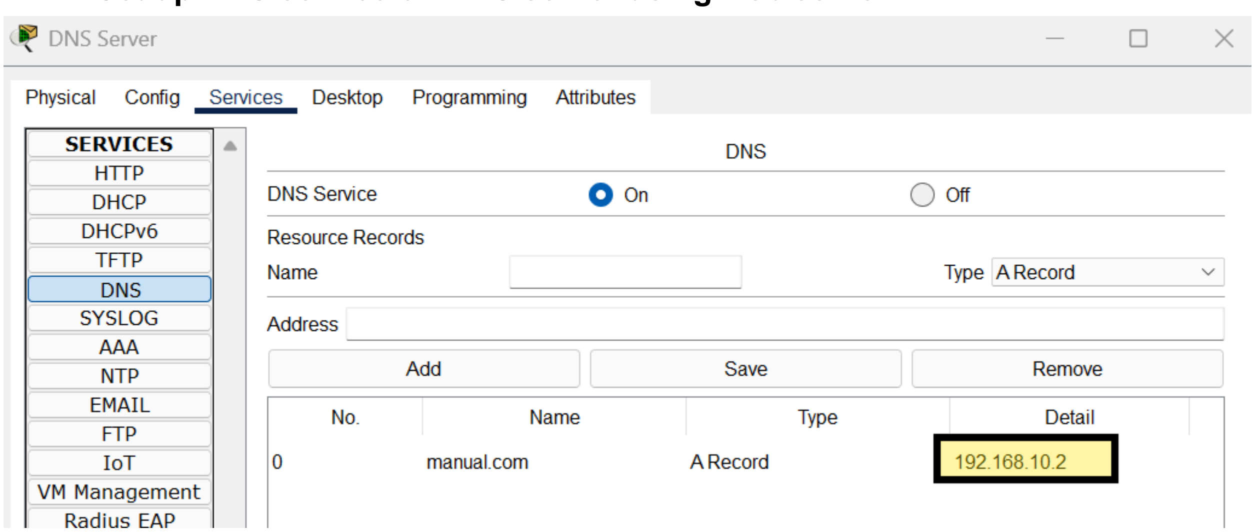
2. **Set up a static IP for the designated web server that the DNS server will be referencing (using a separate web server)**



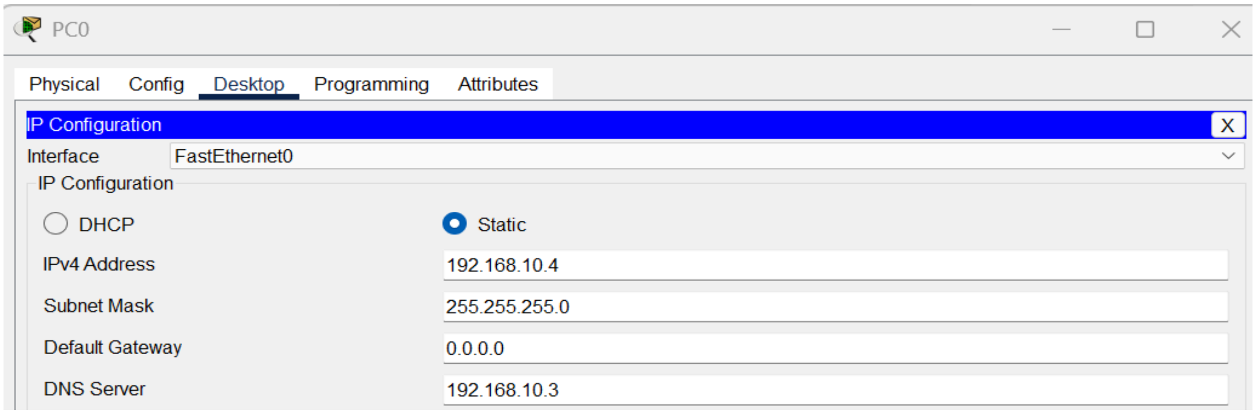
3. **Turn on HTTP service on the web server (on by default), edit web pages as needed.**

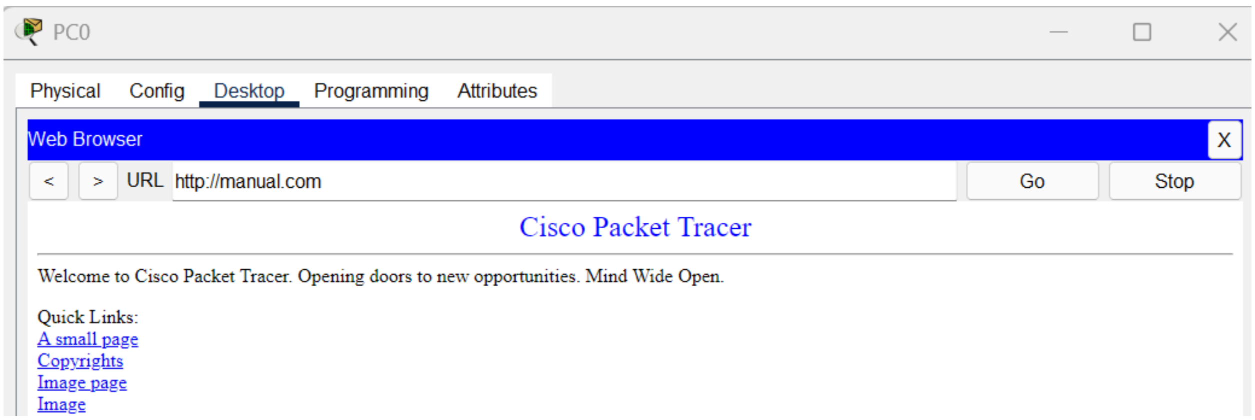


4. **Set up DNS service on DNS server using web server IP**



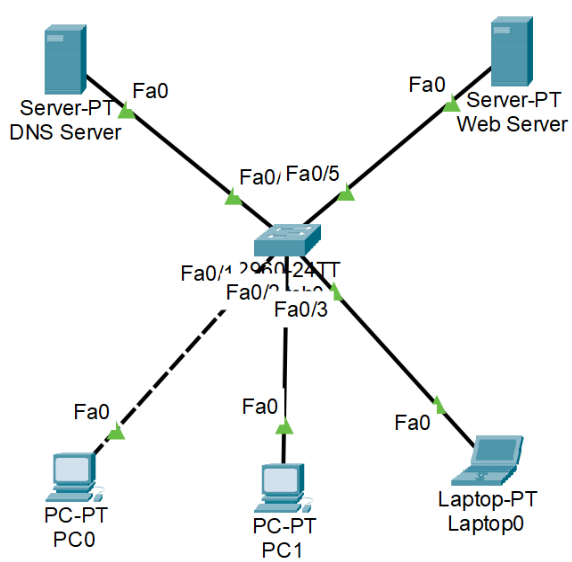
5. **Access DNS service using devices in network by configuring DNS server in IP configuration and opening web browser for resolving domain names**





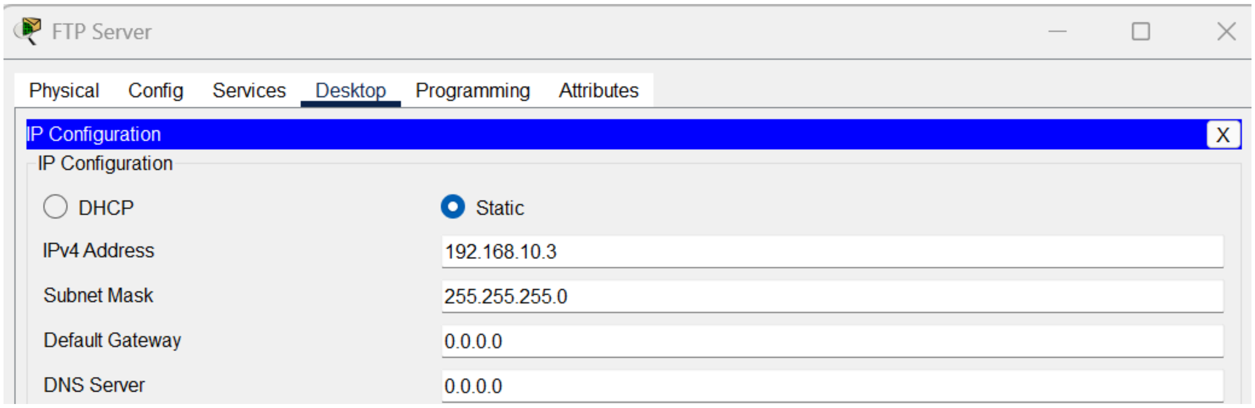
NOTE: DNS server and Web server can both be configured on the same singular server. (Sir focused heavily on separate servers in lab 6 revision)

Topology Example

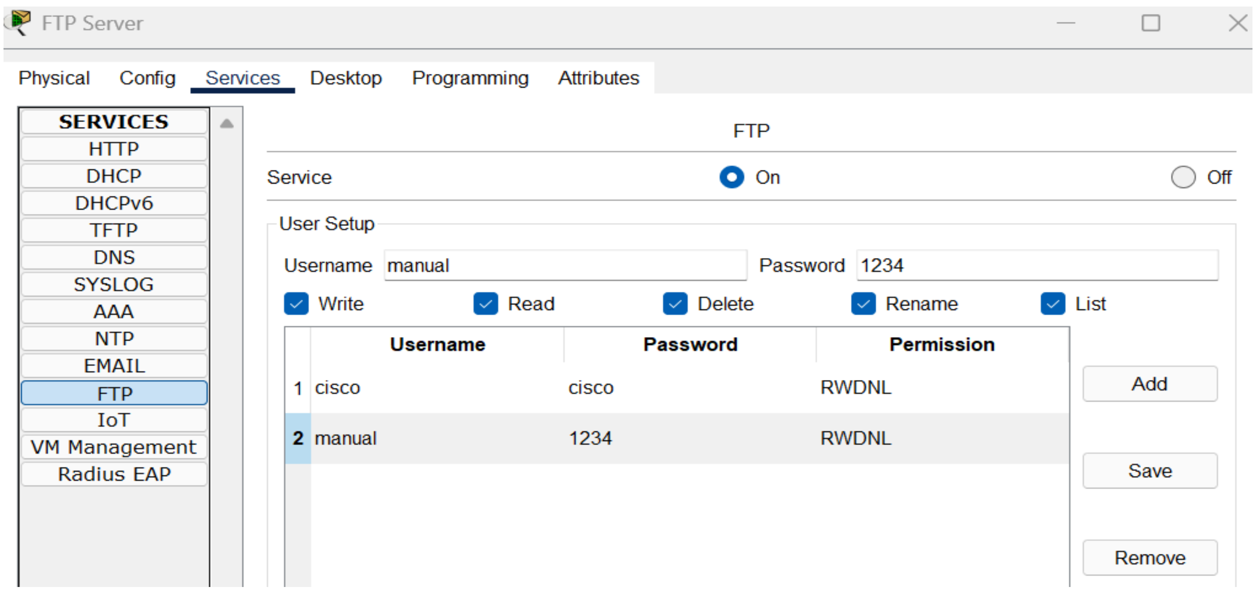


FTP Configuration

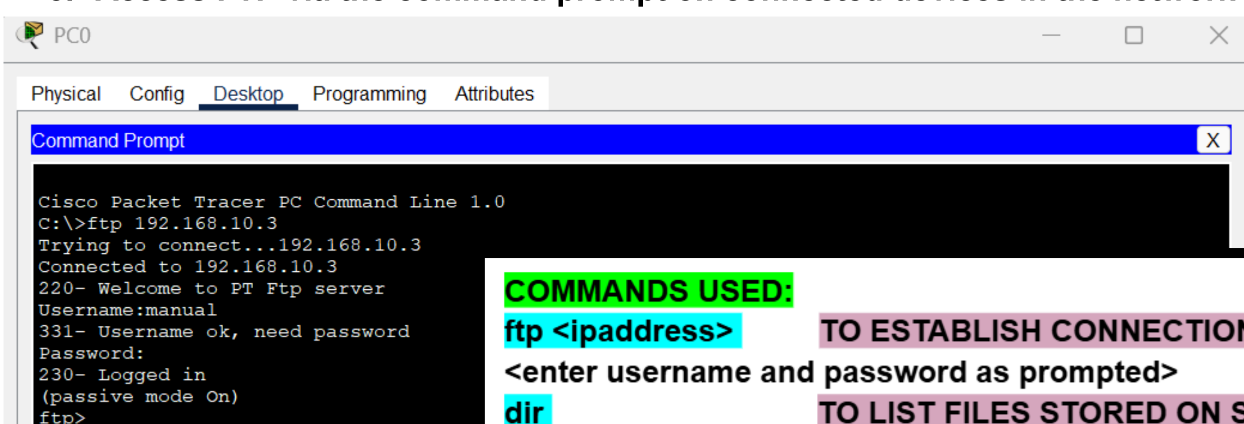
1. **Set up a static IP for the designated FTP server**



2. **Set up FTP service on the designated FTP server**



3. **Access FTP via the command prompt on connected devices in the network**



COMMANDS USED:

ftp <ipaddress> **TO ESTABLISH CONNECTION**

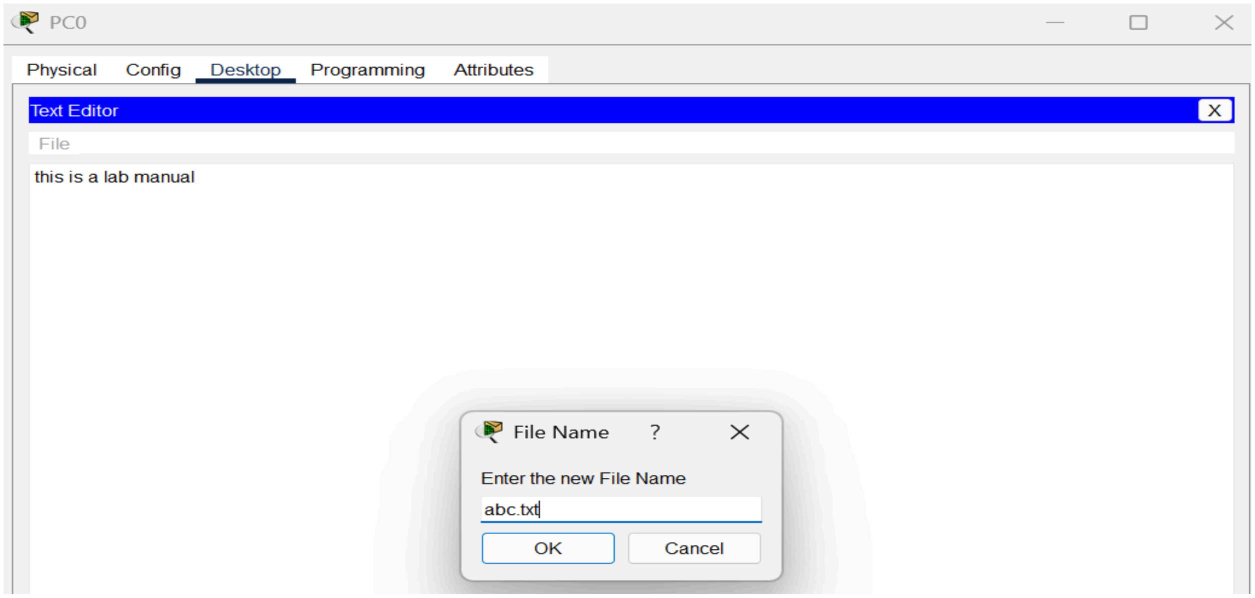
<enter username and password as prompted>

dir **TO LIST FILES STORED ON SERVER**

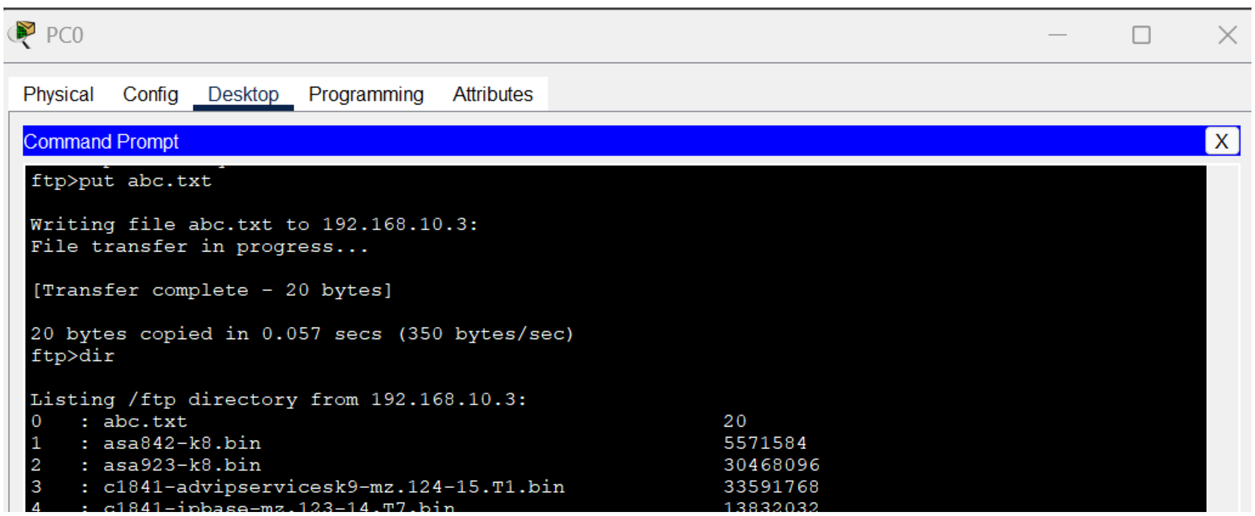
put <filename> **TO UPLOAD FILE ON SERVER**

get <filename> **TO DOWNLOAD FILE FROM SERVER**

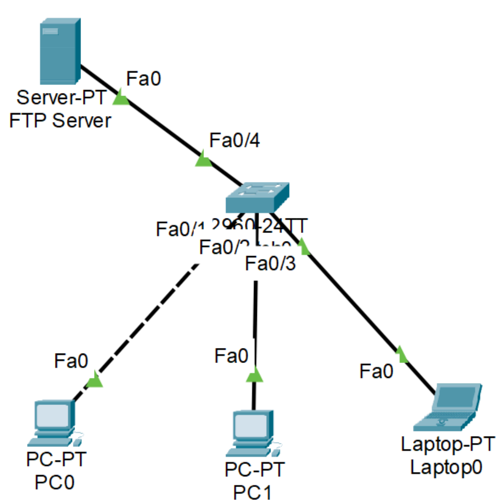
4. **Create a file using text editor to upload on to server**



5. **Upload file onto the server**

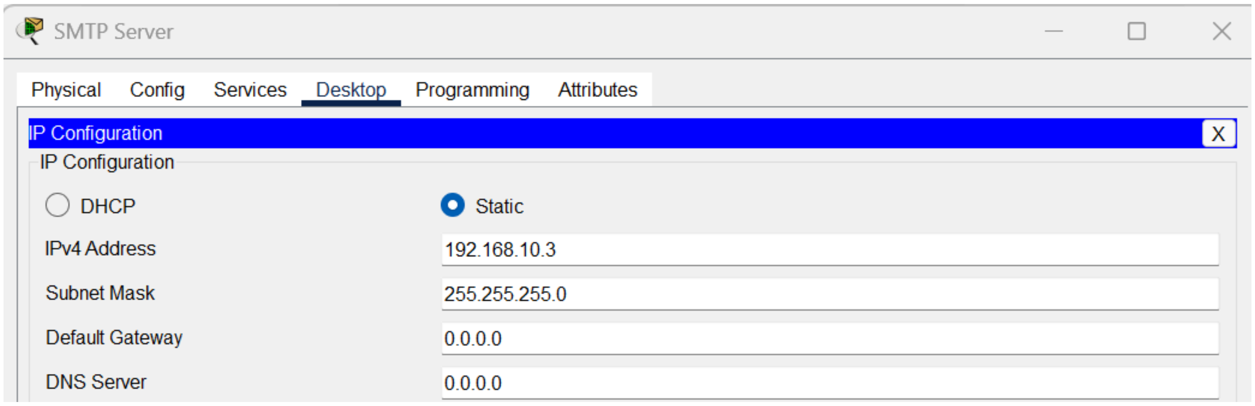


Topology Example

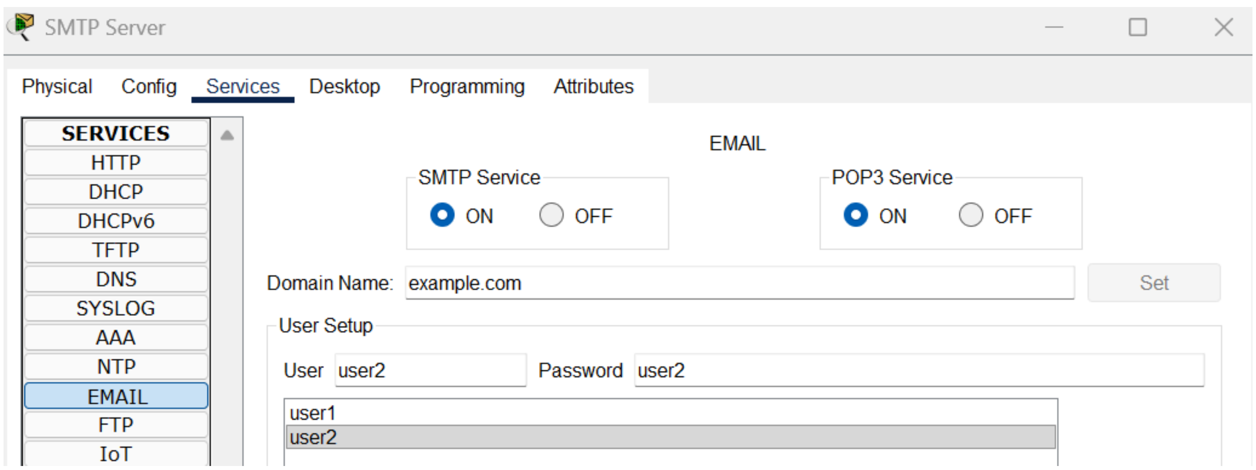


SMTP Configuration

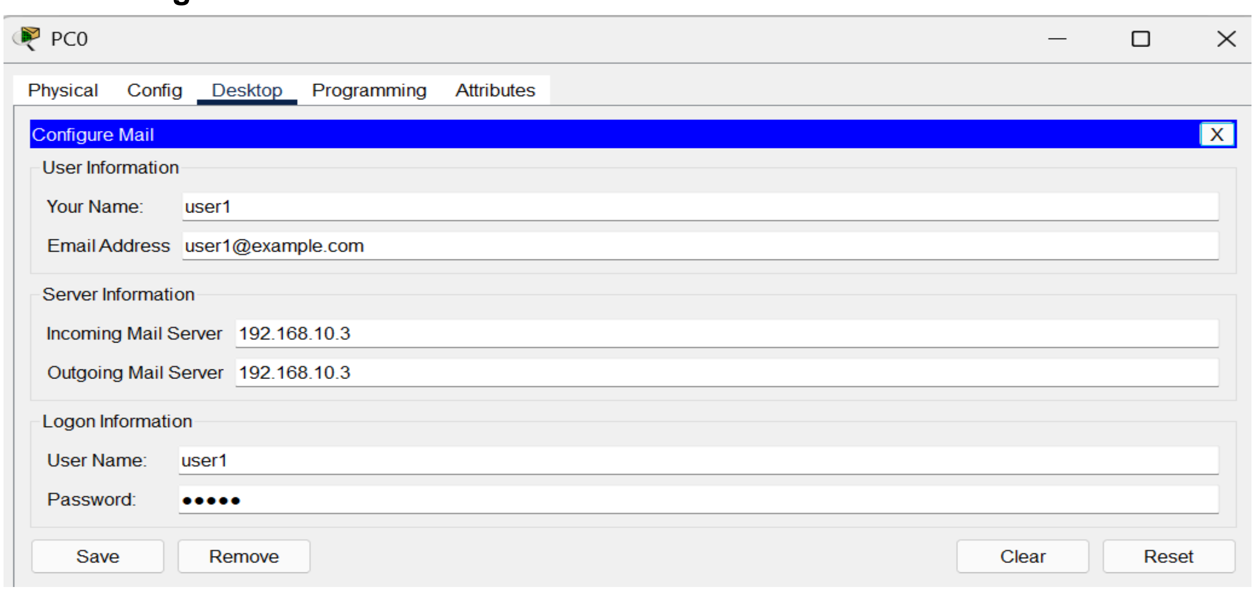
1. **Set up a static IP for the designated SMTP server**



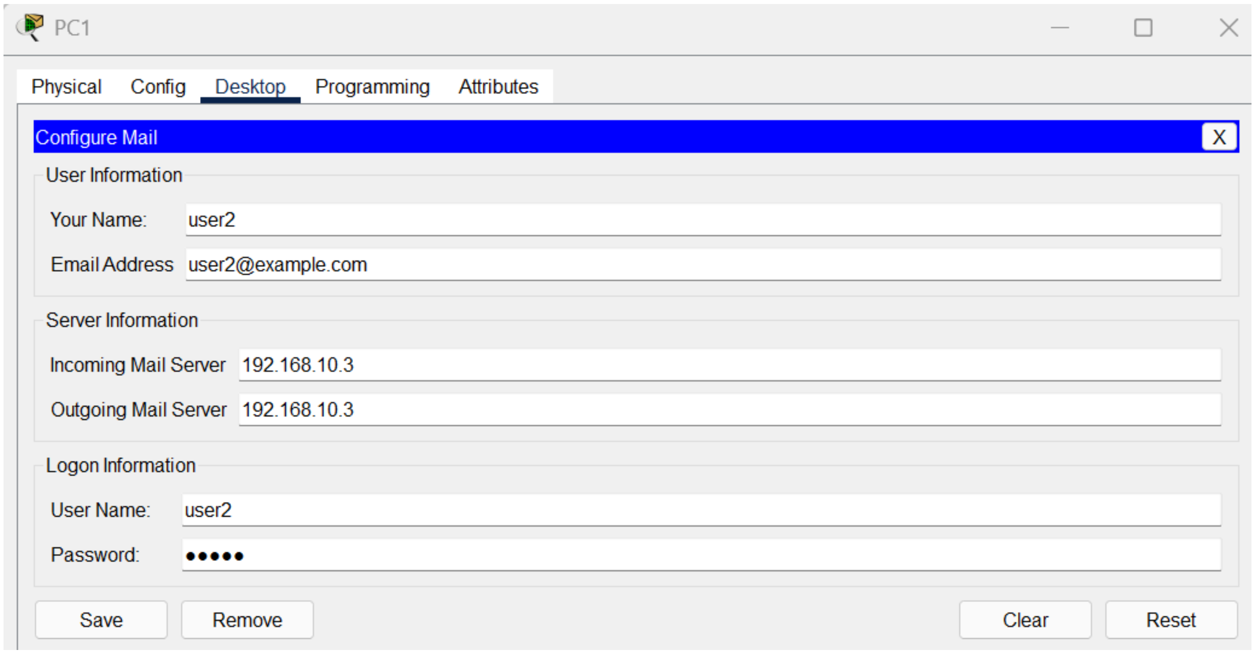
2. **Set up a SMTP service on the designated SMTP server and add users**



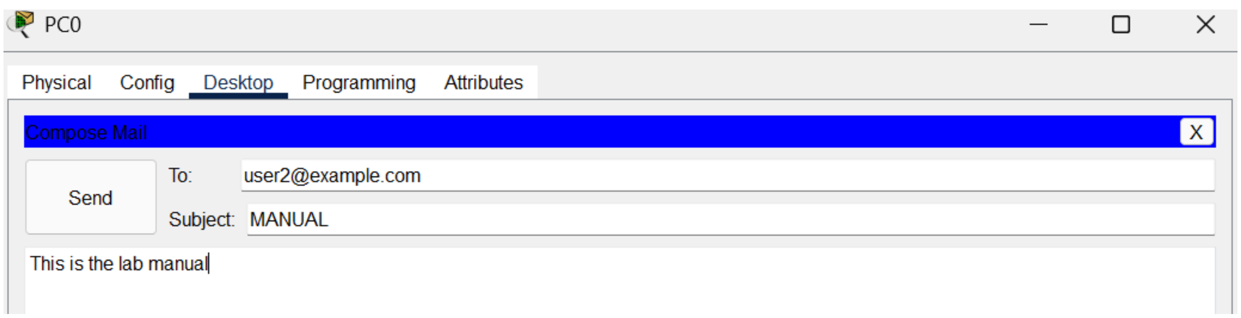
3. **Access SMTP on sender device on network using email interface and configure mail**

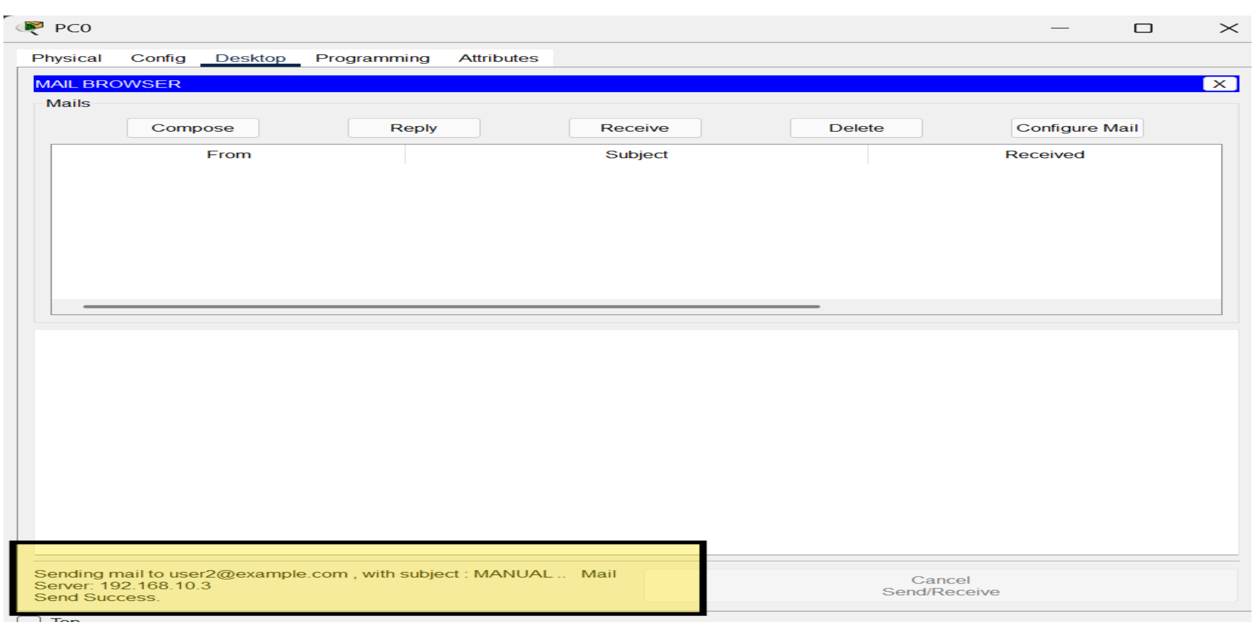


4. **Access SMTP on receiver device on network using email interface and configure mail**

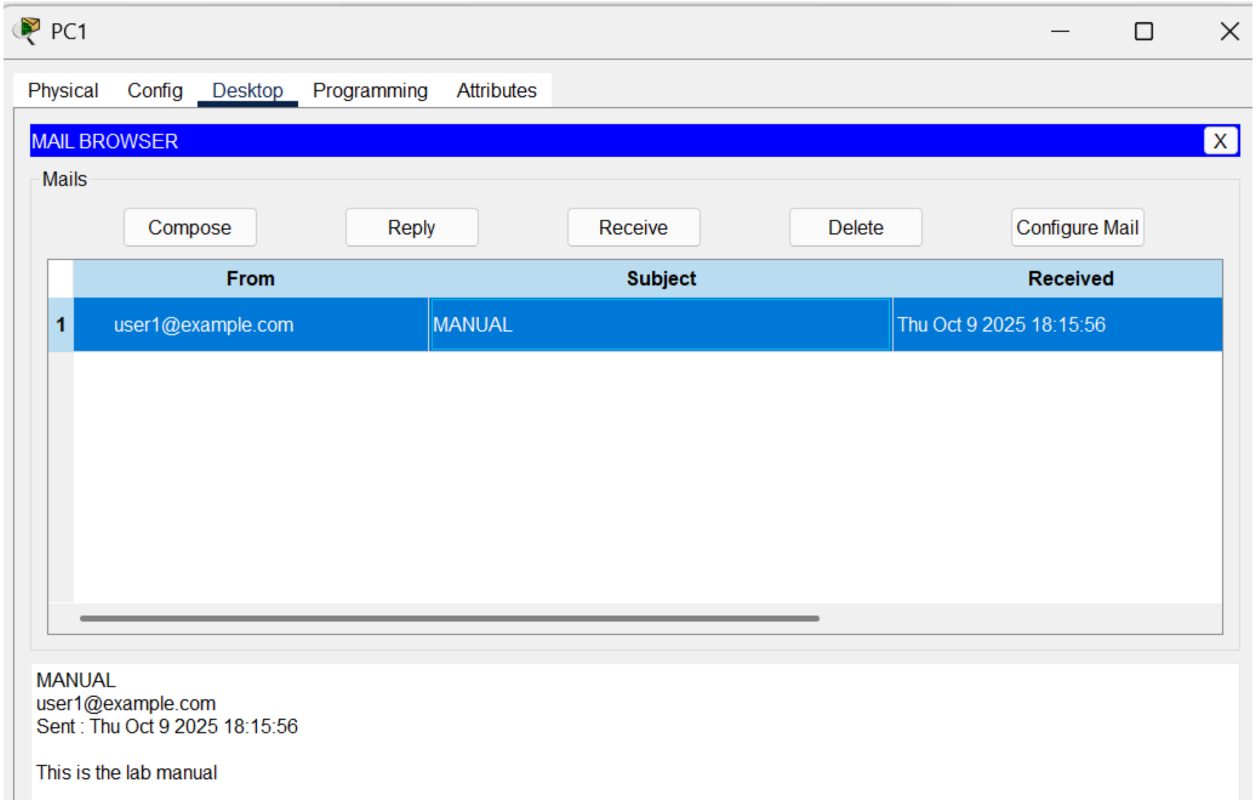


5. **Compose mail on sender device and click on send**

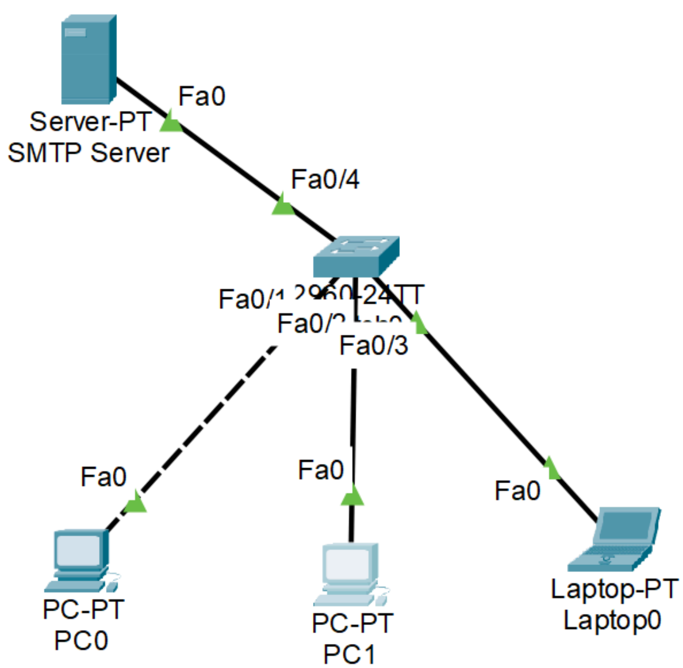




6. **Access mail on receiver side and click receive**

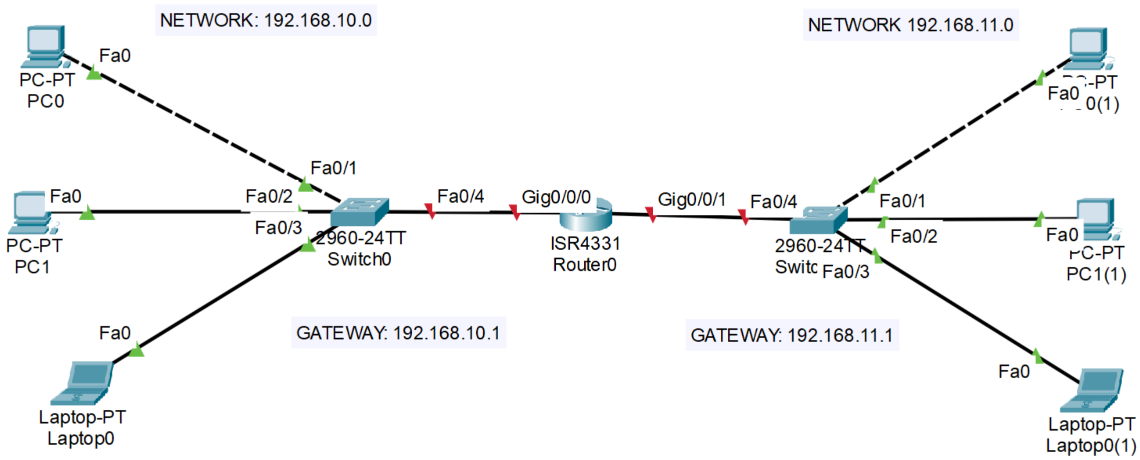


Topology Example

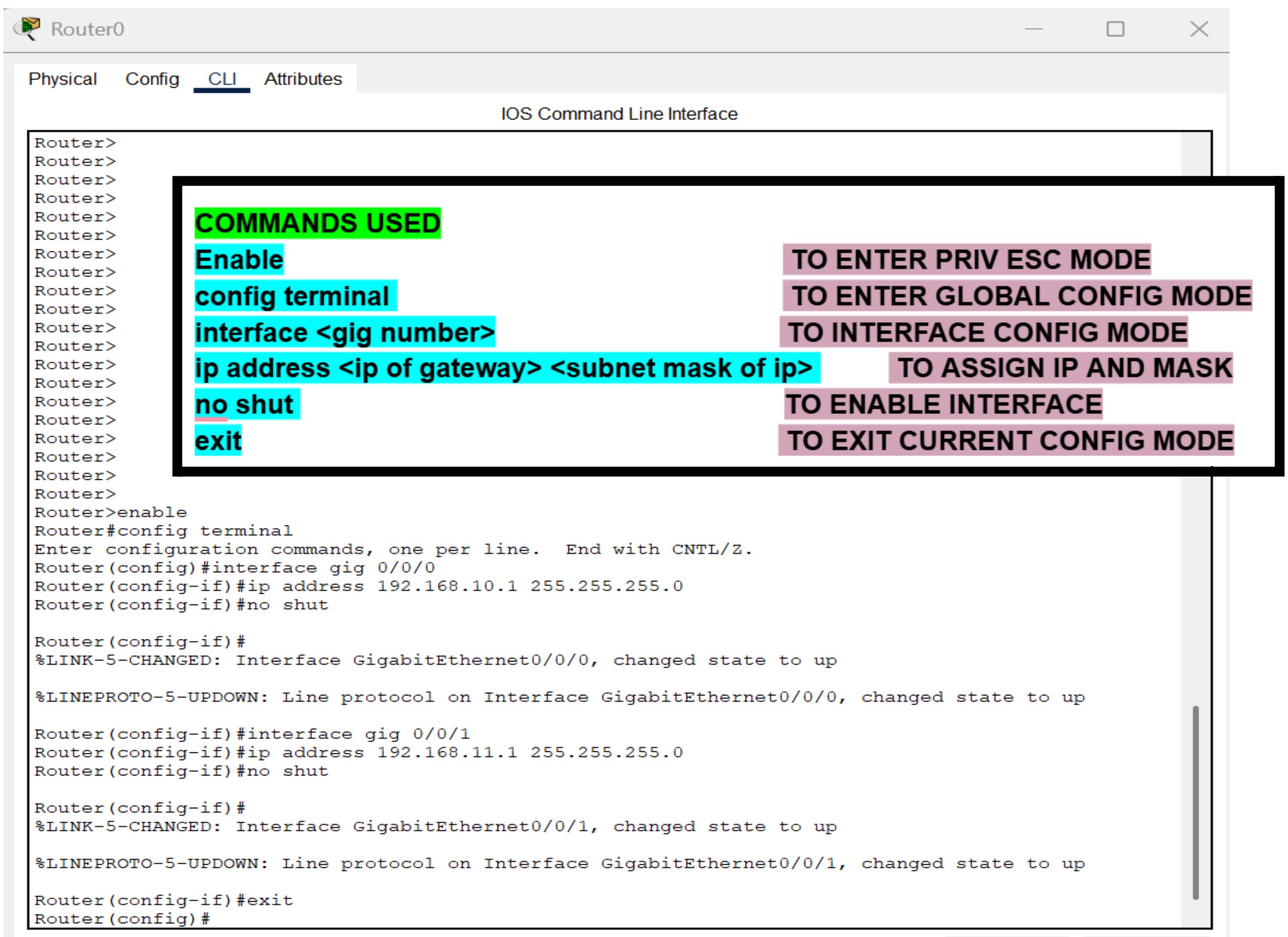


Router Configuration

1. **Set up networks on each side of the router, either using static IPs or using the DHCP service (make sure to set up respective gateways for the devices too)**

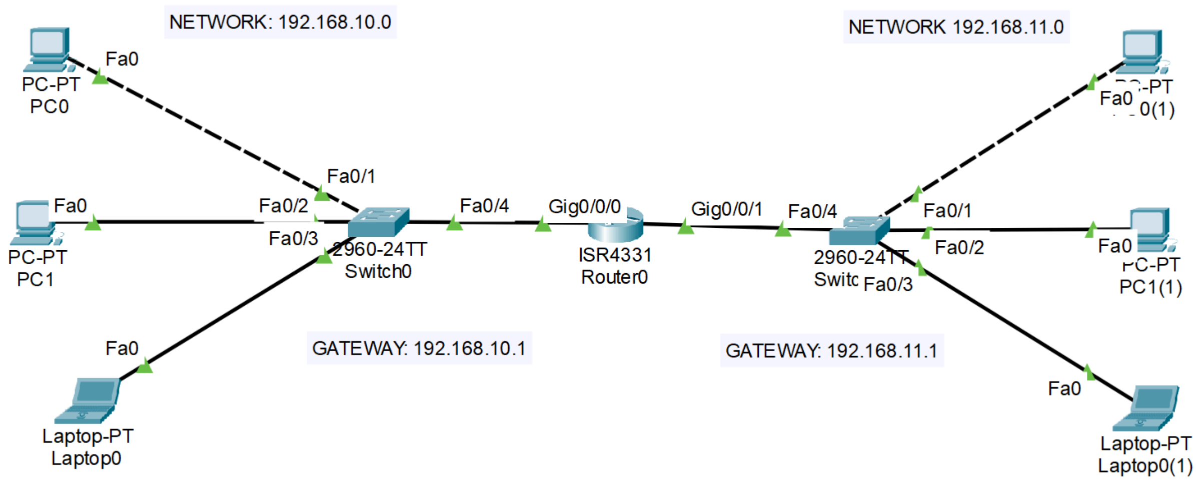


2. **Configure router using CLI (no marks for GUI)**




```
Router>
Router>
Router>
Router>
Router>
Router>
Router>
Router>
Router>
Router>
Router>
Router>
Router>
Router>
Router>
Router>
Router>
Router>
Router>
Router>
Router>enable
Router#config terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#interface gig 0/0/0
Router(config-if)#ip address 192.168.10.1 255.255.255.0
Router(config-if)#no shut

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/0, changed state to up

Router(config-if)#interface gig 0/0/1
Router(config-if)#ip address 192.168.11.1 255.255.255.0
Router(config-if)#no shut

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0/1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/1, changed state to up

Router(config-if)#exit
Router(config)#
```

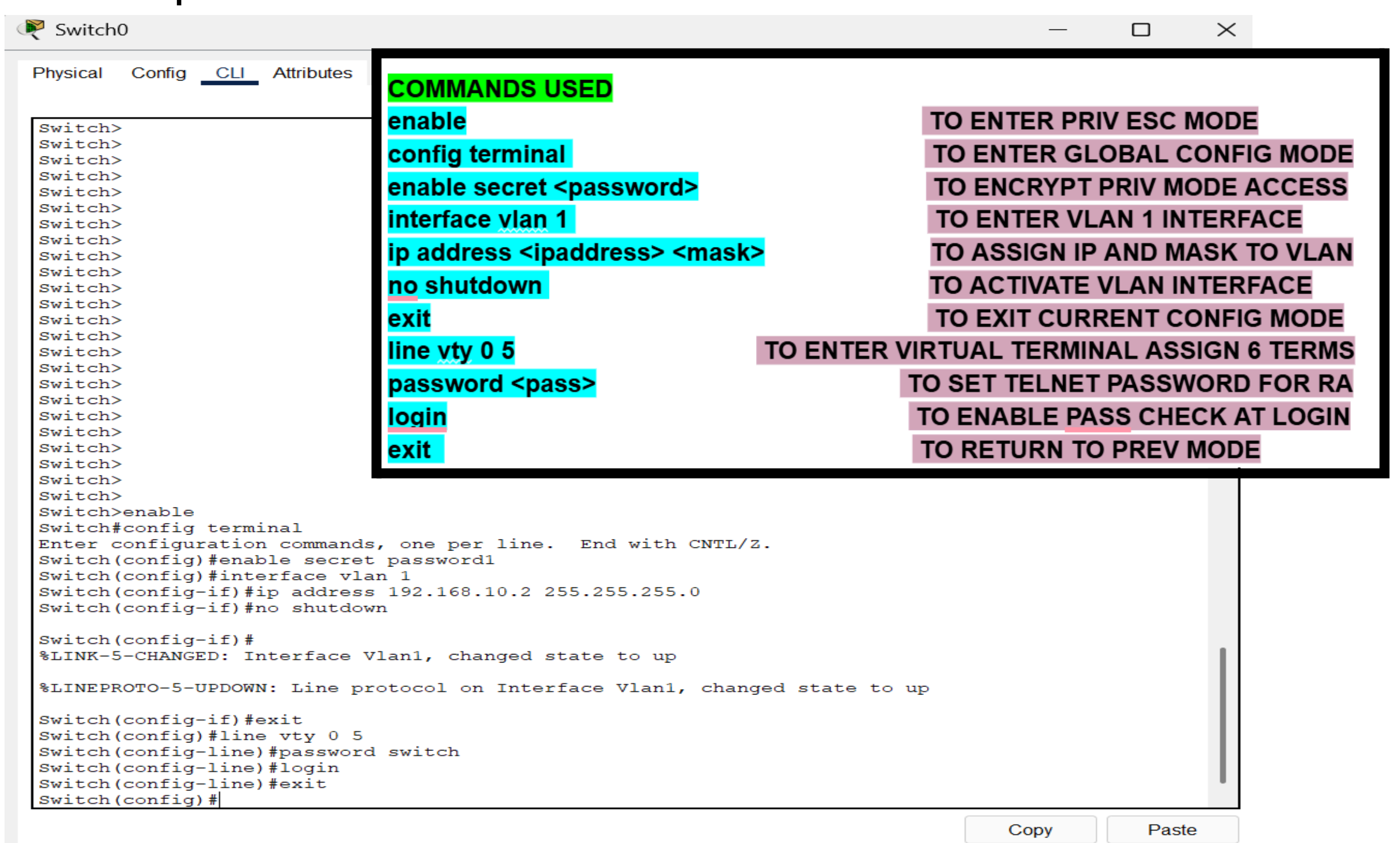
Topology Example



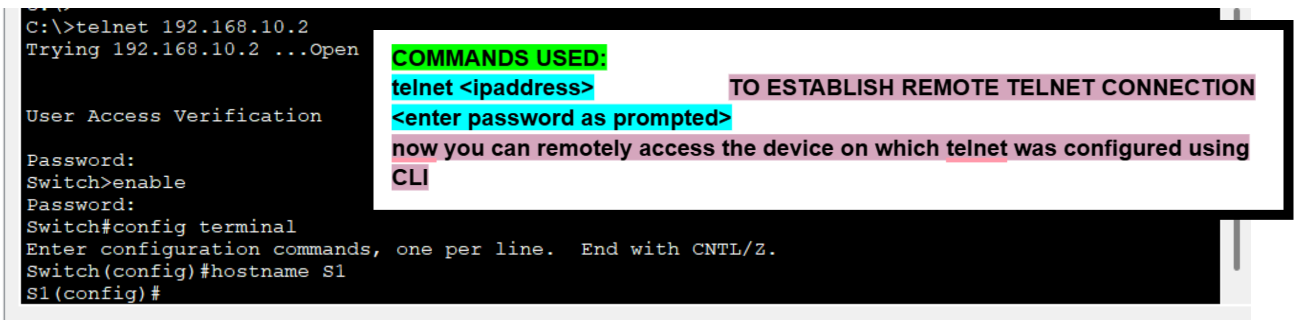
Telnet Configuration

NOTE: Telnet is a remote access protocol to connect to network devices and manage them via CLI, which can be configured on switches and routers

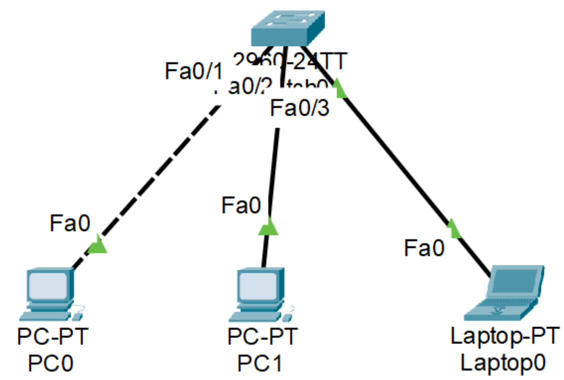
1. Set up telnet session on switch via CLI



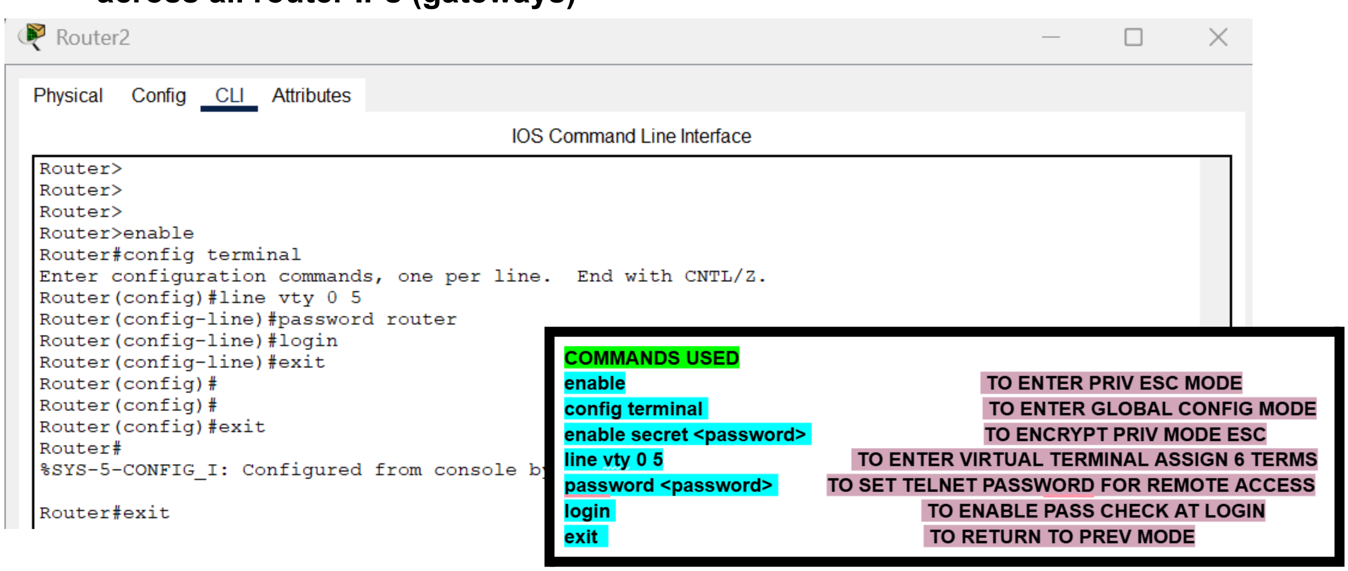
2. **Access the telnet device remotely using any reachable device on network**



Topology Example



1. **To configure telnet on router via CLI, just add virtual terminals since routers already have IP addresses, unlike switches, which enables telnet across all router IPs (gateways)**



2. **Access router via telnet using any reachable device on network**

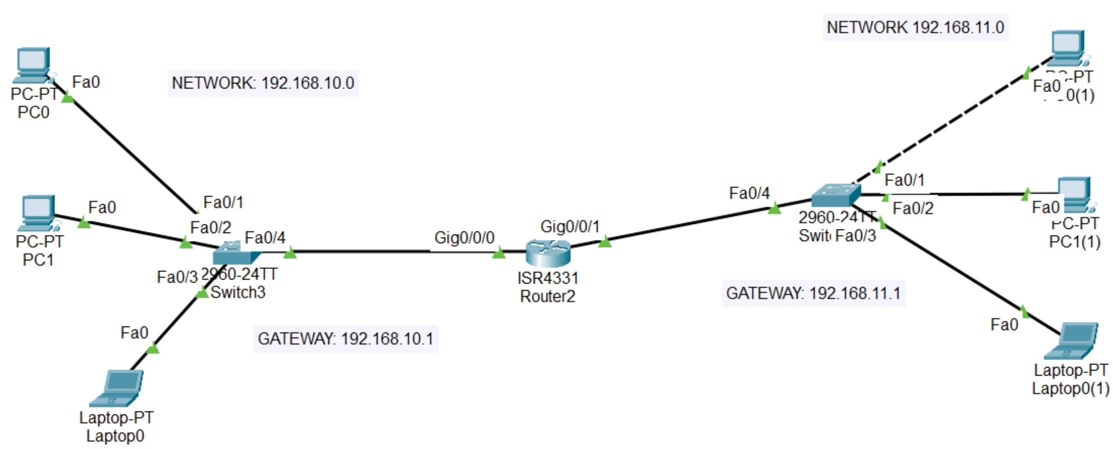
```
C:\>telnet 192.168.10.1
Trying 192.168.10.1 ...Open


User Access Verification

Password:
Router>
```

COMMANDS USED:

telnet <ipaddress>	**TO ESTABLISH REMOTE TELNET CONNECTION**
<enter password as prompted>	

now you can remotely access the device on which telnet was configured using CLI

Topology Example



SSH Configuration

NOTE: SSH is a secure remote access protocol to connect to network devices and manage them via CLI, which can be configured on switches and routers

1. **Set up SSH connection on switch interface**

COMMANDS USED

enable	**TO ENTER PRIV ESC MODE**
config terminal	**TO ENTER GLOBAL CONFIG MODE**
hostname <name>	**SET DEVICE NAME**
ip domain-name <domain.com>	**DEFINE DOMAIN NAME**
username admin privilege 15 secret admin123	**CREATE ADMIN USER**
enable secret <password>	**TO ENCRYPT PRIV MODE ACCESS**
crypto key generate rsa	**GENERATE SSH KEYS**
interface vlan 1	**TO ENTER VLAN 1 INTERFACE**
ip address <ipaddress> <mask>	**TO ASSIGN IP AND MASK TO VLAN**
no shutdown	**TO ACTIVATE VLAN INTERFACE**
exit	**TO EXIT CURRENT CONFIG MODE**
line vty 0 5	**TO ENTER VIRTUAL TERMINAL ASSIGN 6 TERMS**
login local	**TO ENABLE PASS CHECK AT LOGIN**
transport input ssh	**ENABLE SSH ACCESS**
exit	**TO RETURN TO PREV MODE**

Switch2

Physical Config CLI Attributes

IOS Command Line Interface

```
Switch>
Switch>
Switch>
Switch>
Switch>
Switch>
Switch>
Switch>
Switch>
Switch>
Switch>
Switch>enable
Switch#config terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Switch(config)#hostname S1
S1(config)#ip domain-name example.com
S1(config)#username admin privilege 15 secret admin123
S1(config)#enable secret password1
S1(config)#crypto key generate rsa
The name for the keys will be: S1.example.com
Choose the size of the key modulus in the range of 360 to 4096 for your
  General Purpose Keys. Choosing a key modulus greater than 512 may take
  a few minutes.

How many bits in the modulus [512]: 1024
% Generating 1024 bit RSA keys, keys will be non-exportable...[OK]

S1(config)#interface vlan 1
*Mar 1 0:2:12.261: %SSH-5-ENABLED: SSH 1.99 has been enabled
S1(config-if)#ip address 192.168.10.1 255.255.255.0
S1(config-if)#no shutdown

S1(config-if)#
%LINK-5-CHANGED: Interface Vlan1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface Vlan1, changed state to up

S1(config-if)#exit
S1(config)#line vty 0 5
S1(config-line)#login local
S1(config-line)#transport input ssh
S1(config-line)#exit
S1(config)#
```

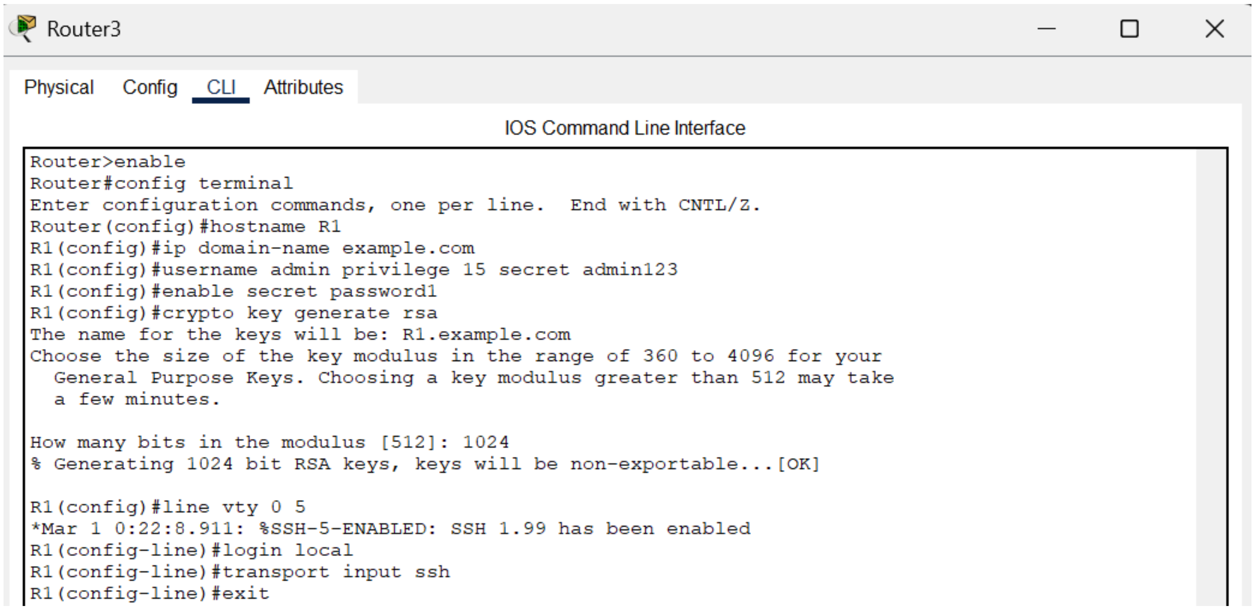
Copy Paste

2. Connect via ssh to switch using any reachable device on network

```
C:\>ssh -l admin 192.168.10.1

Password:


S1#config terminal
Enter configuration commands, one per line.  End with CNTL/Z.
S1(config)#
```

3. To set up SSH connection on router interface

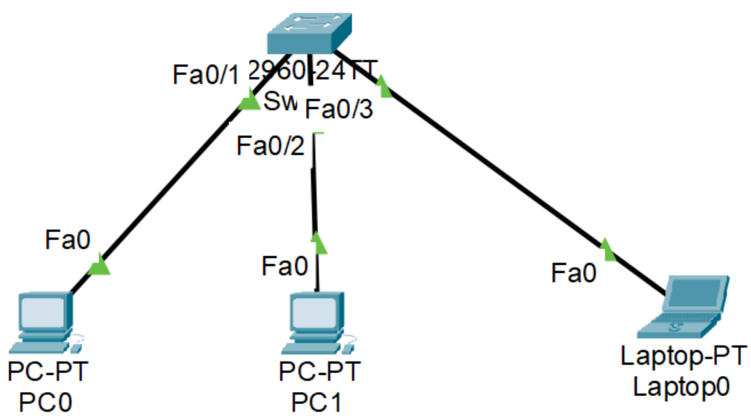
COMMANDS USED:	
enable	ENTER PRIVILEGED EXEC MODE
config terminal	ENTER GLOBAL CONFIGURATION MODE
hostname <name>	SET DEVICE HOSTNAME
ip domain-name <domain.com>	DEFINE DEVICE DOMAIN NAME
username admin privilege 15 secret admin123	CREATE LOCAL ADMIN USER
enable secret <password>	SET ENCRYPTED ENABLE (PRIV) PASSWORD
crypto key generate rsa	GENERATE RSA KEYS FOR SSH
line vty 0 5	ENTER VIRTUAL TERMINAL LINES (0–5)
login local	REQUIRE LOCAL USER AUTHENTICATION
transport input ssh	ENABLE SSH ACCESS (DISABLE TELNET)
exit	RETURN TO PREVIOUS MODE




```
Router>enable
Router#config terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#hostname R1
R1(config)#ip domain-name example.com
R1(config)#username admin privilege 15 secret admin123
R1(config)#enable secret password1
R1(config)#crypto key generate rsa
The name for the keys will be: R1.example.com
Choose the size of the key modulus in the range of 360 to 4096 for your
  General Purpose Keys. Choosing a key modulus greater than 512 may take
  a few minutes.

How many bits in the modulus [512]: 1024
% Generating 1024 bit RSA keys, keys will be non-exportable...[OK]

R1(config)#line vty 0 5
*Mar 1 0:22:8.911: %SSH-5-ENABLED: SSH 1.99 has been enabled
R1(config-line)#login local
R1(config-line)#transport input ssh
R1(config-line)#exit
```

NOTE: The steps will be the same for configuration on a router skipping VLAN since router has IPs, the ssh connection will apply to all active IPs in the router. Additionally, you can configure router gateways using ssh access. Note that since the admin has privilege 15, we do not need to use enable for privilege escalation.

Topology Example



ACL Configuration

🔹 **ACL (Access Control List):**
A set of rules that **filters or permits network traffic** based on defined conditions.

🔹 **Basic ACL (Standard ACL):**
Filters traffic **only by source IP address**.

🔹 **Extended ACL:**
Filters traffic by **source IP, destination IP, protocol, and port number**.

- **Reflexive ACL:** Allows **return traffic** for outbound sessions; **temporary, session-based**.
- **Dynamic ACL:** Grants access **after authentication**; **auto-removes** rules after timeout.

🔹 **Numbered ACL:**
An ACL identified by a **number range** (e.g., 1–99 for standard, 100–199 for extended).

🔹 **Named ACL:**
An ACL identified by a **custom name** instead of a number (more readable and editable).

🔹 **Wildcard Mask:**
Used in ACLs to **determine which bits to match** in an IP address —
✅ **0 = match bit exactly**
✅ **1 = ignore bit**
👉 It is the **opposite of a subnet mask** (where 1 = match, 0 = ignore).

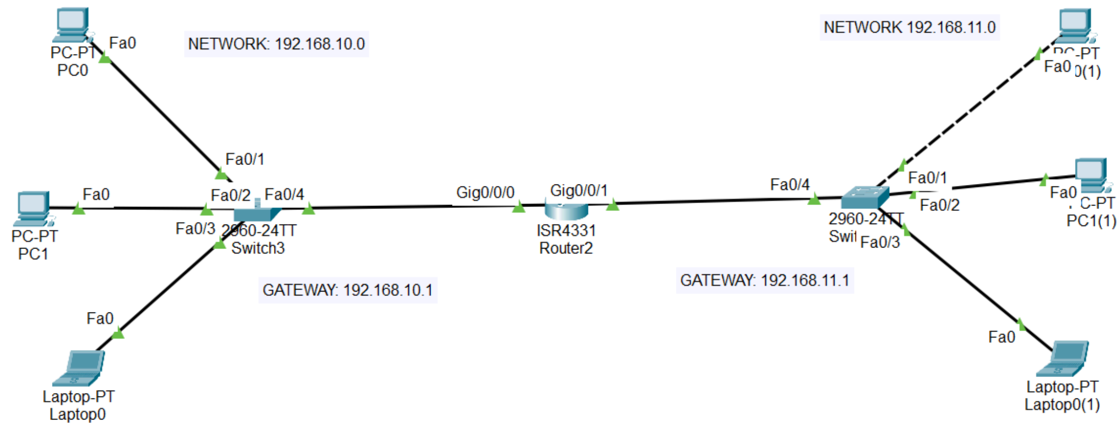
🔹 **0.0.0.0 Wildcard Mask:**
Means **match all bits exactly** — used to specify a **single host**.
Example:
`access-list 10 deny 192.168.1.5 0.0.0.0` → blocks **only** 192.168.1.5.

🔹 **Inbound ACL:**
Traffic is **filtered as it enters** an interface **before routing decisions** are made.

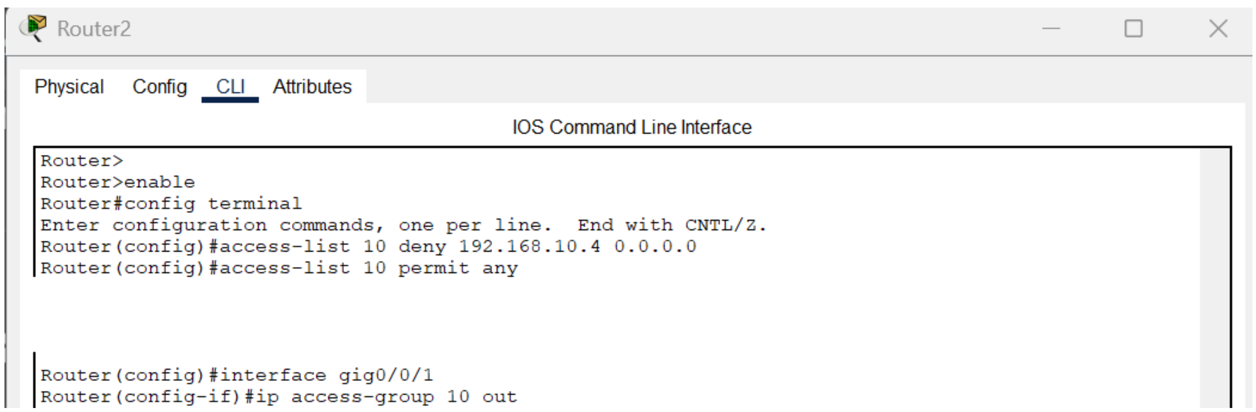
🔹 **Outbound ACL:**
Traffic is **filtered as it leaves** an interface **after routing decisions** are made.

NOTE: You need to decide if the blocking is to be done inbound or outbound. For example, if you only block PC0 (192.168.10.4) from accessing Network B (192.168.11.0), but want to allow PC0 to still access Network C (192.168.12.0), outbound works best here since inbound would block all traffic from PC0, no matter where it's going

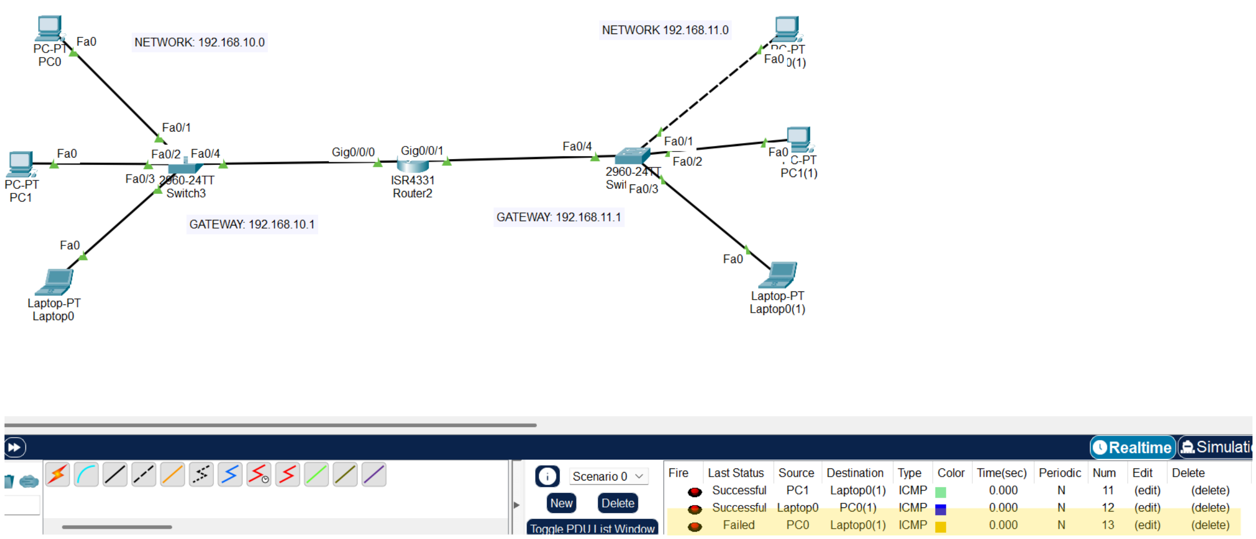
1. **Configure a network topology to implement access control lists**



2. **To block a single IP using numbered ACL, assuming the scenario where we do not want PC0 (192.168.10.4) to access Laptop0(1) (192.168.11.6), we perform the following steps via router**



3. **Check ACL establishment using simple PDU**



4. **To block an entire network using numbered ACL, assuming a scenario where we do not want 192.168.11.0 to access 192.168.10.0, we perform the following steps**

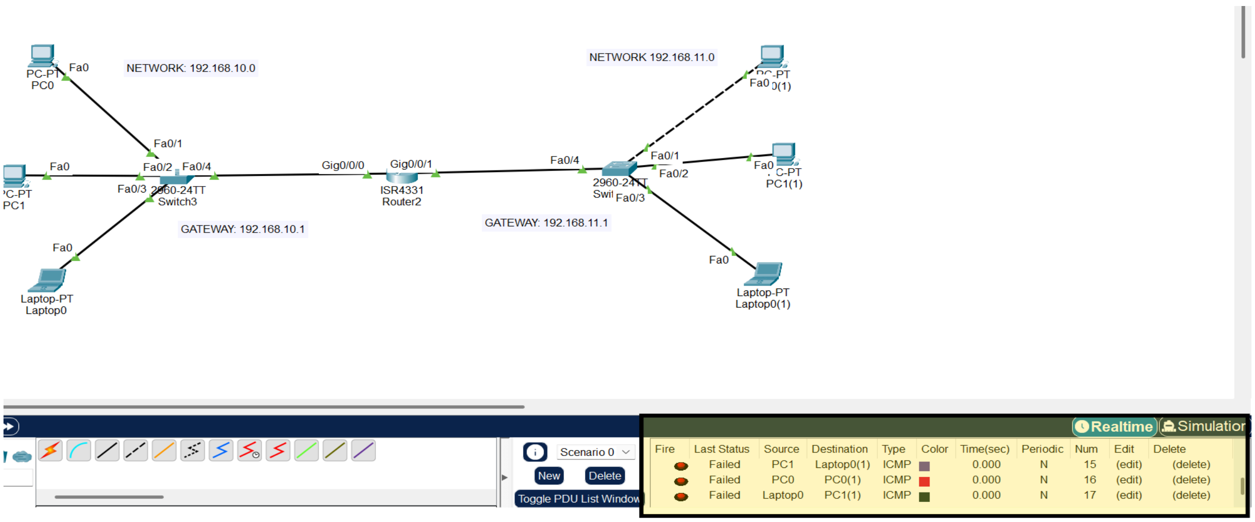
Router2

Physical | Config | CLI | Attributes

IOS Command Line Interface

```
Router#enable
Router#config terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#access-list 20 deny 192.168.11.0 0.0.0.255
Router(config)#access-list 20 permit any
Router(config)#interface g0/0/1
Router(config-if)#ip access-group 20 in
```

5. **Check ACL establishment using simple PDU**



Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	Failed	PC1	Laptop0(1)	ICMP		0.000	N	15	(edit)	(delete)
	Failed	PC0	PC0(1)	ICMP		0.000	N	16	(edit)	(delete)
	Failed	Laptop0	PC1(1)	ICMP		0.000	N	17	(edit)	(delete)

6. **To block a single IP using named ACL, assuming the scenario where we do not want PC0 (192.168.10.4) to access Laptop0(1) (192.168.11.6), we perform the following steps via router**

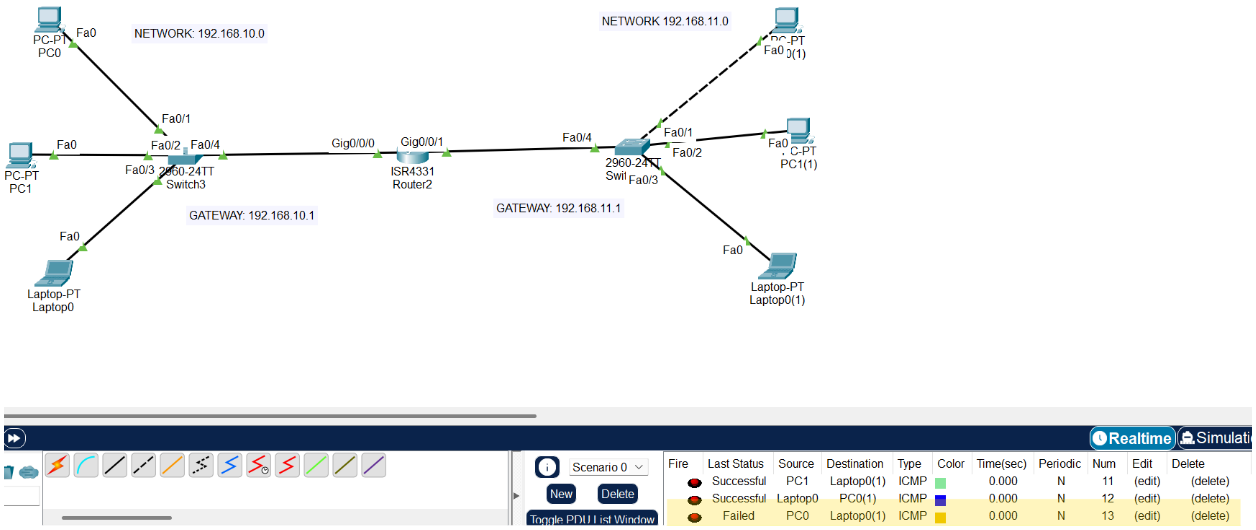
Router3

Physical | Config | CLI | Attributes

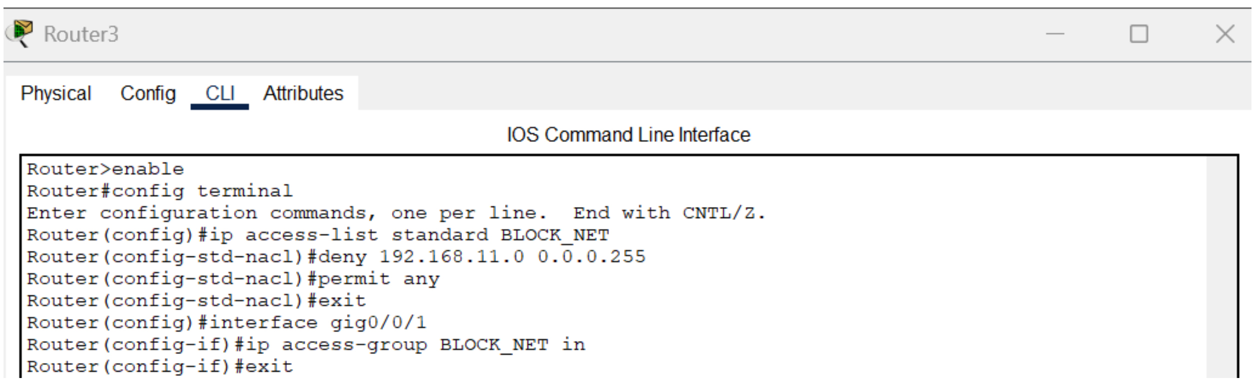
IOS Command Line Interface

```
Router>enable
Router#config terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#ip access-list standard BLOCK_SINGLE
Router(config-std-nacl)#deny host 192.168.10.4
Router(config-std-nacl)#permit any
Router(config-std-nacl)#exit
Router(config)#interface gig0/0/1
Router(config-if)#ip access-group BLOCK_SINGLE out
Router(config-if)#exit
```

7. **Check ACL establishment using simple PDU**

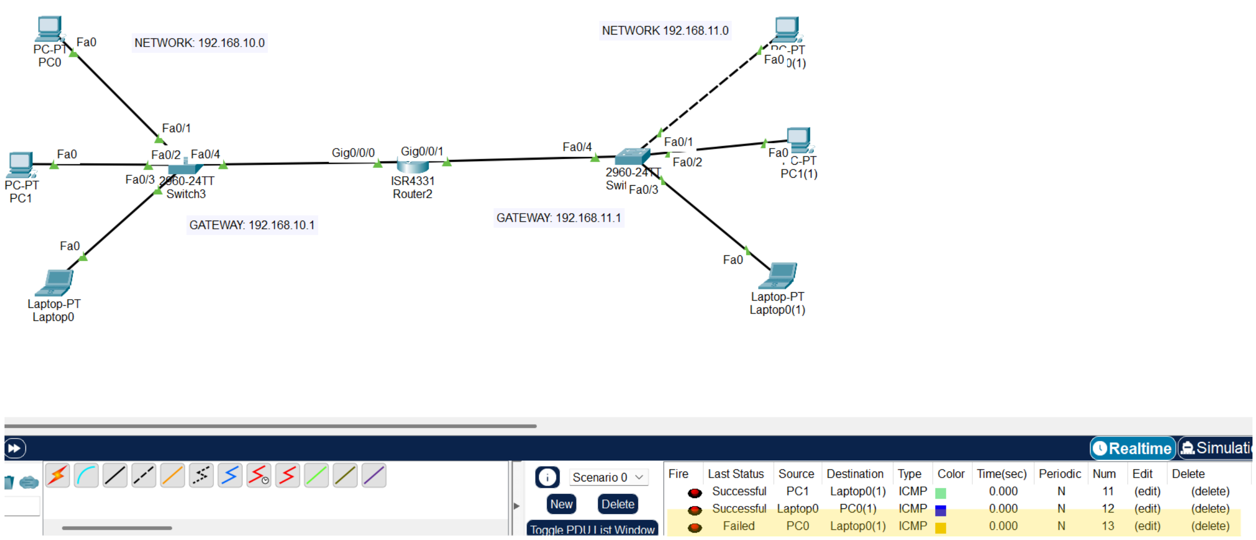


8. **To block an entire network using named ACL, assuming a scenario where we do not want 192.168.11.0 to access 192.168.10.0, we perform the following steps**




```
Router>enable
Router#config terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#ip access-list standard BLOCK_NET
Router(config-std-nacl)#deny 192.168.11.0 0.0.0.255
Router(config-std-nacl)#permit any
Router(config-std-nacl)#exit
Router(config)#interface gig0/0/1
Router(config-if)#ip access-group BLOCK_NET in
Router(config-if)#exit
```

9. **Check ACL establishment using simple PDU**



THEORY STUFF

S.No	Linux Command	Windows Command	Usage / Purpose
1	ifconfig	ipconfig	Display IP configuration, subnet mask, default gateway, interface info
2	ifconfig -a	ipconfig /all	Detailed info for all network interfaces, including MAC, DNS, IP
3	hostname	hostname	Display the system/host name
4	nslookup	nslookup	Query DNS to resolve domain names to IPs and vice versa
5	ping	ping	Test connectivity between your system and remote host (IP/hostname)
6	traceroute	tracert	Trace the route packets take to reach a destination, showing hops and latency
7	netstat	netstat	Display active connections, open ports, routing info, and network statistics
8	route	route print	Show the routing table and how data is directed to various destinations

IP Address Classes

Class A	1 – 127	(Network 127 is reserved for loopback and internal testing) Leading bit pattern	0	00000000.00000000.00000000.00000000 Network . Host . Host . Host
Class B	128 – 191	Leading bit pattern	10	10000000.00000000.00000000.00000000 Network . Network . Host . Host
Class C	192 – 223	Leading bit pattern	110	11000000.00000000.00000000.00000000 Network . Network . Network . Host
Class D	224 – 239	(Reserved for multicast)		
Class E	240 – 255	(Reserved for experimental, used for research)		

Private Address Space

Class A	10.0.0.0 to 10.255.255.255
Class B	172.16.0.0 to 172.31.255.255
Class C	192.168.0.0 to 192.168.255.255

Default Subnet Masks

Class A	255.0.0.0
Class B	255.255.0.0
Class C	255.255.255.0

Topic	Details / Notes
IPv4 Address Classes (Binary IDs)	**Class A:** 1–127, binary `0xxxxxxx`, default mask 255.0.0.0, **largest network (~16 million hosts)**, first IP = network ID, last IP = broadcast, **reserved:** 0.0.0.0 (DHCP/unknown), 127.x.x.x (loopback). **Class B:** 128–191, binary `10xxxxxx`, default mask 255.255.0.0, moderate-sized networks. **Class C:** 192–223, binary `110xxxxx`, default mask 255.255.255.0, **smallest network (~254 hosts).** **Class D:** 224–239, binary `1110xxxx`, multicast. **Class E:** 240–255, binary `1111xxxx`, experimental/research.
IPv4 vs IPv6	**IPv4:** 32-bit, ~4.3 billion addresses, dotted decimal notation, needs NAT for address shortage. **IPv6:** 128-bit, huge address space (~3.4×10^38), hexadecimal notation, built-in security (IPSec), no NAT needed, auto-configuration possible.
Private Address Spaces	**Class A:** 10.0.0.0 – 10.255.255.255 **Class B:** 172.16.0.0 – 172.31.255.255 **Class C:** 192.168.0.0 – 192.168.255.255
Broadcast & Network IDs	First IP = network ID, last IP = broadcast IP. Example: 192.168.1.0 network → network ID 192.168.1.0, broadcast 192.168.1.255.
Hubs vs Switches	Hub: broadcasts to all ports, Layer 1 (Physical), no MAC learning. Switch: forwards frames to specific port, Layer 2 (Data Link), uses MAC table, more efficient & secure.
Subnetting Formula	If you fix `k` bits for subnet: - Number of subnets = 2^k - Hosts per subnet = 2^(n–k) – 2 (subtract network & broadcast IDs)
Address Types	**Physical (MAC):** 48-bit hardware address, Data Link Layer. **Logical (IP):** 32-bit IPv4 or 128-bit IPv6, Network Layer.
ARP (Address Resolution Protocol)	Resolves IP addresses (logical) to MAC addresses (physical) on a LAN, essential for packet delivery at Data Link Layer.

Topology Type	Number of Devices (n)	Number of Cables	Number of Ports Required	Explanation	Description (Short phrases)
Bus	n	(n – 1)	2n	Devices share a single backbone cable; each device uses 2 ports.	Linear topology, simple setup, low cost, low fault tolerance, moderate scalability (can add devices but performance may degrade), single cable failure breaks network.
Star	n	n	2n	Each device connects to a central hub or switch.	Centralized topology, easy to manage, moderate cost, moderate fault tolerance, good scalability (easy to add devices), hub failure affects all.
Ring	n	n	2n	Each device connects to two others forming a closed loop.	Circular topology, predictable data flow, moderate fault tolerance, limited scalability (adding devices requires rewiring), one link failure disrupts network.
Mesh (Full)	n	n(n – 1)/2	n(n – 1)	Every device connects directly to every other device.	Fully connected topology, highly reliable, high fault tolerance, excellent scalability, expensive and complex setup.

OSI Layer	Abbreviation	Packet/Header Name	Main Function (One Phrase)	Things Used / Identifiers
7	Application	Data	Provides network services to apps	URLs, File names, SMTP, HTTP
6	Presentation	Data	Formats and encrypts data	Encryption, Compression, ASCII, JPEG
5	Session	Data	Manages sessions and connections	Session IDs, Sockets
4	Transport	Segment	Ensures reliable delivery	TCP/UDP, Ports, Sequence numbers
3	Network	Packet	Determines path and routing	IP addresses, Routing tables
2	Data Link	Frame	Provides node-to-node delivery	MAC addresses, Switches, Ethernet
1	Physical	Bits	Transmits raw bits over medium	Cables, Hubs, Repeaters, Voltage signals

Protocol / Concept	Purpose / Description	Details / Notes
DHCP	Automatically assigns IP addresses	**Initial client IP:** 0.0.0.0 (client has no IP) **Link-local placeholder:** 169.254.x.x if no DHCP responds **Server first configuration:** assign pool, subnet, gateway, DNS; enable DHCP service on interface **DORA Process:** • **Discover:** Client broadcasts to find DHCP server (255.255.255.255) • **Offer:** Server responds with offered IP (unicast/broadcast) • **Request:** Client broadcasts request for offered IP • **Acknowledge:** Server confirms assignment (unicast/broadcast) Provides lease time, subnet mask, gateway, DNS.
DNS	Resolves domain names to IP addresses	DNS translates human-readable hostnames into IPs. Hierarchical, distributed, fault-tolerant. Authoritative servers handle zones. Common Records: A (FQDN → IP), AAAA (FQDN → IPv6), MX (mail server), CNAME (alias), NS (authoritative server), SOA (zone info, admin, refresh). Essential for SMTP: outgoing mail uses MX to find recipient server; incoming mail relies on MX for delivery.
HTTP	Transfers web content	TCP port 80, plain text, Application Layer, request-response model, 4xx/5xx status codes.
HTTPS	Secure web content transfer	HTTP over TLS/SSL, TCP port 443, encrypted, requires certificate, secures sensitive sites.
FTP	Transfers files between client/server	TCP ports 20/21, client-server model, Active/Passive mode, can use FTPS/SFTP for encryption, separate control/data connections.
SMTP	Sends email	TCP port 25 (server-server), 587 (client submission), standard for outgoing mail; often paired with POP3/IMAP.
SSH	Secure remote login	TCP port 22, encrypted, key/password authentication, secure replacement for Telnet.
Telnet	Unsecure remote login	TCP port 23, plaintext, vulnerable to sniffing.
ACL	Controls network traffic	**Standard:** filter by source IP only, **Extended:** filter by source/destination IP, port, protocol, **Named:** custom name, **Numbered:** numbered (1–99 standard, 100–199 extended).

HTTP Client Error Codes (4xx)

Status Code	Error Type	Description
400	Bad Request	The request is malformed or invalid. Server can't process it.
401	Unauthorized	Authentication required or failed. Must include a valid `WWW-Authenticate` header.
403	Forbidden	Request understood, but server refuses to authorize it. Authentication won't help.
404	Not Found	Requested resource not found on the server.
408	Request Timeout	The client took too long to send the request; server stopped waiting.

HTTP Server Error Codes (5xx)

Status Code	Error Type	Description
500	Internal Server Error	Generic server-side error; unexpected condition occurred.
501	Not Implemented	Server doesn't recognize or support the request method.
502	Bad Gateway	Server acting as gateway/proxy received invalid response from upstream server.
503	Service Unavailable	Server is overloaded or under maintenance; temporary state.

🔹 HTTP vs HTTPS Comparison

Feature	HTTP	HTTPS
Full Form	Hypertext Transfer Protocol	Hypertext Transfer Protocol Secure
URL Begins With	`http://`	`https://`
Port Used	80	443
Security	Unsecured	Secured (uses SSL/TLS)
Encryption	No encryption	Data is encrypted
Layer of Operation	Application Layer	Transport Layer
Certificates Required	No	Yes (SSL/TLS certificates)
Use Cases	General browsing, non-sensitive sites	Banking, login pages, payment gateways, emails
Data Visibility	Data sent in plain text	Data encrypted, prevents eavesdropping
Trust Validation	None	Requires a signed public key certificate

🔹 SSH vs Telnet Comparison

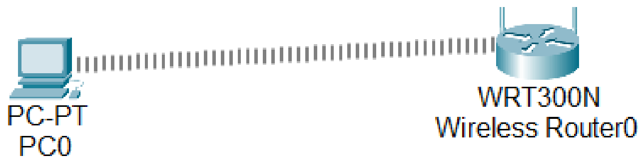
Feature	Telnet	SSH (Secure Shell)
Full Form	Teletype Network	Secure Shell
Port Used	23	22
Security	Unsecured – sends data (including passwords) in plain text	Secured – encrypts all communication using cryptographic protocols
Encryption	None	End-to-end encryption (RSA, AES, etc.)
Authentication	Username and password sent in clear text	Secure authentication (passwords or key-based)
Layer of OSI Model	Application Layer	Application Layer
Protocol Type	Simple remote terminal protocol	Encrypted remote login and command execution protocol
Data Transmission	Plain text	Cipher text (encrypted)

NOTE: Socket programming not included

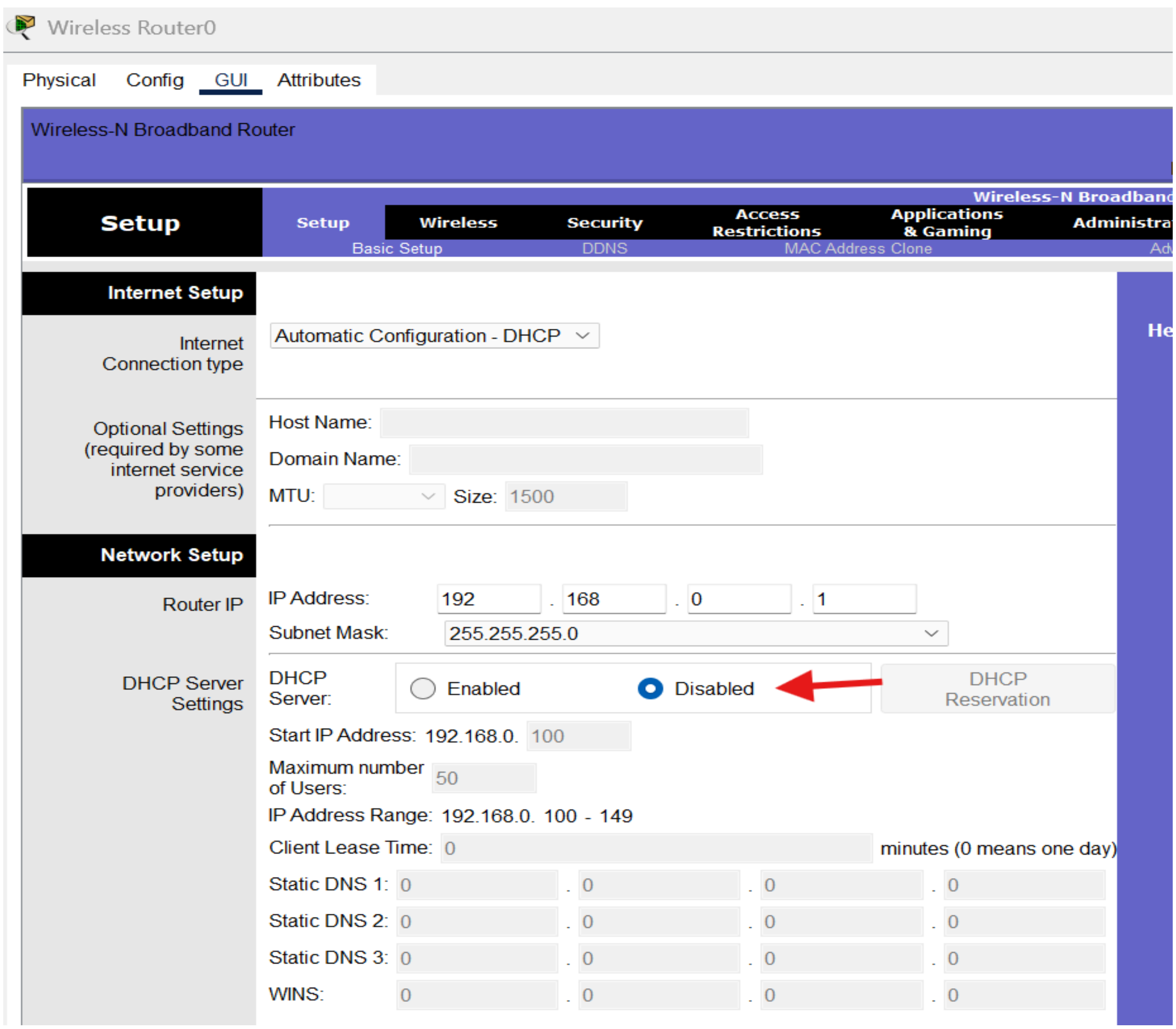
CN LAB MANUAL POST MID 1

Wireless Routing Configuration

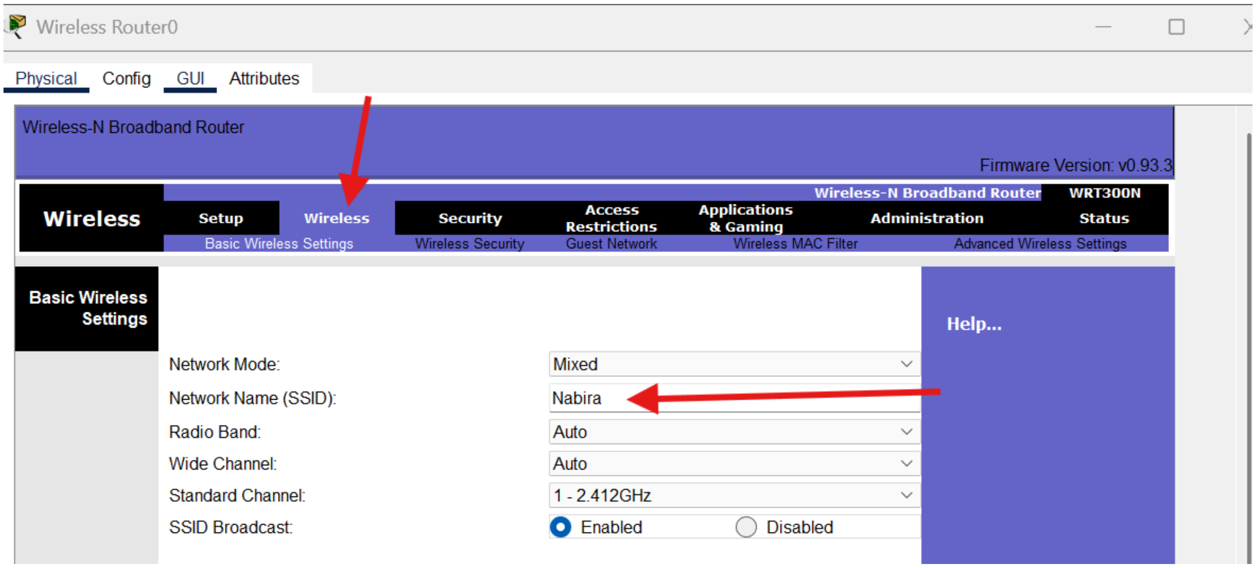
Example Topology



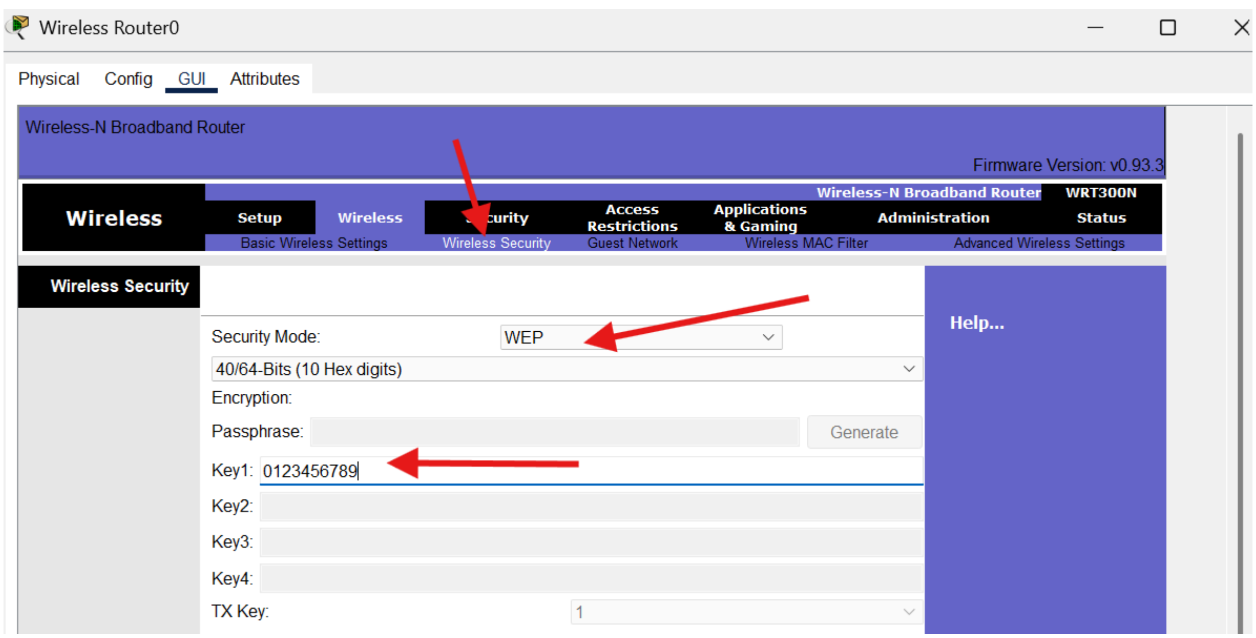
1. **Go into router → GUI → setup, and disable DHCP, make sure to save changes at the bottom**



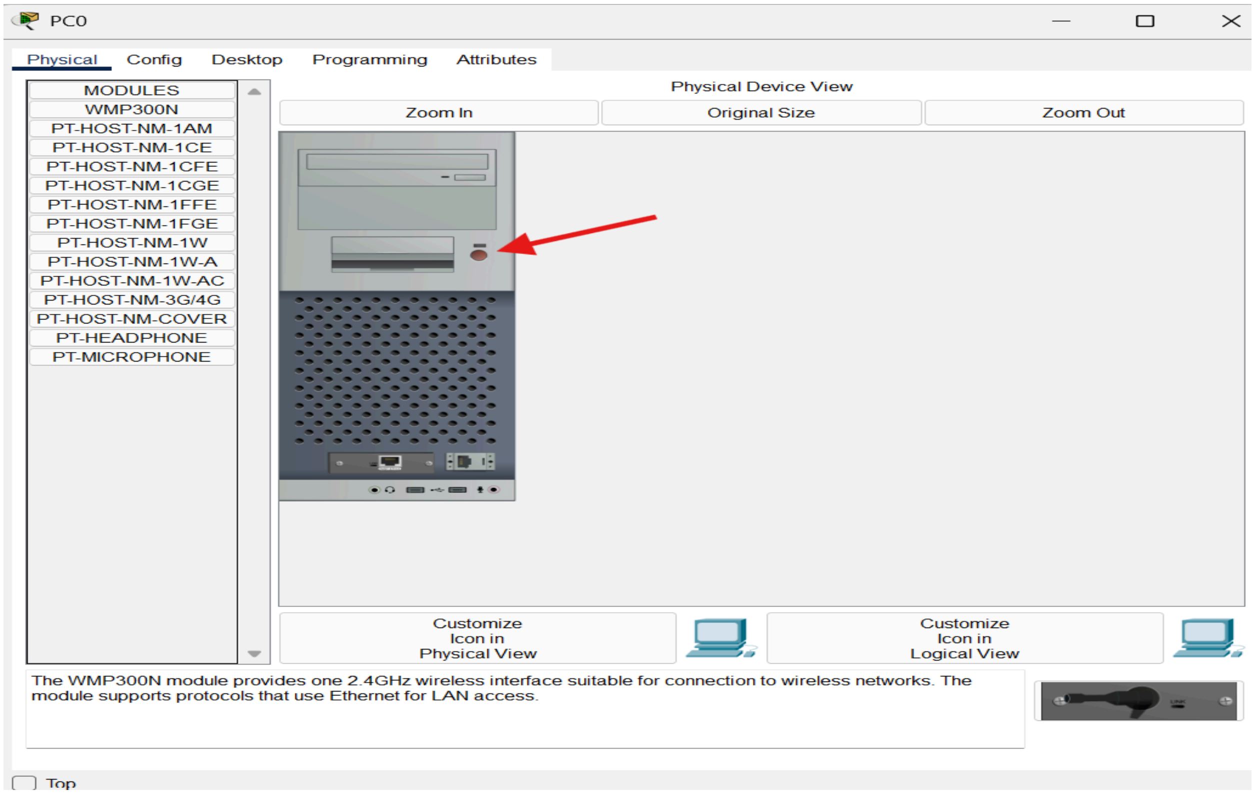
2. **Go into router → GUI → wireless → Basic wireless settings, and change the network name from default to any name, make sure to save settings at the bottom**



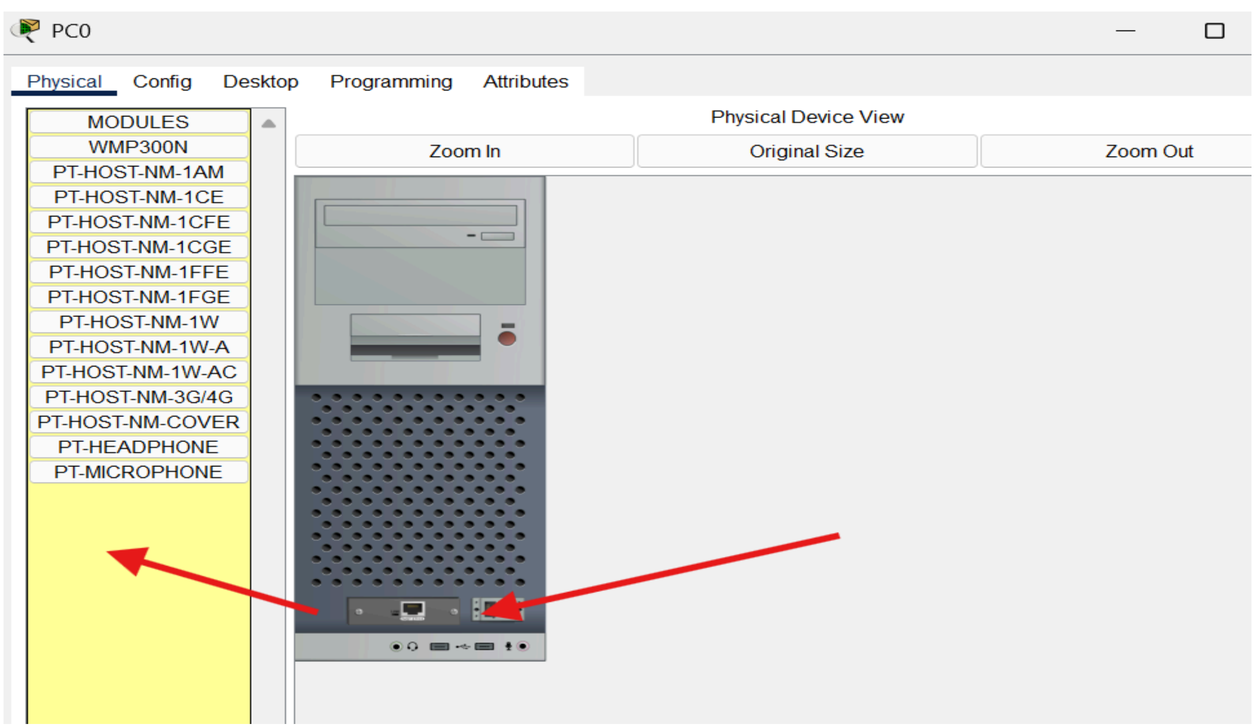
3. **Go into router → GUI → wireless → wireless security, and set security mode to WEP, set a key (for example 0-9) for 4064 bits - 10 hex digits, make sure to save settings at bottom**

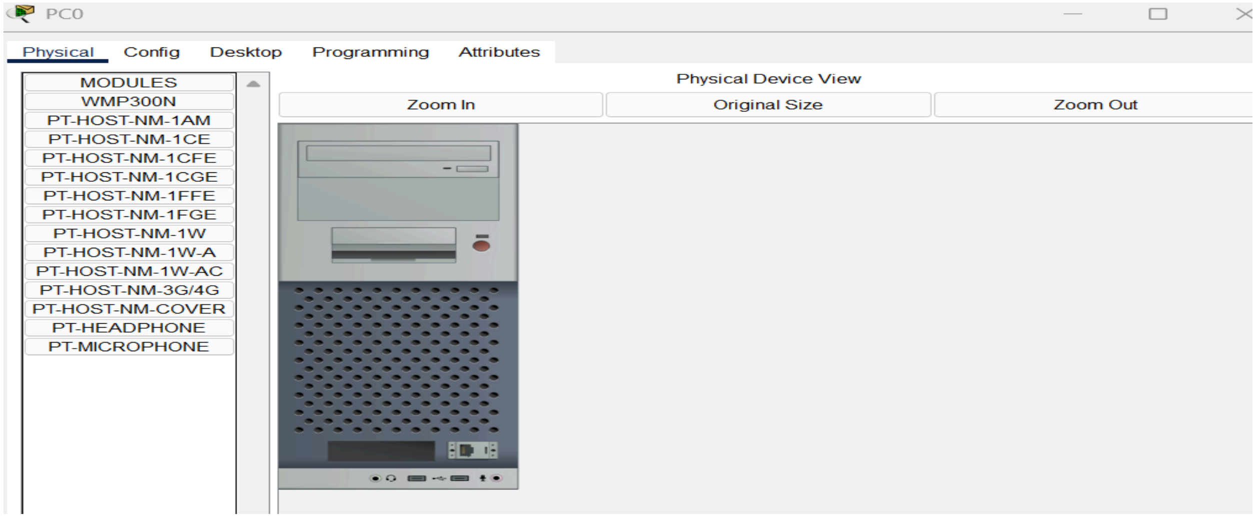


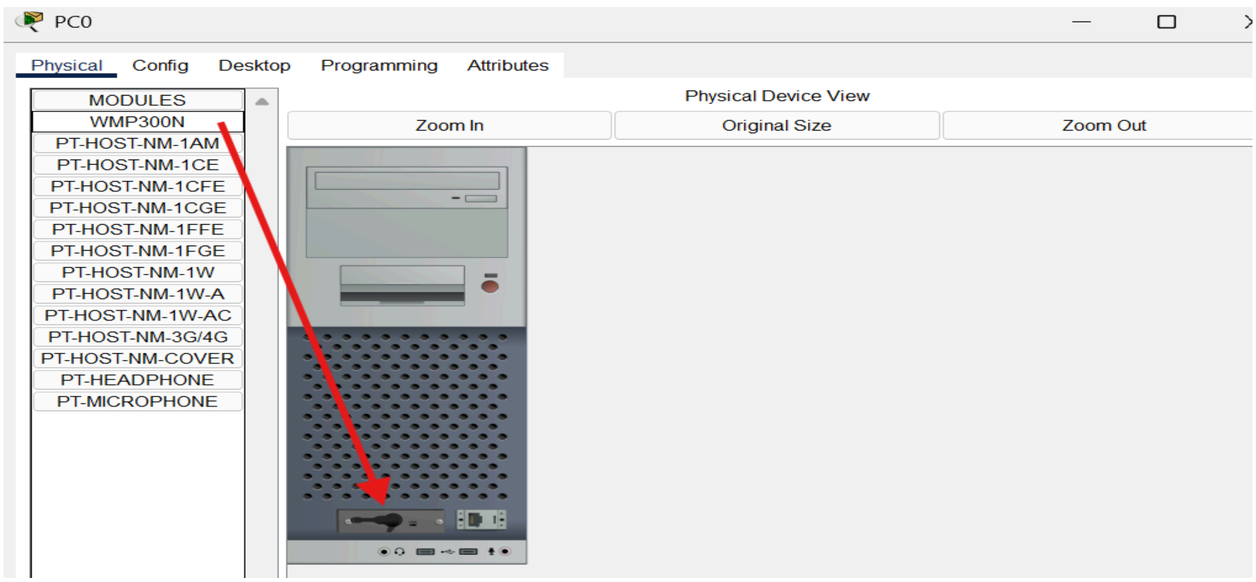
4. **Go into PC and switch it off via button**

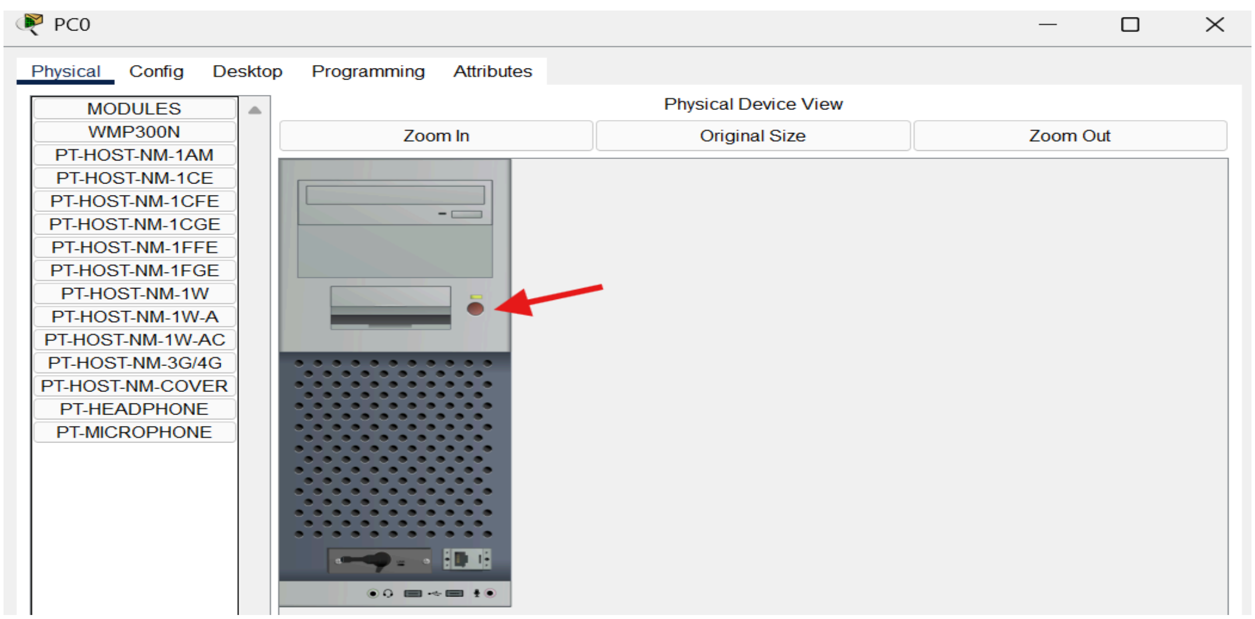


5. **Drag and remove port, and add WMP300N PORT, and switch PC back on**

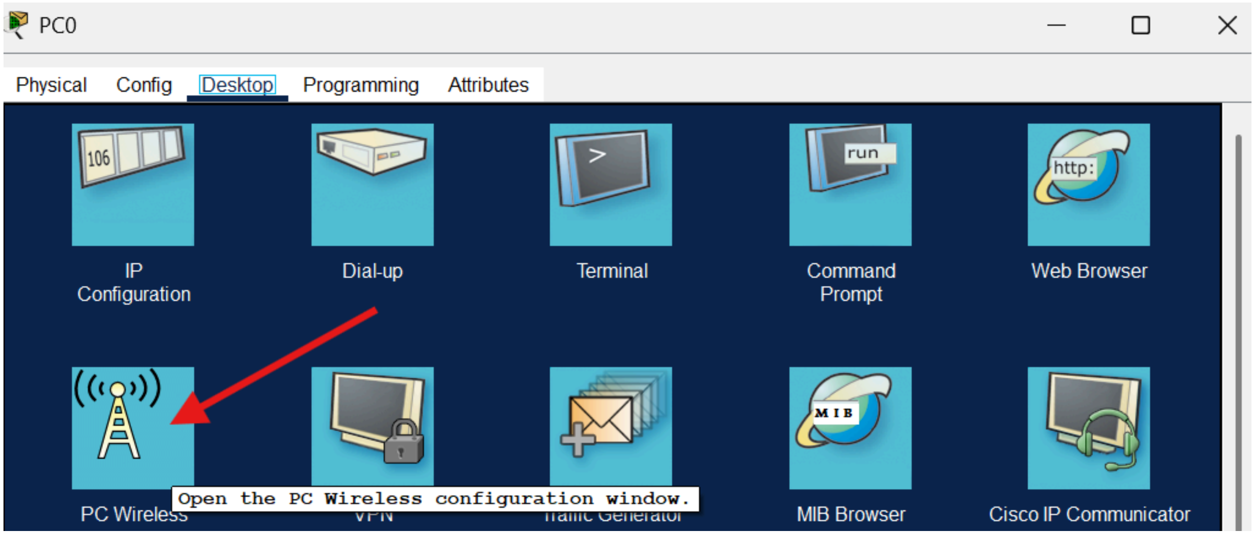


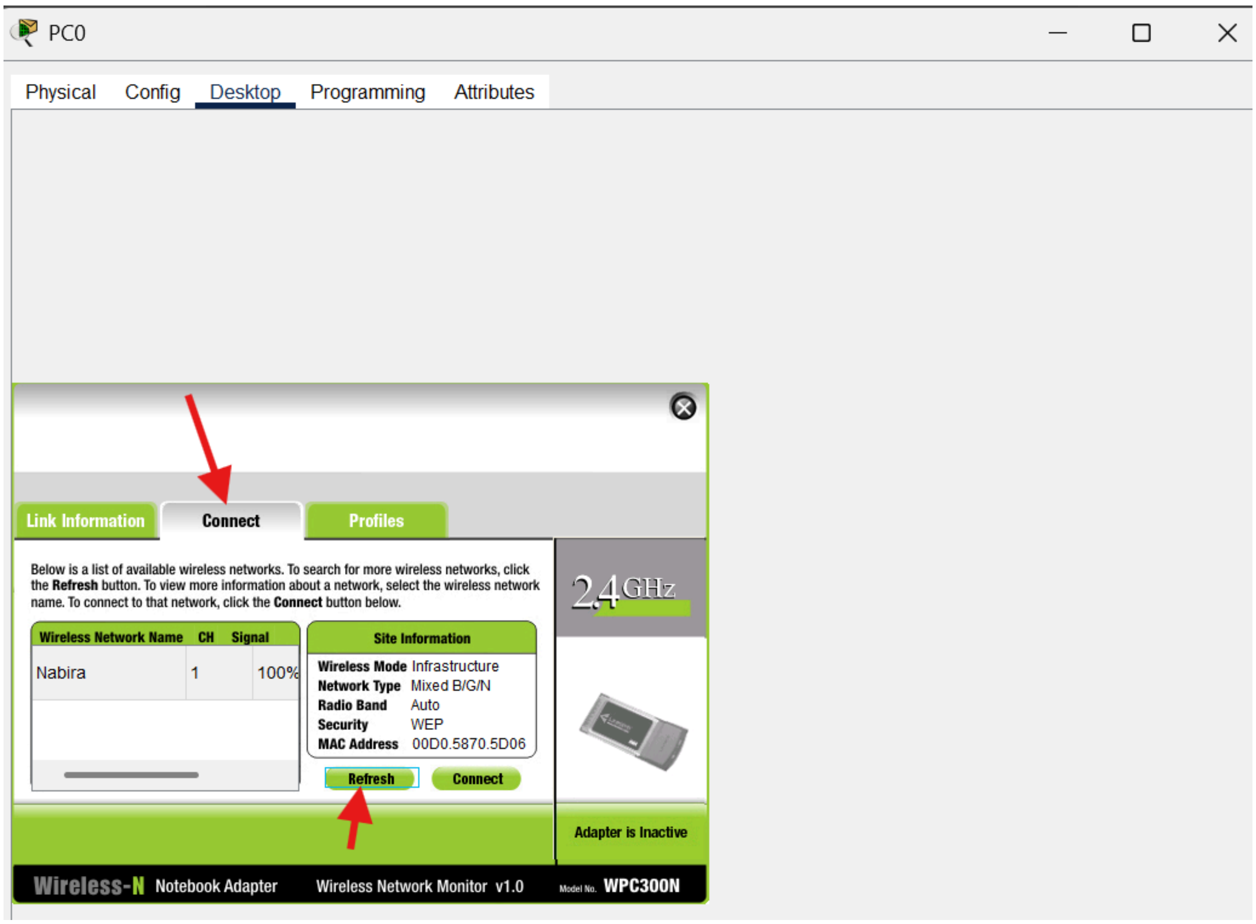

PC0
Physical
Config
Desktop
Programming
Attributes
MODULES
WMP300N
PT-HOST-NM-1AM
PT-HOST-NM-1CE
PT-HOST-NM-1CFE
PT-HOST-NM-1CGE
PT-HOST-NM-1FFE
PT-HOST-NM-1FGE
PT-HOST-NM-1W
PT-HOST-NM-1W-A
PT-HOST-NM-1W-AC
PT-HOST-NM-3G/4G
PT-HOST-NM-COVER
PT-HEADPHONE
PT-MICROPHONE
Physical Device View
Zoom In
Original Size
Zoom Out


PC0
Physical
Config
Desktop
Programming
Attributes
MODULES
WMP300N
PT-HOST-NM-1AM
PT-HOST-NM-1CE
PT-HOST-NM-1CFE
PT-HOST-NM-1CGE
PT-HOST-NM-1FFE
PT-HOST-NM-1FGE
PT-HOST-NM-1W
PT-HOST-NM-1W-A
PT-HOST-NM-1W-AC
PT-HOST-NM-3G/4G
PT-HOST-NM-COVER
PT-HEADPHONE
PT-MICROPHONE
Physical Device View
Zoom In
Original Size
Zoom Out

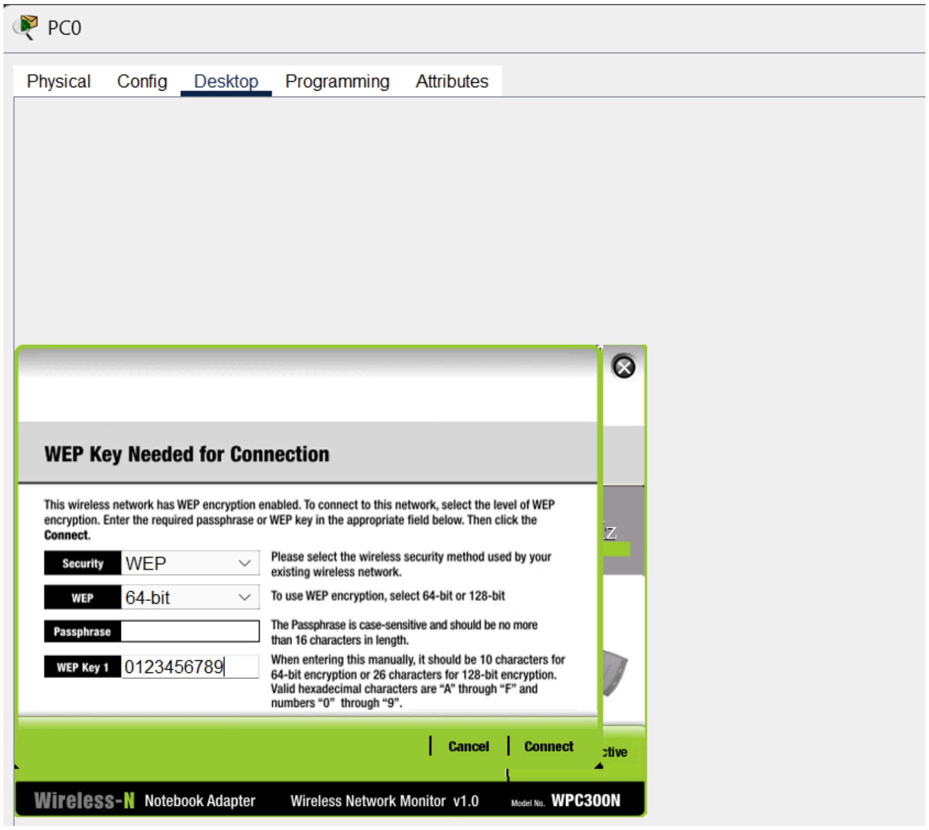

PC0
Physical
Config
Desktop
Programming
Attributes
MODULES
WMP300N
PT-HOST-NM-1AM
PT-HOST-NM-1CE
PT-HOST-NM-1CFE
PT-HOST-NM-1CGE
PT-HOST-NM-1FFE
PT-HOST-NM-1FGE
PT-HOST-NM-1W
PT-HOST-NM-1W-A
PT-HOST-NM-1W-AC
PT-HOST-NM-3G/4G
PT-HOST-NM-COVER
PT-HEADPHONE
PT-MICROPHONE
Physical Device View
Zoom In
Original Size
Zoom Out

6. **Go into PC Desktop → PC Wireless –. Connect, and click on refresh to load your wireless configuration**



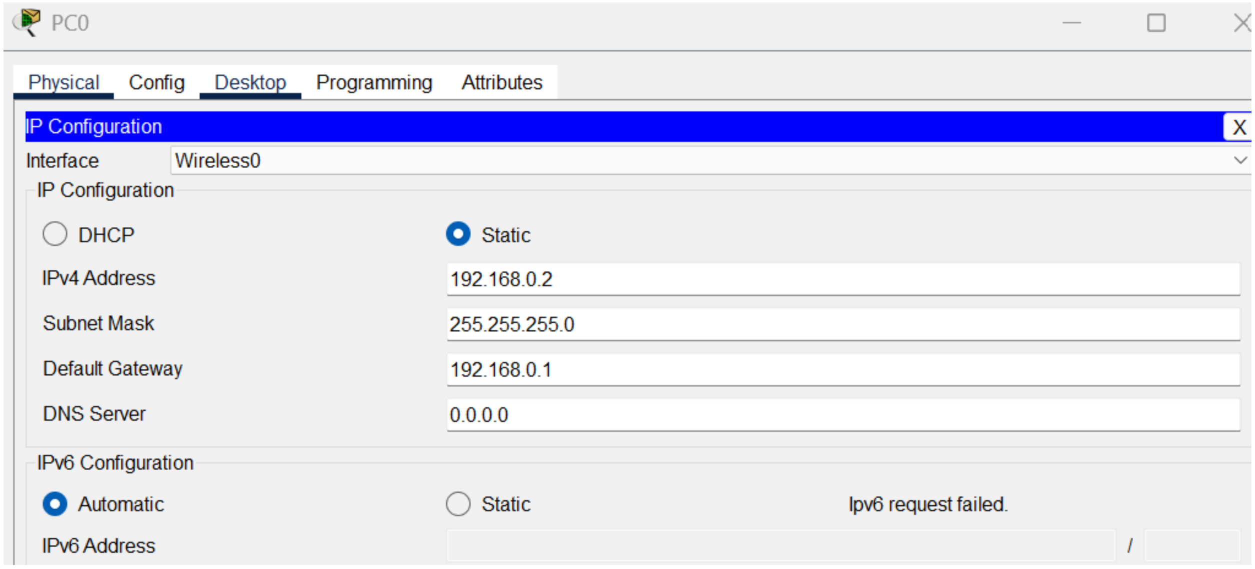


7. **Click on your wireless network name, click on connect, enter your key to connect to your wireless router, click on connect again**

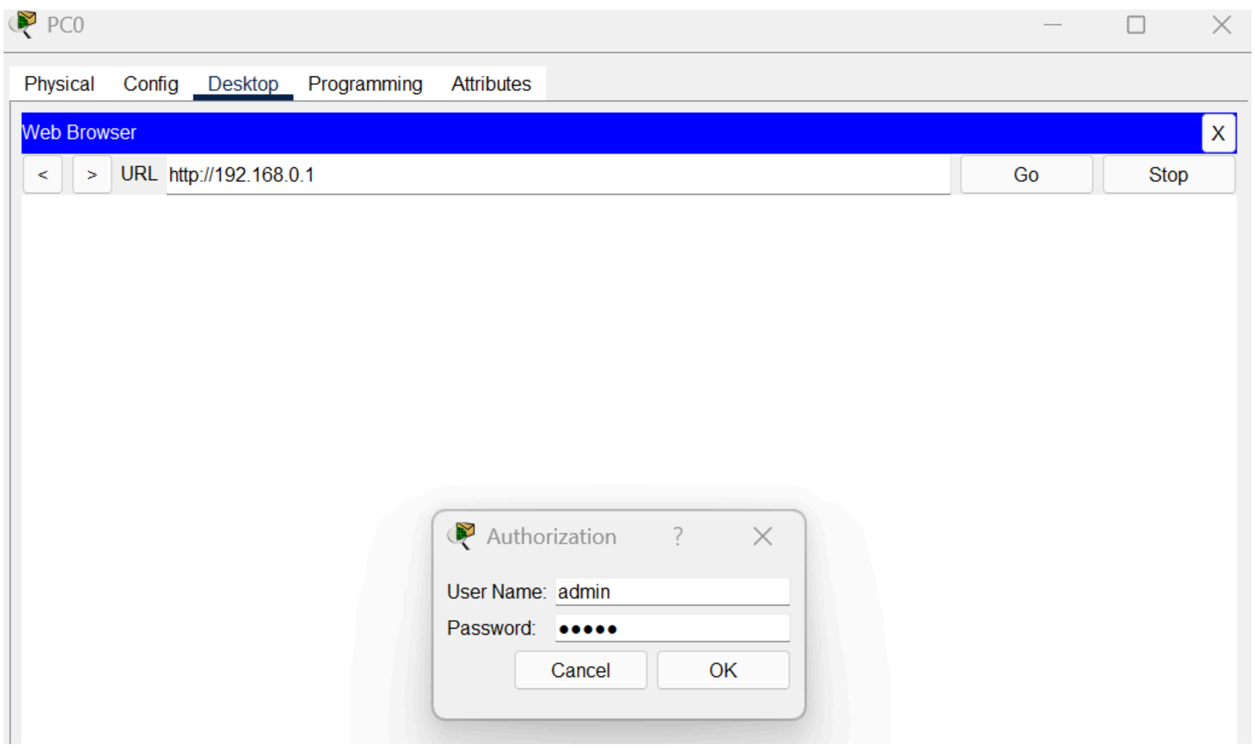

PC0
Physical
Config
Desktop
Programming
Attributes
WEP Key Needed for Connection
This wireless network has WEP encryption enabled. To connect to this network, select the level of WEP encryption. Enter the required passphrase or WEP key in the appropriate field below. Then click the Connect.
Security
WEP
Please select the wireless security method used by your existing wireless network.
WEP
64-bit
To use WEP encryption, select 64-bit or 128-bit
Passphrase
The Passphrase is case-sensitive and should be no more than 16 characters in length.
WEP Key 1
0123456789
When entering this manually, it should be 10 characters for 64-bit encryption or 26 characters for 128-bit encryption. Valid hexadecimal characters are "A" through "F" and numbers "0" through "9".
Cancel
Connect
Wireless-N Notebook Adapter
Wireless Network Monitor v1.0
Model No. WPC300N

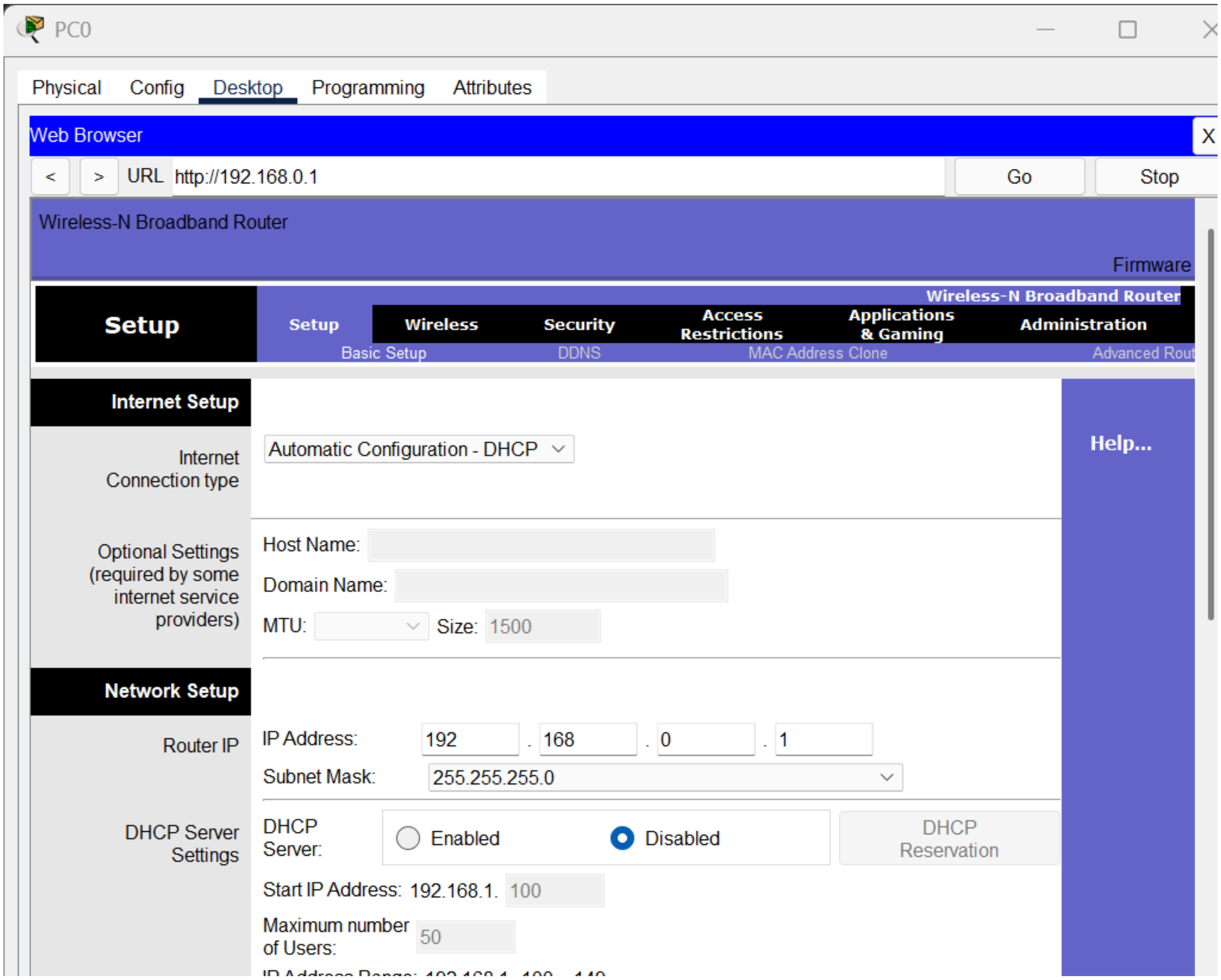

PC0
Physical
Config
Desktop
Programming
Attributes
Link Information
Connect
Profiles
More Information
Infrastructure Mode
2.4GHz
You have successfully connected to the access point
Signal Strength
Link Quality
Adapter is Active
Wireless-N Notebook Adapter
Wireless Network Monitor v1.0
Model No. WPC300N

8. **Set your PC static IP now, along with router gateway**



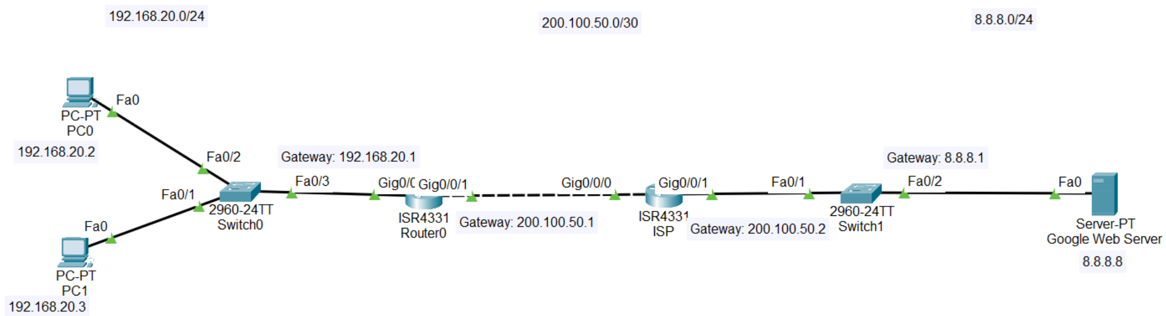
9. **Go into PC web browser, enter router ip, and enter credentials admin, admin.**



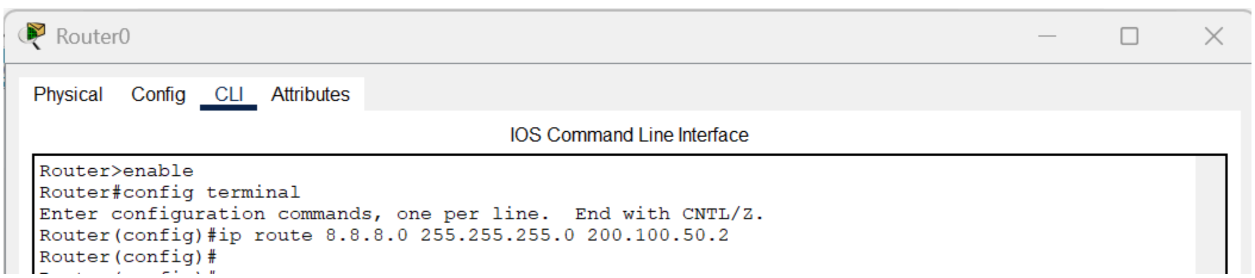

PC0
Physical
Config
Desktop
Programming
Attributes
Web Browser
URL http://192.168.0.1
Go
Stop
Wireless-N Broadband Router
Firmware
Wireless-N Broadband Router
Setup
Setup
Wireless
Security
Access Restrictions
Applications & Gaming
Administration
Basic Setup
DDNS
MAC Address Clone
Internet Setup
Internet Connection type
Automatic Configuration - DHCP
Help...
Optional Settings (required by some internet service providers)
Host Name:
Domain Name:
MTU:
Size: 1500
Network Setup
Router IP
IP Address: 192 . 168 . 0 . 1
Subnet Mask: 255.255.255.0
DHCP Server Settings
DHCP Server:
Enabled
Disabled
DHCP Reservation
Start IP Address: 192.168.1. 100
Maximum number of Users: 50

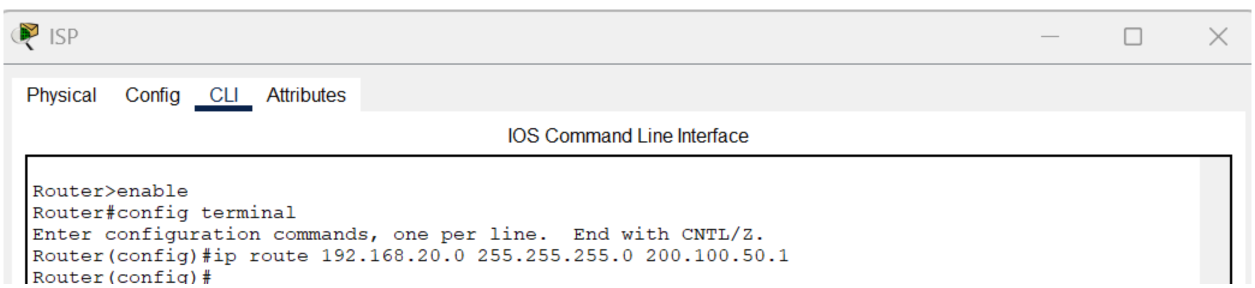
Static Routing

1. **Set up CLI router configurations as usual, according to the given gateways and their corresponding interfaces**



2. **Add unlinked/indirect network route to the corresponding router. (For example, Router 0 is indirectly connected to 8.8.8.0/24 network with a subnet mask 255.255.255.0 and a next hop 200.100.50.2)**





Routing Notes

1. Connectivity:

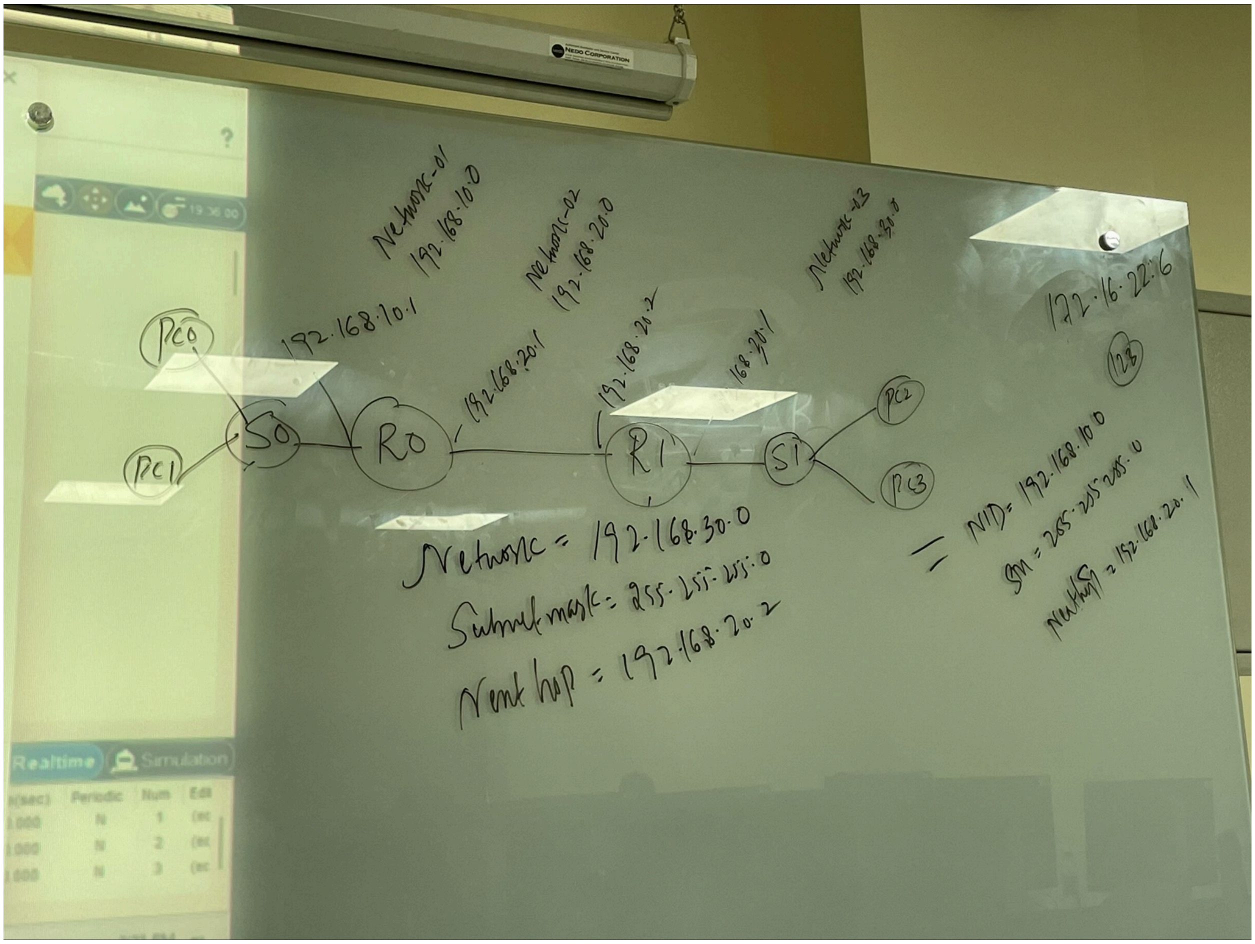
- **Directly connected networks:** Can communicate without routing.
- **Indirectly connected networks:** Require routing to communicate.

2. Types of Routing:

- **Static Routing:**
  - Routes are manually configured by the administrator.
  - No routing protocol involved.
  - You define paths for all indirectly connected networks yourself.
- **Dynamic Routing:**
  - Uses a routing protocol to automatically discover and manage paths.
  - Each router interface must belong to a different network.
  - Routing protocols use metrics to determine the best path.

3. Router-to-Router Connections:

- Interfaces connecting routers must be on the **same network** to communicate.
- Example:
  - Router1 interface: 192.168.10.1 (gateway)
  - Router2 interface: 192.168.10.2 (gateway)



NAT (Network Address Translation)

Internet Assigned Numbers Authority.

1. IP Types

- **Public IP:** Globally unique, assigned by IANA; must be purchased.
- **Private IP:** Not unique, reused within LANs; not routable on the Internet.

2. Purpose of NAT

- Allows devices with private IPs to access the Internet by translating them to public IPs.
- Acts as an intermediate solution during transition from IPv4 to IPv6.

3. Types of NAT

- **Static NAT:** One-to-one mapping between private and public IP.
- **Dynamic NAT:** Maps private IPs to a **pool of public IPs** (many-to-many).
- **PAT (Port Address Translation):** Maps multiple private IPs to a **single public IP**, using different ports (many-to-one).

4. NAT Terminology

- **Inside local:** Source IP of the host in private network.
- **Inside global:** Public IP mapped to the inside host.
- **Outside local:** IP of the external destination as seen from inside.
- **Outside global:** IP of the external destination as seen on the Internet.

Static NAT Configuration

Static NAT

Creates a permanent 1:1 mapping between one private IP and one public IP.

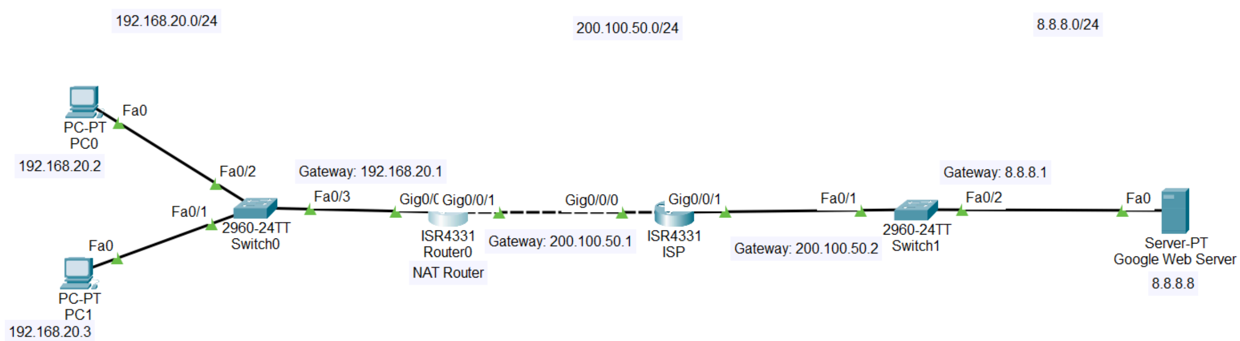
Dynamic NAT

Maps private IPs to a pool of public IPs on a first-come, first-served basis.

PAT (NAT Overload)

Allows many private hosts to share a single public IP using different port numbers.

Example Topology



✅ **Therefore:**

- The **inside local address** (e.g., 192.168.20.2) must be part of the **inside** network.
- The **inside global address** (e.g., 200.100.50.1, or from a public pool) must be part of the **outside** network.

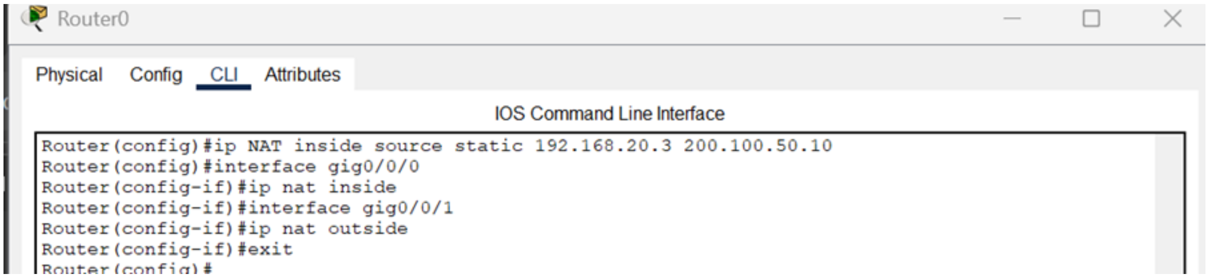
So it **must belong to or be routed through the external interface network**, not the internal one.

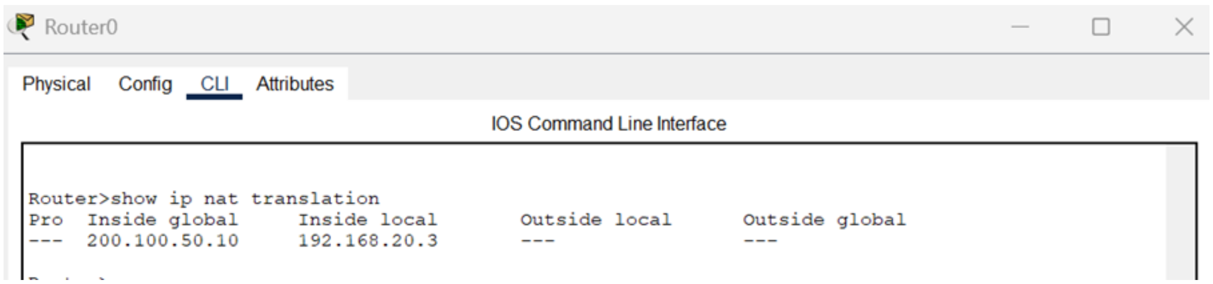
192.168.20.2 → 192.168.20.10	❌	No	Both are in the same inside network

- **Inside Local:** Private IP address of the internal device (e.g., 192.168.20.2).
- **Inside Global:** Public IP assigned by the router to represent the inside host on the outside network (e.g., 200.100.50.1).
- **Outside Local:** The external host's IP as seen from inside the network (may be private).
- **Outside Global:** The actual public IP address of the external host (e.g., 8.8.8.8).

NOTE: SIR DID THIS IN CLASS, BUT GPT SAYS YOU CANNOT MAP THE PRIVATE LOCAL IP TO A PUBLIC GLOBAL IP ON THE SAME NETWORK. IT HAS TO BE THE EXTERNAL ONE

1. **Add NAT configuration via CLI on NAT router for each device/private IP in the network you want to map.**

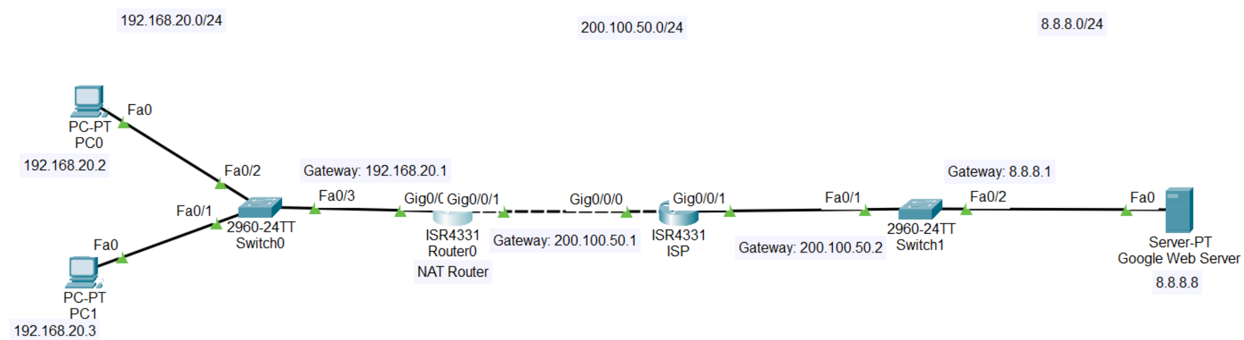




Step	IOS Command	Precise Explanation (Inline Phrase)
1	`ip nat inside source static 192.168.20.3 200.100.50.10`	Creates a permanent 1:1 mapping from private IP **192.168.20.3** to public IP **200.100.50.10**
2	`interface gig0/0/0`	Enters configuration mode for the **LAN interface**
3	`ip nat inside`	Marks **gig0/0/0** as the **inside NAT interface** (private side)
4	`exit`	Leaves interface configuration mode
5	`interface gig0/0/1`	Enters configuration mode for the **WAN/ISP interface**
6	`ip nat outside`	Marks **gig0/0/1** as the **outside NAT interface** (public side)

Dynamic NAT Configuration

Example Topology



1. **Add NAT configuration via CLI on NAT router for the network**

Router0

Physical Config CLI Attributes

IOS Command Line Interface

```
Router>enable
Router#config termina
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#ip nat pool NAT-POOL 200.100.50.10 200.100.50.20 netmask 255.255.255.0
Router(config)#access-list 1 permit 192.168.20.0 0.0.0.255
Router(config)#ip nat inside source list 1 pool NAT-POOL
Router(config)#interface gig0/0/0
Router(config-if)#ip nat inside
Router(config-if)#exit
Router(config)#interface gig0/0/1
Router(config-if)#ip nat outside
Router(config-if)#exit
Router(config)#exit
```

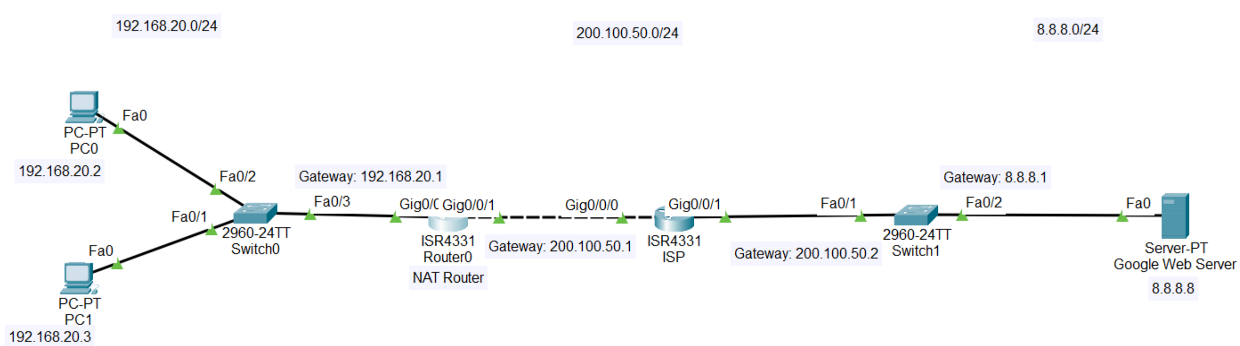
2. **Generate msg transfer**
3. **Check mapping**

```
Router#show ip nat translation
Pro  Inside global      Inside local       Outside local      Outside global
icmp 200.100.50.10:11   192.168.20.3:11    8.8.8.8:11         8.8.8.8:11
```

Step	IOS Command	Precise Explanation (Inline Phrase)
1	`ip nat pool NAT-POOL 200.100.50.10 200.100.50.20 netmask 255.255.255.0`	Creates a **NAT address pool** using public IPs from **200.100.50.10 to 200.100.50.20**
2	`access-list 1 permit 192.168.20.0 0.0.0.255`	Defines which **inside private network** is allowed to use NAT (192.168.20.0/24)
3	`ip nat inside source list 1 pool NAT-POOL`	Links the ACL with the NAT pool so eligible private IPs get translated dynamically
4	`interface gig0/0/0`	Enters the **LAN interface** configuration
5	`ip nat inside`	Sets **gig0/0/0** as the **inside NAT interface**
6	`exit`	Leaves interface mode
7	`interface gig0/0/1`	Enters the **WAN/ISP interface** configuration
8	`ip nat outside`	Sets **gig0/0/1** as the **outside NAT interface**
9	`exit`	Ends interface configuration
10	`exit`	Exits global configuration

PAT NAT Configuration

Example Topology



1. **Add NAT configuration via CLI on NAT router for the network**

Router0

Physical | Config | CLI | Attributes

IOS Command Line Interface

```
Router>enable
Router#config terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#access-list 1 permit 192.168.20.0 0.0.0.255
Router(config)#ip nat inside source list 1 interface gig0/0/1 overload
Router(config)#interface gig0/0/0
Router(config-if)#ip nat inside
Router(config-if)#exit
Router(config)#interface gig0/0/1
Router(config-if)#ip nat outside
Router(config-if)#exit
Router(config)#
```

Step	IOS Command	Precise Explanation (Inline Phrase)
1	`access-list 1 permit 192.168.20.0 0.0.0.255`	Defines which **inside private network** will use PAT (192.168.20.0/24)
2	`ip nat inside source list 1 interface gig0/0/1 overload`	Enables **PAT**, translating many private IPs to **one public IP** on gig0/0/1 using port numbers
3	`interface gig0/0/0`	Enters configuration mode for the **LAN interface**
4	`ip nat inside`	Marks **gig0/0/0** as the **inside NAT interface**
5	`exit`	Leaves interface configuration
6	`interface gig0/0/1`	Enters configuration mode for the **WAN/Internet interface**
7	`ip nat outside`	Marks **gig0/0/1** as the **outside NAT interface**
8	`exit`	Ends interface configuration

```
Pro   Inside global       Inside local       Outside local      Outside global
icmp  200.100.50.10:11    192.168.20.3:11    8.8.8.8:11         8.8.8.8:11
```

🔍 What Each Field Shows (Precise)

Inside Local → 192.168.20.3:11

- Original **private IP** of the inside host.
- ICMP identifier = **11**.

Inside Global → 200.100.50.10:11

- Public IP that NAT assigned to represent the inside host.
- Same ICMP ID (11) used to track this session.

Outside Local → 8.8.8.8:11

- Address NAT uses internally to reach the external host.
- Usually identical to outside global unless special NAT is used.

Outside Global → 8.8.8.8:11

- The **real public IP** of the external host (Google DNS).

📌 What the Translation Means (Precise)

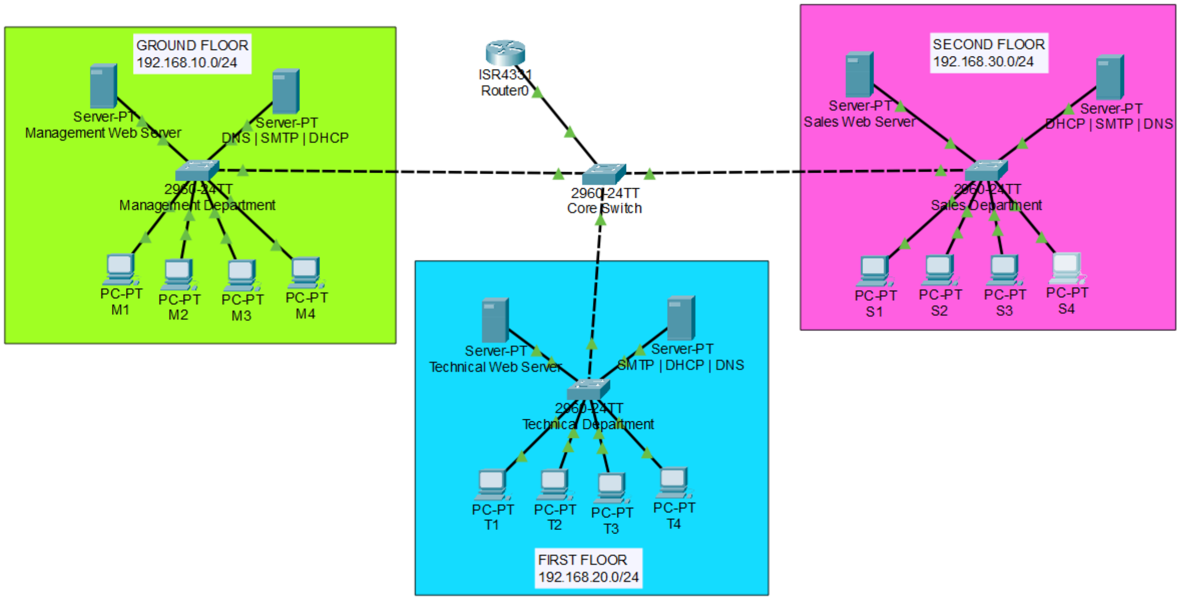
An internal device **192.168.20.3** sent an ICMP echo (ping) with ID **11** to **8.8.8.8**. NAT translated the source to **200.100.50.10**, so that is the IP seen externally.

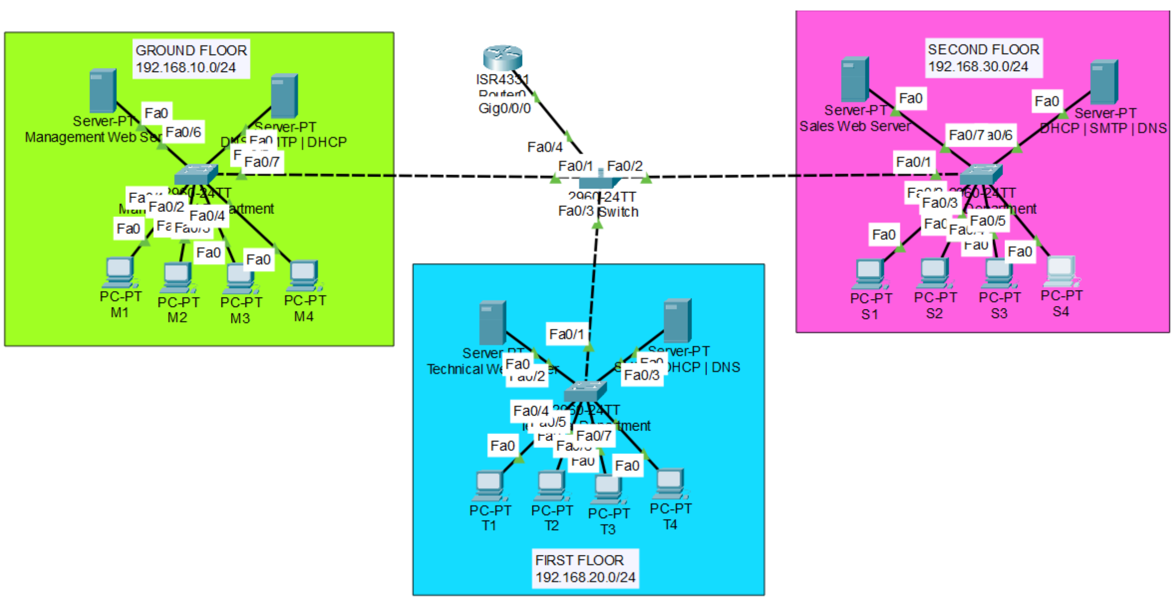
📌 Why Outside Local = Outside Global

The router does **not** translate the destination IP.
Therefore both remain **8.8.8.8**.

VLAN Configuration

Example Topology

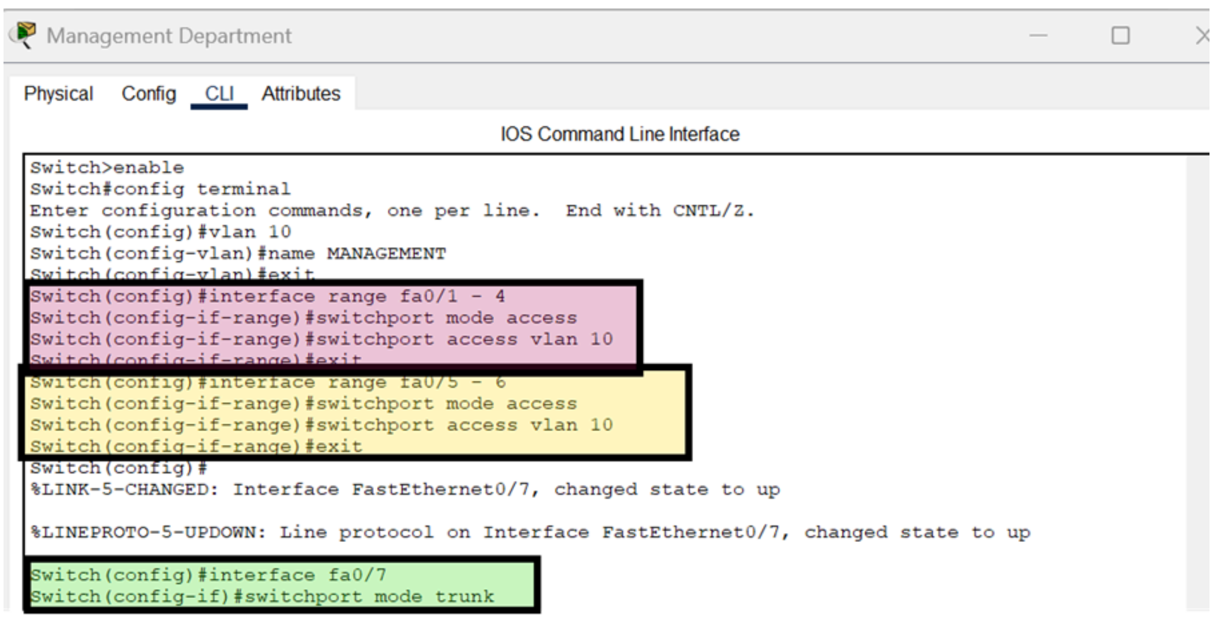


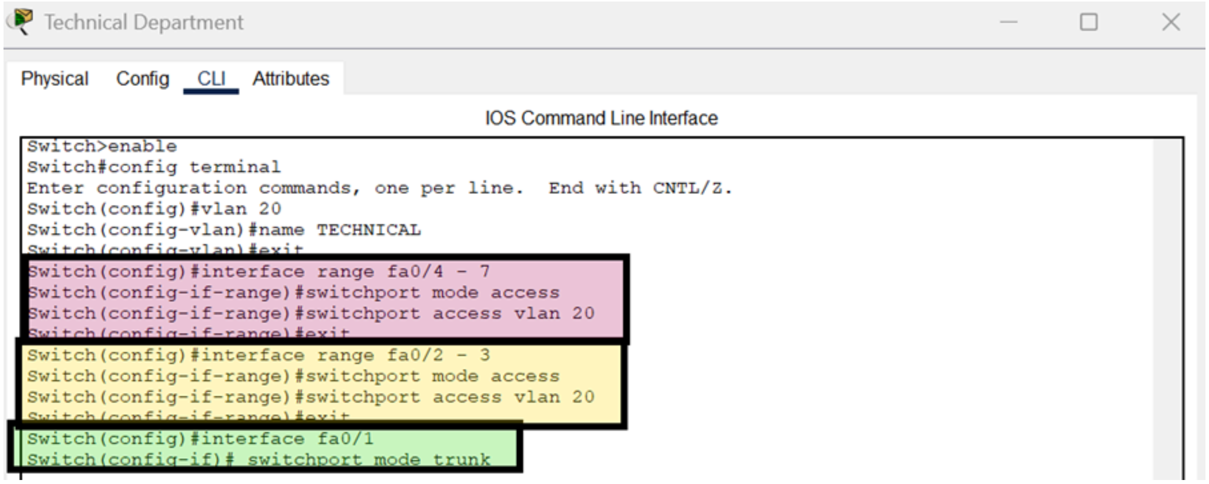


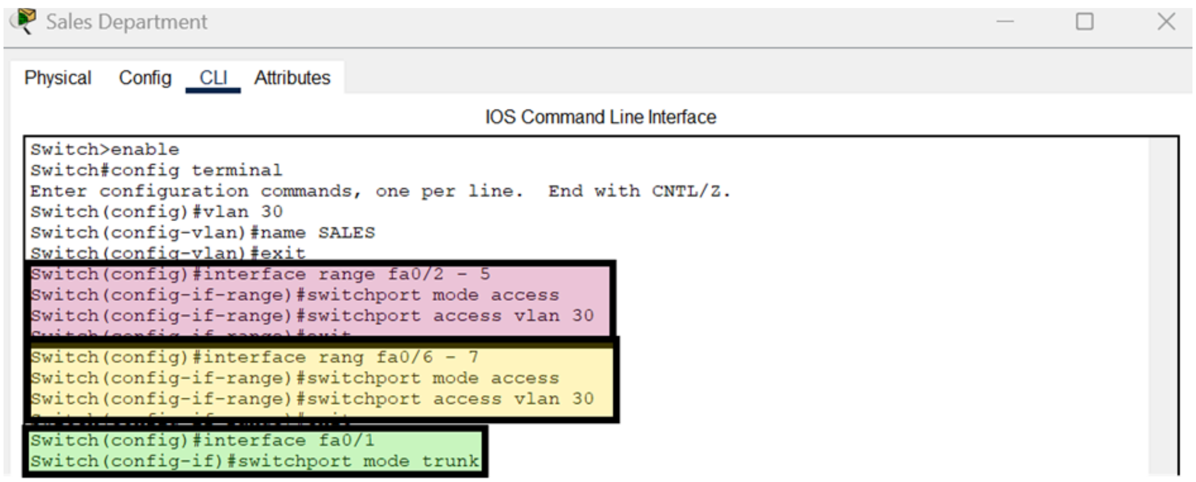
- **Access Port:** A switch port that carries traffic for **a single VLAN** to end devices.
- **Trunk Port:** A switch port that carries traffic for **multiple VLANs** between switches or to a router.
- **VLAN:** A **Virtual LAN** that segments a network logically, isolating broadcast domains.
- **Inter-VLAN Routing:** The process of **forwarding traffic between VLANs**, typically via a router or Layer 3 switch.

- **Broadcast Domain:** A network area where all devices receive every broadcast sent within that segment.
- **Issues with Broadcast Domain:** Causes unnecessary traffic, reduced performance, and security risks due to no isolation.
- **Tagging:** Adding an 802.1Q VLAN ID to frames on a trunk so the receiver knows the frame's VLAN.
- **VLAN Basics:** Logically separates a network into multiple broadcast domains for better segmentation and control.
- **VLAN 1:** The default VLAN on most switches, used for initial management and untagged traffic unless reconfigured.

1. **Add VLAN configuration for access ports and trunk ports on individual switches (Management, Technical, Sales)**

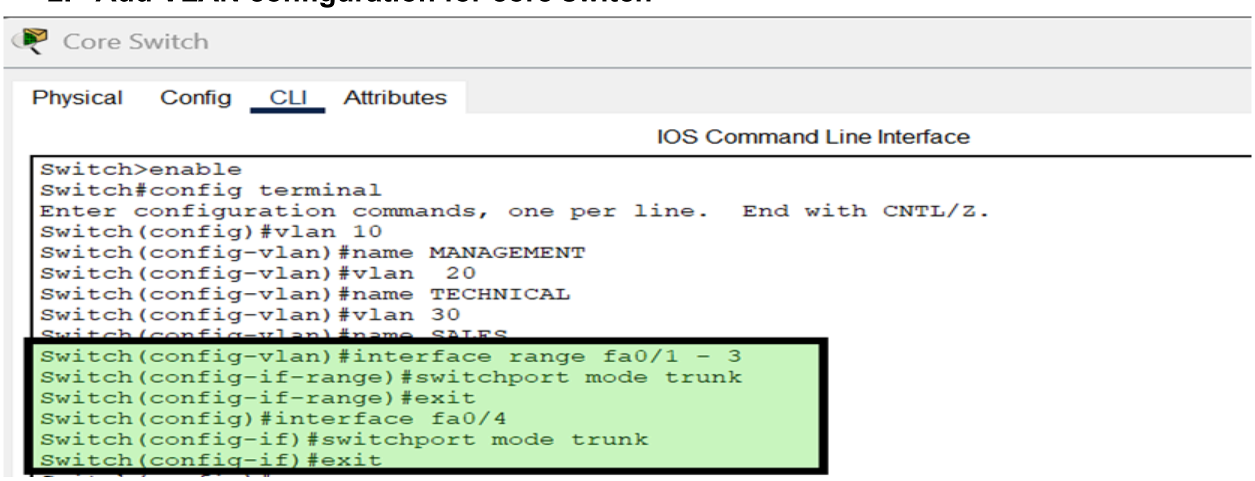






NOTE: PINK IS FOR ALL THE PCS, YELLOW IS FOR THE SERVERS, AND GREEN IS FOR THE TRUNK PORTS, YOU CAN DO PCS AND SERVERS TOGETHER IF THEY FALL IN A CONTINUOUS RANGE

2. **Add VLAN configuration for core switch**



3. **Add VLAN inter routing configuration on router for each VLAN**

Router0

Physical Config CLI Attributes

IOS Command Line Interface

```
Router>enable
Router#config terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#interface gig0/0/0.10
Router(config-subif)#encapsulation dot1Q 10
Router(config-subif)#ip address 192.168.10.1 255.255.255.0
Router(config-subif)#interface gig0/0/0.20
Router(config-subif)#encapsulation dot1Q 20
Router(config-subif)#ip address 192.168.20.1 255.255.255.0
Router(config-subif)#interface gig0/0/0.30
Router(config-subif)#encapsulation dot1Q 30
Router(config-subif)#ip address 192.168.30.1 255.255.255.0
Router(config-subif)#exit
```

4. **Enable router interface for no shutdown**

Router0

Physical Config CLI Attributes

IOS Command Line Interface

```
Router>enable
Router#config terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#interface gig0/0/0
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/0, changed state to up

%LINK-5-CHANGED: Interface GigabitEthernet0/0/0.10, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/0.10, changed state to up

%LINK-5-CHANGED: Interface GigabitEthernet0/0/0.20, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/0.20, changed state to up

%LINK-5-CHANGED: Interface GigabitEthernet0/0/0.30, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/0.30, changed state to up

Router(config-if)#exit
```

Routing Algorithms

1. Static Routing

- Manually configured routes.
- Works for directly or indirectly connected networks.
- No automatic path selection.

---

2. Dynamic Routing

Routers automatically learn paths using routing protocols.

A. Distance Vector Routing

- Examples: **RIP, EIGRP**
- Router knows **only neighbor information.**
- Metric: **Hop count (RIP)** or **composite metric (EIGRP).**
- Chooses **lowest hop count** path.
- If hop count ties → **load balancing** happens.
- Slower convergence; risk of routing loops.

B. Link-State Routing

- Example: **OSPF**
- Router builds a **complete network topology.**
- Metric: **Cost = Reference Bandwidth / Interface Bandwidth** (Reference = **100 Mbps** in classic OSPF).
- Fast convergence and highly scalable.

C. Path Vector Routing

- Example: **BGP**
- Maintains **AS-path information.**
- Used **between autonomous systems.**

Interior vs Exterior Routing Protocols

- **IGP:** Works inside a single AS (RIP, OSPF, EIGRP).
- **EGP:** Works between different ASes (BGP).

Why do we assign a router-id in BGP?
To uniquely identify the BGP router and provide a stable ID for route selection and peering.

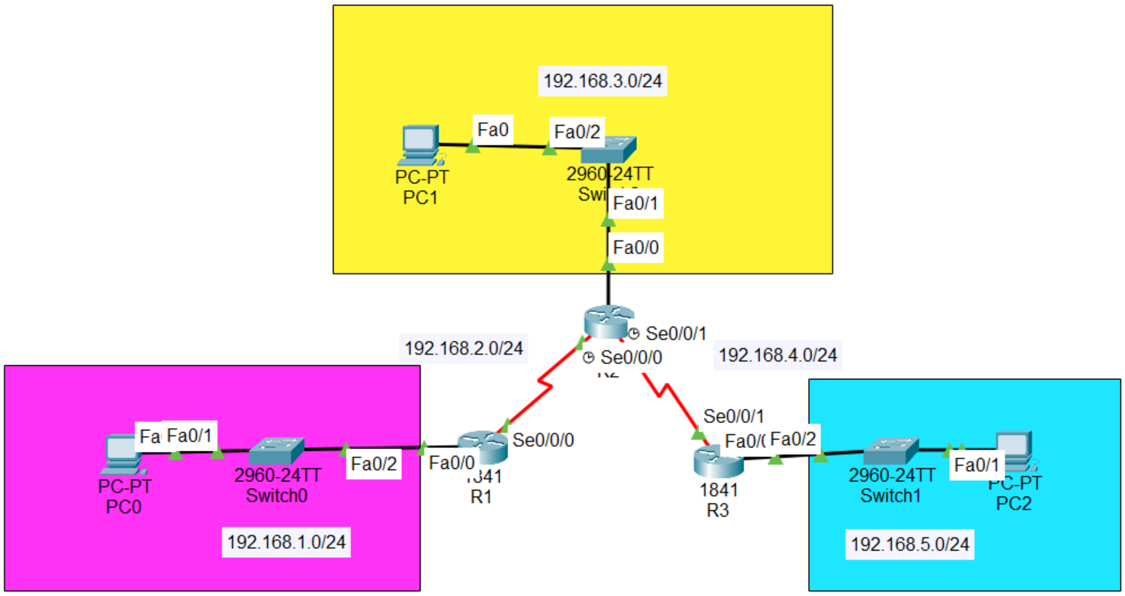
Why and what is Area 0 in OSPF?
Area 0 is the OSPF backbone, and all other areas must connect to it for inter-area communication.

Difference between RIP v1 and RIP v2:
RIP v1 is classful with no subnet info and uses broadcast updates, while RIP v2 is classless, carries subnet masks, uses multicast, and supports authentication.

Feature	RIP	OSPF	BGP
Routing Type	Distance-vector	Link-state	Path-vector
Metric	Hop count	Cost	AS-path + attributes
Max Hop Count	15	Unlimited	Unlimited
Classless?	v1 ❌ — v2 ✔	✔	✔
Convergence	Slow	Fast	Slow
Updates Sent	Entire table every 30s	LSAs when changes occur	Incremental updates
Algorithm	Bellman-Ford	Dijkstra (SPF)	Best-path selection
Scalability	Small networks	Medium to large networks	Very large / Internet
Transport Protocol	UDP 520	IP protocol 89	TCP port 179
Areas Supported	No	Yes (Area 0 backbone)	No areas; uses AS numbers
Router ID Needed?	No	Yes	Yes
Use Case	Simple LAN networks	Enterprise internal routing	Inter-AS Internet routing

RIP Configuration

Example Topology



1. **Configure routers as usual via CLI and then start with RIP. You connect rip with all the directly connected networks for that router.**

R1

Physical Config CLI Attributes

IOS Command Line Interface

```
Router>enable
Router#config terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#router rip
Router(config-router)#network 192.168.1.0
Router(config-router)#network 192.168.2.0
Router(config-router)#exit
Router(config)#
```

R2

Physical Config CLI Attributes

IOS Command Line Interface

```
Router>enable
Router#config terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#router rip
Router(config-router)#network 192.168.2.0
Router(config-router)#network 192.168.3.0
Router(config-router)#network 192.168.4.0
Router(config-router)#exit
Router(config)#
```

R3

Physical Config CLI Attributes

IOS Command Line Interface

```
Router>enable
Router#config terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#router rip
Router(config-router)#network 192.168.4.0
Router(config-router)#network 192.168.5.0
Router(config-router)#exit
Router(config)#
```

✅ RIP v1 Configuration

pgsql Copy code

```
enable
config terminal
router rip                        ! start RIP process
network <network-id>              ! advertise directly connected network
passive-interface serial0/0       ! stop sending updates on this interface
exit
show ip protocols                 ! show RIP summary
```

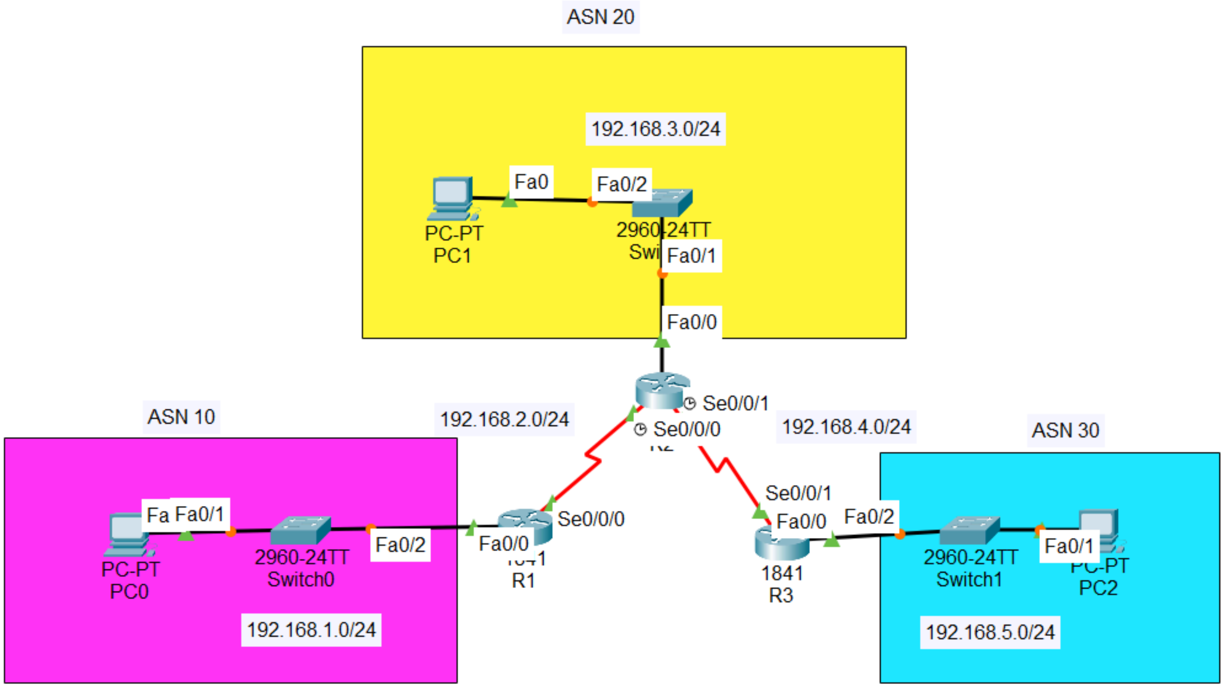
✅ RIP v2 Configuration

pgsql Copy code

```
enable
config terminal
router rip                        ! start RIP process
version 2                         ! enable RIPv2 (classless)
network <network-id>              ! advertise directly connected network
passive-interface serial0/0       ! block RIP updates on this interface
exit
show ip protocols                 ! show RIP summary
```

BGP Configuration

Example Topology



1. **Assign ASN numbers to your different networks. Then assign router ids to each BGP, neighbour ASNs using next hop ip, and assign directly connected networks as well.**

R1

Physical Config CLI Attributes

IOS Command Line Interface

```
Router>enable
Router#config terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#router bgp 10



Router(config-router)#bgp router-id 1.1.1.1



Router(config-router)#neighbor 192.168.2.2 remote-as 20
Router(config-router)#network 192.168.1.0 mask 255.255.255.0
Router(config-router)#network 192.168.2.0 mask 255.255.255.0
Router(config-router)#exit
Router(config)#
```

R2

Physical Config CLI Attributes

IOS Command Line Interface

```
Router>enable
Router#config terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#router bgp 20
Router(config-router)#bgp router-id 2.2.2.2
Router(config-router)#%BGP-5-ADJCHANGE: neighbor 192.168.4.1 Up

Router(config-router)#neighbor 192.168.2.1 remote-as 10
Router(config-router)#%BGP-5-ADJCHANGE: neighbor 192.168.2.1 Up




Router(config-router)#neighbor 192.168.4.1 remote-as 30
Router(config-router)#network 192.168.2.0 mask 255.255.255.0
Router(config-router)#network 192.168.4.0 mask 255.255.255.0
Router(config-router)#exit
Router(config)#
```

R3

Physical Config CLI Attributes

IOS Command Line Interface

```
Router>enable
Router#config terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#router bgp 30
Router(config-router)#bgp router-id 3.3.3.3
Router(config-router)#%BGP-5-ADJCHANGE: neighbor 192.168.4.2 Up




Router(config-router)#neighbor 192.168.4.2 remote-as 20
Router(config-router)#network 192.168.4.0 mask 255.255.255.0
Router(config-router)#network 192.168.5.0 mask 255.255.255.0
Router(config-router)#exit
```

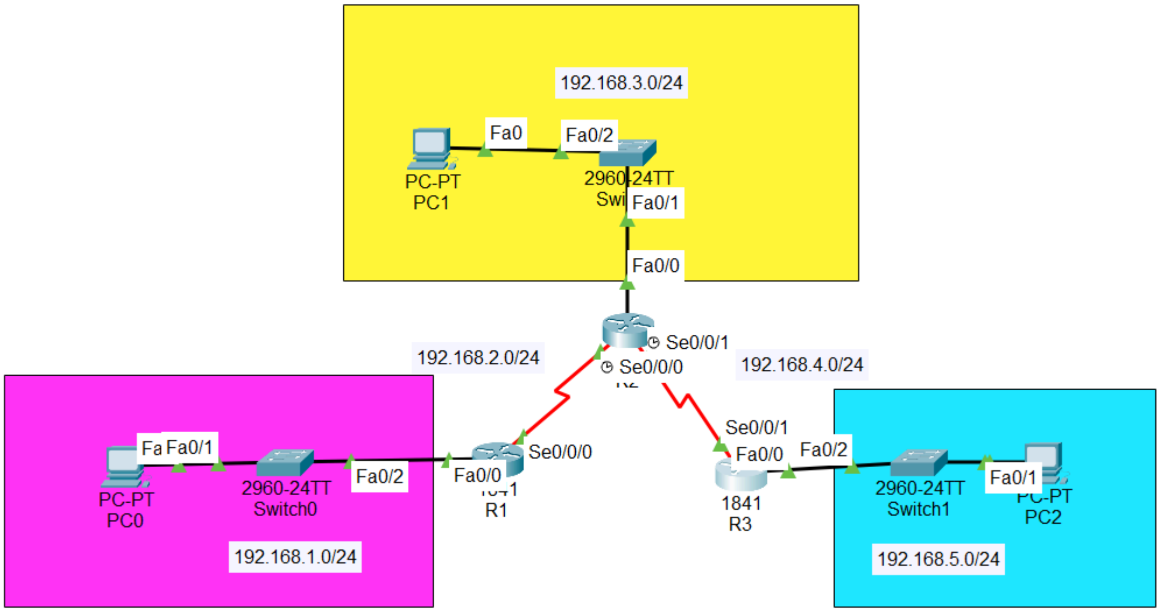
✅ BGP Configuration (with all show commands)

pgsql Copy code

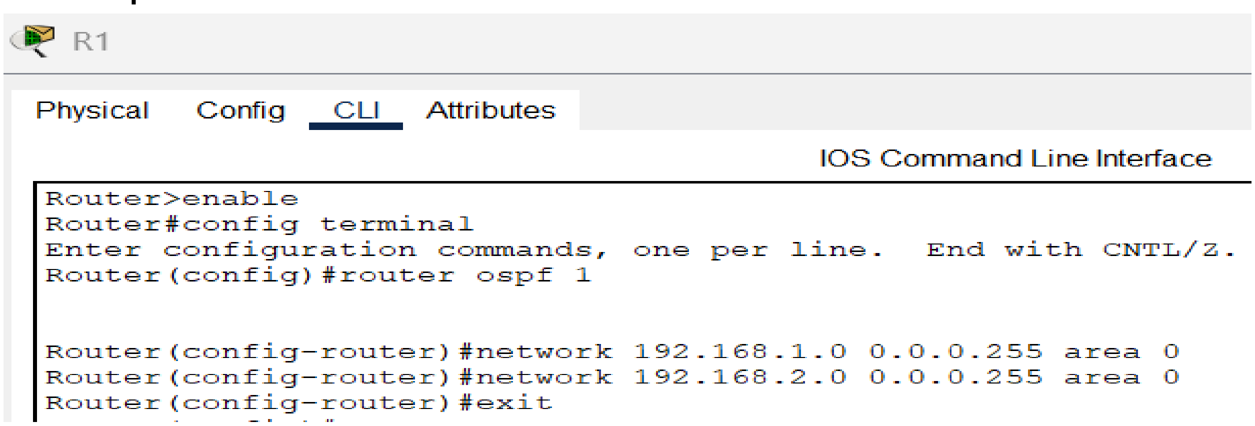
```pgsql
enable
config terminal
router bgp <as-number>                            ! start BGP process
bgp router-id <id>                                ! set stable router ID
neighbor <next-hop-ip> remote-as <as-number>      ! form BGP neighbor relationship
network <network-id> mask <subnet-mask>           ! advertise local network
exit
show ip bgp summary                                ! view BGP sessions status
show ip bgp neighbors                               ! detailed neighbor info
show ip route                                       ! check routing table
show ip protocols                                   ! verify routing protocols
```

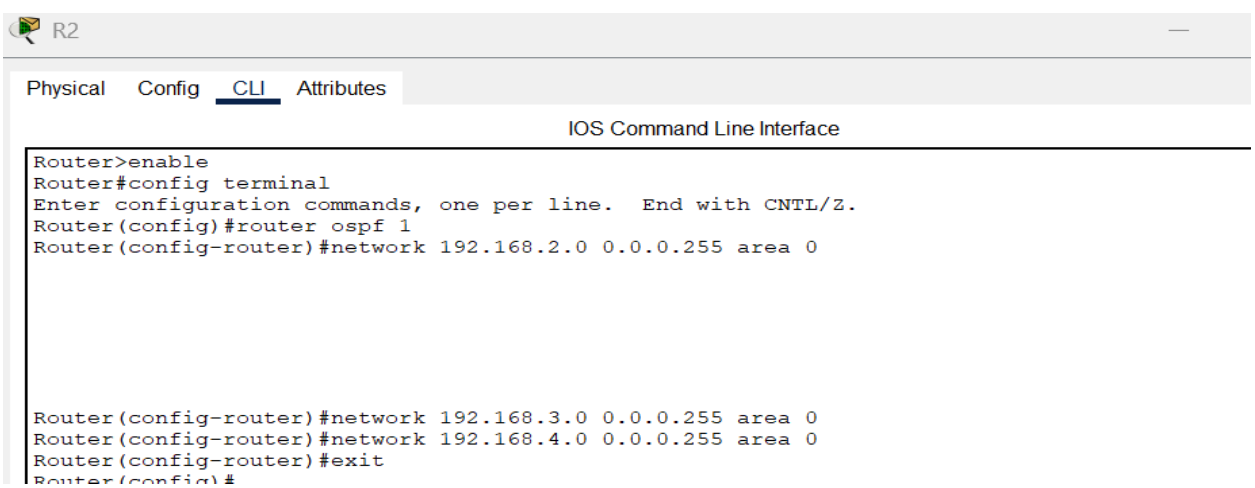
OSPF Configuration

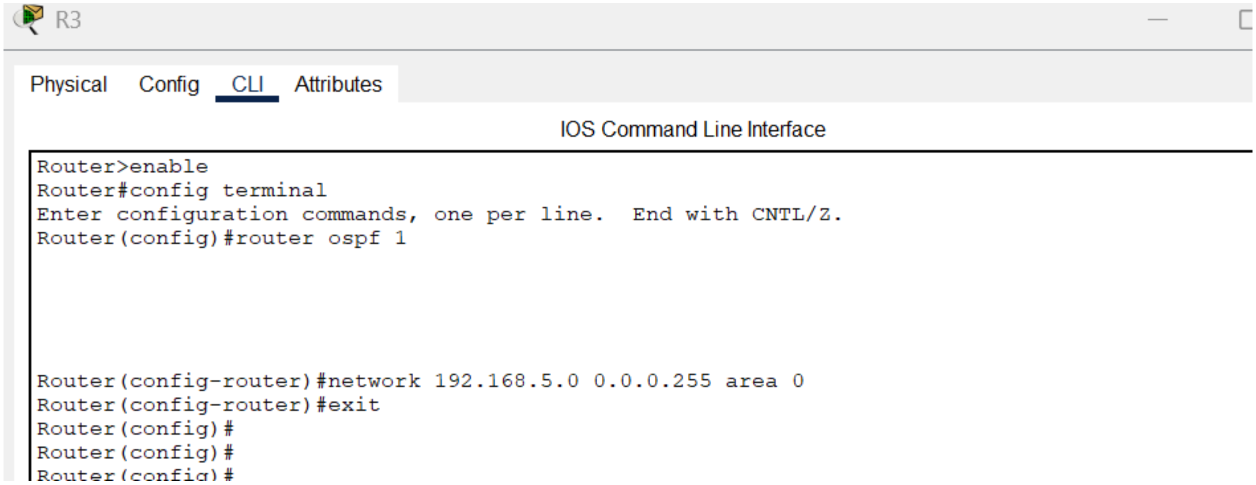
Example Topology



1. **Assign ospf number, and directly connected networks with wildcard to area 0 of ospf.**







✅ OSPF Configuration

pgsql

```
enable
config terminal
router ospf <process-id>                  ! start OSPF process
network <network-id> <wildcard> area 0    ! advertise network in area 0
exit
show ip ospf                               ! show OSPF summary
```

NS3 THEORY

✅ What is ns-3 (Network Simulator)?

ns-3 (Network Simulator 3) is an **open-source, discrete-event network simulator** used to **model, design, and analyze computer networks**.
It lets you **build virtual networks**, test **routing protocols** (RIP, OSPF, BGP, EIGRP, etc.), and evaluate the performance of **TCP, UDP, Wi-Fi, LTE, 5G, and congestion control**—all **without real hardware**.

ns-3 simulates **realistic network behaviour**, including delay, loss, congestion, throughput, and packet flow, allowing you to reproduce scenarios for **learning, debugging, research, and performance evaluation**.

✅ Why do we use ns-3?

We use **ns-3** because it provides a **realistic, flexible, and cost-free way to study and test computer networks without physical hardware**. It allows us to simulate network protocols, topology behaviour, congestion, delays, bandwidth limits, and packet loss in a controlled environment.

🔶 2. TCP Retransmission (How TCP Handles Loss)

A TCP segment is **retransmitted** when:

- The packet is **lost**
- The packet **never reached** the destination
- **No ACK** is received before **timeout**

Retransmissions affect TCP performance because they slow down the sending rate.

🔶 3. Advertised Window Size (Flow Control)

This is the **receiver window (rwnd)**.

- Receiver tells sender how much data it can accept
- Sender **must NOT exceed** this limit
- Prevents receiver buffer overflow

Flow Control Formula:

```sql
Actual window = min(rwnd, cwnd)
```

Where **cwnd** is the congestion window (explained next).

🔶 4. TCP Congestion Control — Key Components

Receiver Window (rwnd)

Flow control — controlled by receiver capacity.

Congestion Window (cwnd)

Congestion control — controlled by network condition.

Sending Window

```sql
Sending Window = min(rwnd, cwnd)
```

TCP Congestion Control Phases

1. Slow Start

- Starts with small window (usually 1 MSS).
- Increases **exponentially** each RTT.
- Continues until reaching **ssthresh** (slow start threshold).

2. Congestion Avoidance

- After reaching threshold, cwnd grows **linearly** (additive increase).

3. Congestion Detection

Triggered when:

- Packet loss occurs (timeout or duplicate ACKs)

Two responses:

a) Timeout:

- cwnd = 1
- ssthresh = cwnd/2
- Restart slow start

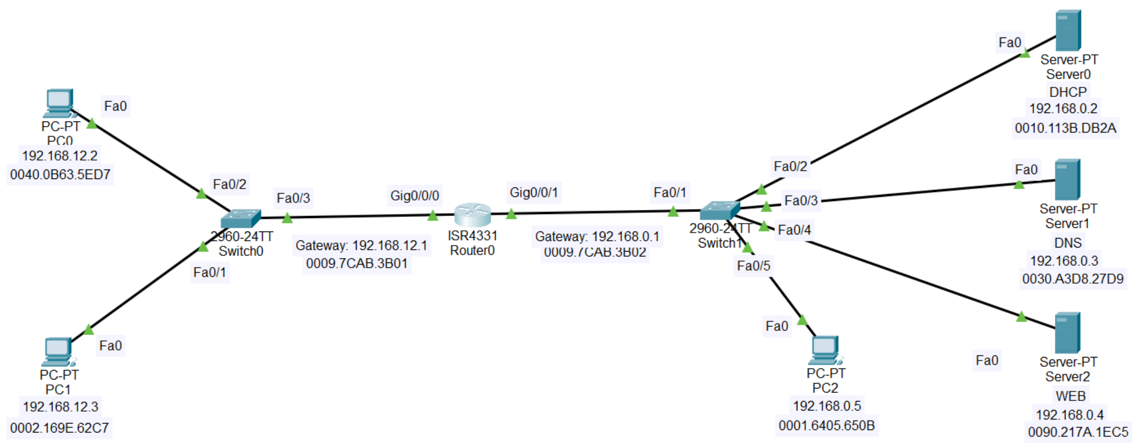
b) Fast Retransmit + Fast Recovery:

- On 3 duplicate ACKs
- cwnd = cwnd/2
- Enter congestion avoidance

- NS3 is a UNIX-based discrete-event network simulator supporting TCP, UDP, routing, and multicast, primarily scripted using Tcl.
- `vi` is a modal UNIX editor with command mode (manage text) and insert mode (edit text); press Esc to switch modes.
- Start `vi filename`, enter insert mode with `a` or `i`, edit text, and navigate using arrow keys.
- Save changes with `:w`, quit with `:q`, save and quit with `:wq`, or quit without saving with `:q!`.
- TCP congestion control simulation includes slow start (exponential increase), congestion avoidance (additive increase), and congestion detection (multiplicative decrease).

EXTENDED ACL

Example Topology



1. **Extended Numbered - Block SINGLE PC to a DESTINATION PORT.Block PC0 (192.168.12.2) from accessing WEB server (192.168.0.4) on HTTP port 80:**

```
Router>enable
Router#config terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#access-list 100 deny tcp 192.168.12.2 0.0.0.0 192.168.0.4 0.0.0.0 eq 80
Router(config)#access-list 100 permit ip any any
Router(config)#interface fa0/1
%Invalid interface type and number
Router(config)#interface Gig0/0/0
Router(config-if)#ip access-group 100 out
```

2. **Extended Numbered - Block an entire subnet from accessing HTTP. Block 192.168.12.0/24 from accessing 192.168.0.4:**

Router0

Physical Config CLI Attributes

IOS Command Line Interface

```
Router>enable
Router#config terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#access-list 101 deny tcp 192.168.12.0 0.0.0.255 192.168.0.4 0.0.0.0 eq 80



Router(config)#access-list 101 permit ip any any
Router(config)#interface gig0/0/0
Router(config-if)#ip access-group 101 out
```

3. **Extended Named - Block SINGLE PC to a DESTINATION PORT.Block PC0 (192.168.12.2) from accessing WEB server (192.168.0.4) on HTTP port 80:**

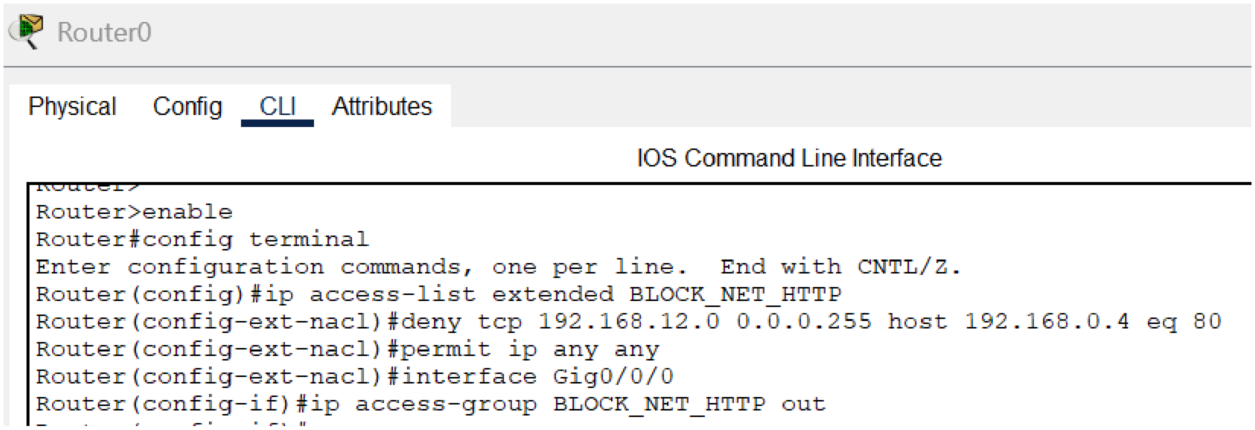
Router0

Physical Config CLI Attributes

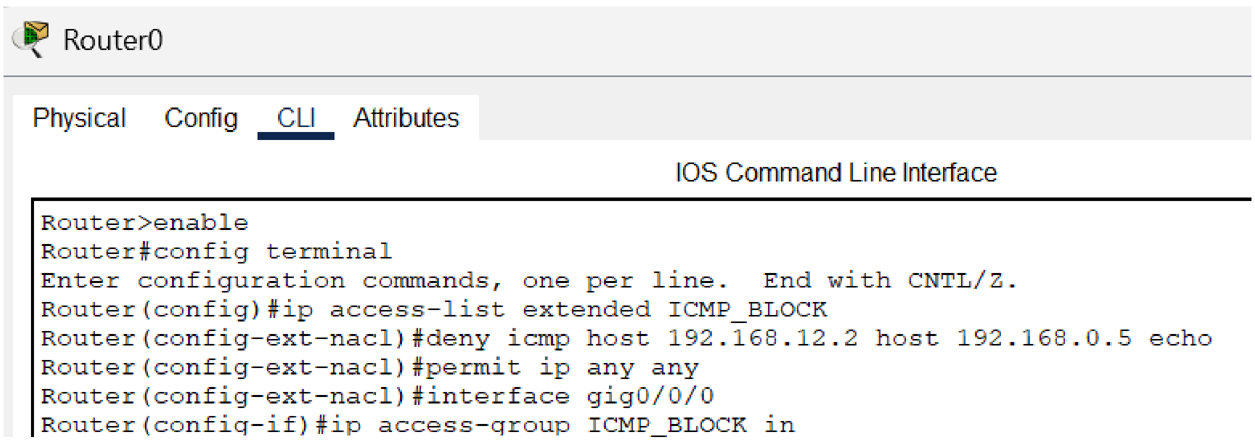
IOS Command Line Interface

```
Router>enable
Router#config terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#ip access-list extended BLOCK_PC0_HTTP
Router(config-ext-nacl)#deny tcp 192.168.12.2 0.0.0.0 192.168.0.4 0.0.0.0 eq 80
Router(config-ext-nacl)#permit ip any any
Router(config-ext-nacl)#interface Gig0/0/0
Router(config-if)#ip access-group BLOCK_PC0_HTTP out
```

4. **Extended Named - Block an entire subnet from accessing HTTP. Block 192.168.12.0/24 from accessing 192.168.0.4:**



5. **Extended Named - ICMP block, ping request**



6. **Extended Numbered - ICMP block, ping request (different topology)**

```
Router>enable
Router#config terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#access-list 103 deny icmp 192.168.30.3 0.0.0.0 192.168.20.0 0.0.0.255 echo
Router(config)#permit ip any any
                          ^
% Invalid input detected at '^' marker.

Router(config)#access-list 103 permit ip any any
Router(config)#interface gig0/1
Router(config-if)#ip access-group 103 out
```

HTTP block stops **TCP traffic on port 80**, while **ICMP block** stops **ping/echo requests**; both only affect the specified protocol, leaving other traffic untouched.

1. `0.0.0.0` in an ACL is used as a wildcard mask to specify a single IP.
2. `host <IP>` replaces `<IP> 0.0.0.0` to match a single IP more succinctly.
3. `permit ip any any` allows all IP traffic between all sources and destinations; `any any` appears twice for **source** and **destination**.
4. In `deny tcp ...`, writing `tcp` specifies the protocol so the ACL knows which traffic type to block. You can also use `eq <port>` or a protocol name (e.g., `telnet`, `http`) to filter specific services.
5. You must decide whether to apply the ACL to the **inbound** or **outbound** direction of an interface.

Standard	Extended
Range: 1–99 ✔ Filters only **source IP** ✔ Cannot filter destination or protocol ✔ Place **near destination**	Range: 100–199 ✔ Filters **source, destination, protocol, ports** ✔ Can block TCP/UDP/ICMP ✔ Place **near source**

5 Common Ports (must memorize)

Protocol	Port
HTTP	80
HTTPS	443
FTP	21
SMTP	25
Telnet	23
SSH	22

1 Can we apply more than one ACL?

- **Cisco IOS only allows ONE inbound and ONE outbound ACL per interface per protocol (IP).**
- You **cannot attach multiple IP ACLs inbound** on the same interface.
- Workarounds:
  - Combine rules into **one ACL**
  - Use **route-maps** or **VLAN segmentation**

---

2 How does the router process ACLs?

- **ACLs are processed in a specific direction**: inbound or outbound.
- **Rules are evaluated top-down:**
  1. Router checks first line → if match → applies `permit` or `deny` → stops
  2. If no match → goes to next line
  3. If no line matches → **implicit deny all** at the end
- Only **one ACL per direction** per protocol is chec

4️⃣ ❌ Can you create a Standard & Extended ACL with the SAME name?

Answer: No.

Reason:

- When applying the ACL:

```pgsql
ip access-group ACLNAME in/out
```

- Cisco cannot distinguish whether you meant a **standard** or **extended** ACL → **conflict occurs.**

Example showing the issue:

```pgsql
ip access-list standard test
 deny 192.168.1.1

ip access-list extended test
 deny tcp any any eq 80
```

When applying:

```pgsql
ip access-group test in
```

❌ **Result:** Router becomes confused → **NOT ALLOW**

Subnetting

Binary (4th Octect)	Decimal (4th Octect)	CIDR (for Public IP)	Number of subnets	Number of hosts per subnet	Valid subnets (4th Octect)	Broadcast Address for each subnet
10000000	128	/25	2	128 - 2	0, 128	127, 255
11000000	192	/26	4	64 - 2	0, 64, 128, 192	63, 127, 191, 255
11100000	224	/27	8	32 - 2	Calculate yourself	
11110000	240	/28	16	16 - 2		
11111000	248	/29	32	8 - 2		
11111100	252	/30	64	4 - 2		

Prefix	Mask	Block Size	Hosts
/25	128	128	126
/26	192	64	62
/27	224	32	30
/28	240	16	14
29	248	8	6
/30	252	4	2

Addresses for private network

Class	Net IDs	Blocks
A	10	1
B	172.16 ~ 172.31	16
C	192.168.0 ~ 192.168.255	256

Scenario 1: 10 hosts (12 IPs), then 60 hosts

Step 1: Determine subnet sizes

Hosts	Needed IPs	Subnet Mask	Block Size
10	12	/28	16
60	62	/26	64

> **Note:** Block size = next power of 2 ≥ needed IPs.

Step 2: Allocate subnets smallest-first

1. **10-host subnet (/28)**
   - Block size = 16 → must start at multiple of 16 → **192.168.1.0/28**
   - IP range: `192.168.1.1 – 192.168.1.14`
   - Broadcast: `192.168.1.15`
2. **60-host subnet (/26)**
   - Block size = 64 → must start at multiple of 64 → **192.168.1.64/26**
   - IP range: `192.168.1.65 – 192.168.1.126`
   - Broadcast: `192.168.1.127`

> **Important:** Maintaining the order creates a **gap** between `192.168.1.16 – 192.168.1.63`, which is **unused** due to alignment rules.

Scenario 2: 122 hosts, then 58 hosts

Step 1: Determine subnet sizes

Hosts	Needed IPs	Subnet Mask	Block Size
122	126	/25	128
58	62	/26	64

Step 2: Allocate subnets smallest-first

1. **122-host subnet (/25)**
   - Network: `192.168.1.0/25`
   - IP range: `192.168.1.1 – 192.168.1.126`
   - Broadcast: `192.168.1.127`
2. **58-host subnet (/26)**
   - Block size = 64 → next multiple-of-64 = `192.168.1.128/26`
   - IP range: `192.168.1.129 – 192.168.1.190`
   - Broadcast: `192.168.1.191`

> Here, **no gaps** occur because the /25 ends exactly at a multiple-of-64 boundary.

- Each subnet has a **block size**, which is calculated as `256 - subnet mask`.
- You must start each subnet at a multiple of its block size **counting from 0**.

Example:

- Block size = 16 → valid subnet starts: `0, 16, 32, 48…`
- **Invalid:** starting at 40, because it's not a multiple of 16.

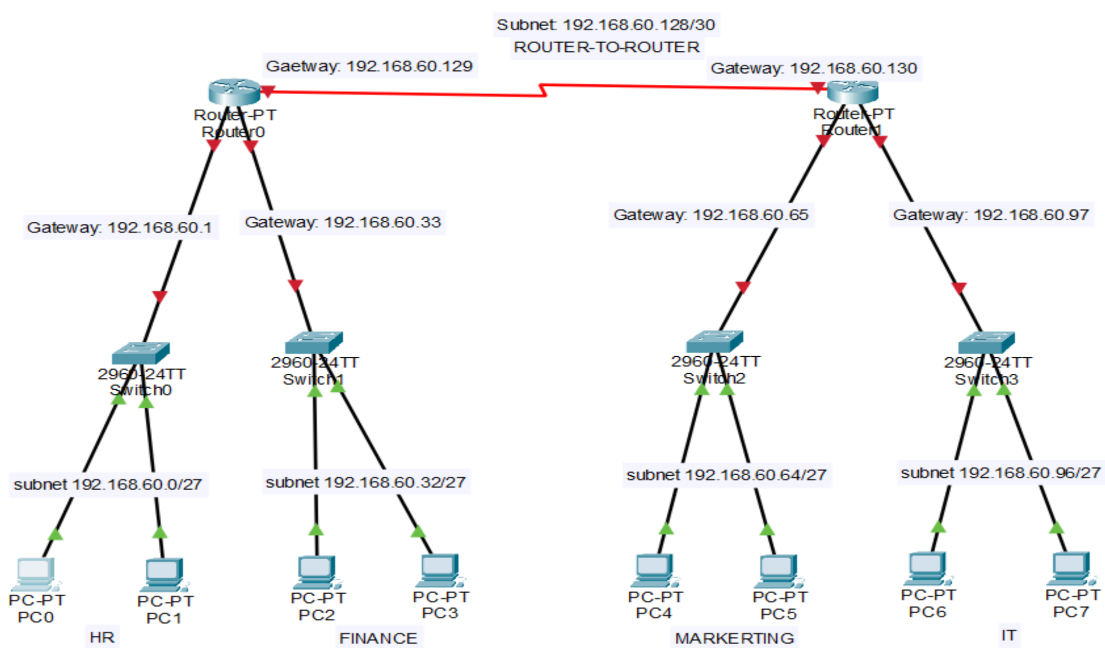
✅ **Why:**

- IP addresses are grouped in continuous blocks. Starting at the wrong number will overlap with another subnet and cause conflicts.

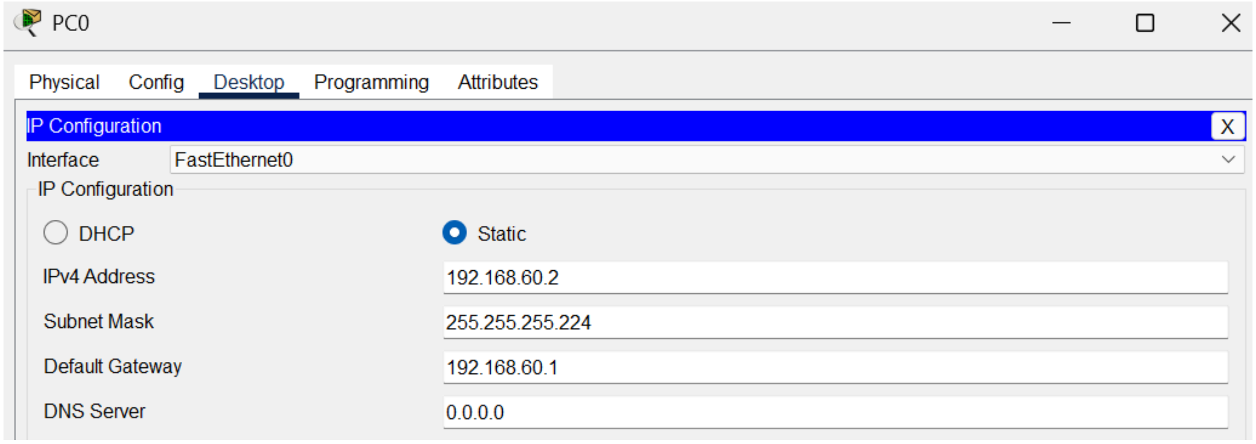
In short: Always start each subnet at a multiple of its block size to avoid overlapping.

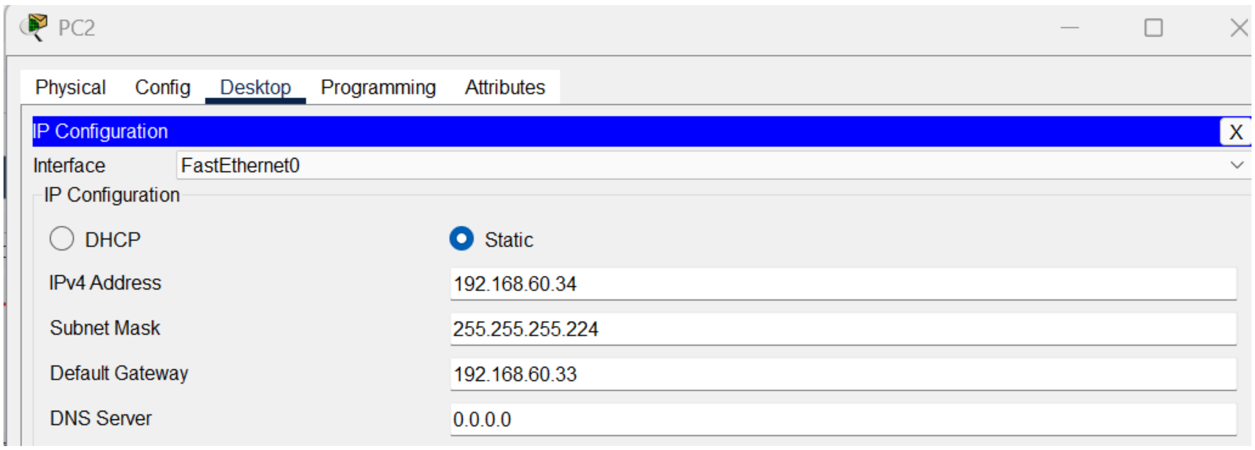
Calculation	Formula / Explanation
Block Size	`256 - Subnet Mask (in last octet)` Example: /27 → 256 − 224 = 32
Number of Subnets	`2^n` Where `n` = number of bits borrowed for subnetting
Number of Hosts per Subnet	`2^h - 2` Where `h` = number of host bits remaining

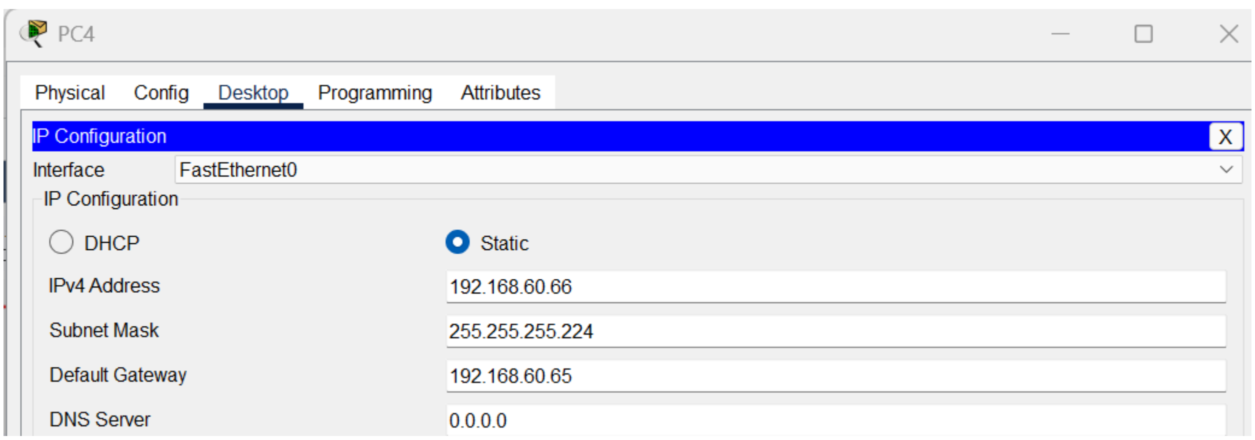
Example Topology

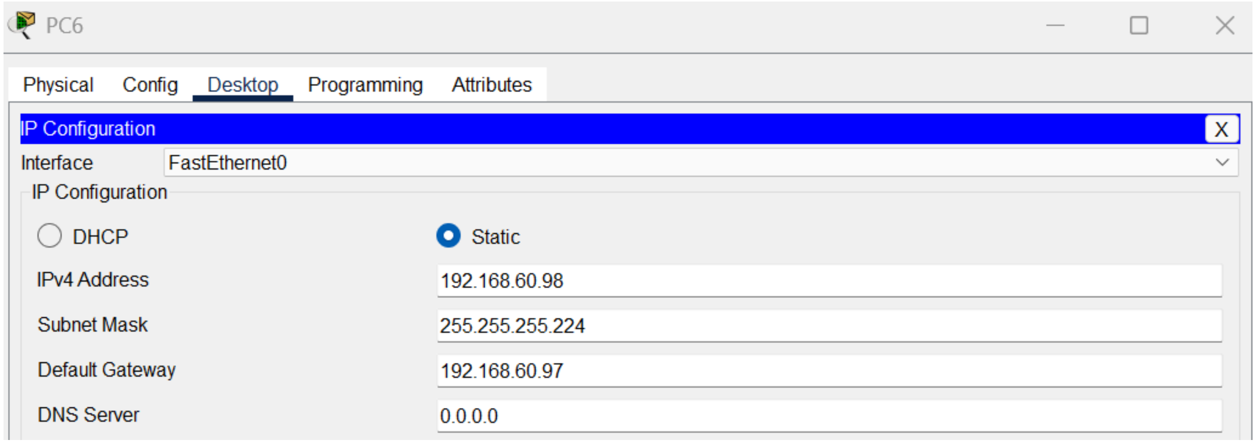


1. **Configure subnets using subnet masks while assigning IPs**


PC0
Physical
Config
Desktop
Programming
Attributes
IP Configuration
X
Interface
FastEthernet0
IP Configuration
DHCP
Static
IPv4 Address
192.168.60.2
Subnet Mask
255.255.255.224
Default Gateway
192.168.60.1
DNS Server
0.0.0.0


PC2
Physical
Config
Desktop
Programming
Attributes
IP Configuration
X
Interface
FastEthernet0
IP Configuration
DHCP
Static
IPv4 Address
192.168.60.34
Subnet Mask
255.255.255.224
Default Gateway
192.168.60.33
DNS Server
0.0.0.0


PC4
Physical
Config
Desktop
Programming
Attributes
IP Configuration
X
Interface
FastEthernet0
IP Configuration
DHCP
Static
IPv4 Address
192.168.60.66
Subnet Mask
255.255.255.224
Default Gateway
192.168.60.65
DNS Server
0.0.0.0



2. **Configure routers as usual, for static routing you will use subnet masks for each subnet created**